

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA KỸ THUẬT ĐIỆN TỬ 1



BÁO CÁO CUỐI KỲ
CẢM BIẾN VÀ CƠ CẤU CHẤP HÀNH
MÔ PHỎNG HỆ THỐNG ROBOT TỰ HÀNH
TURTLEBOT4

Giảng viên hướng dẫn: TS. Phạm Hoàng Anh

Sinh viên thực hiện:
Nguyễn Hữu Hoàng Anh – B23DCDK007
Phạm Tuấn Anh – B23DCDK012
Nguyễn Hoàng Anh – B23DCDK006
Nguyễn Xuân Nhật – B23DCDK094

Hà Nội, ngày 04 tháng 06 năm 2026

LỜI CẢM ƠN

Chúng em xin gửi lời cảm ơn chân thành đến Thầy **TS. Phạm Hoàng Anh**, giảng viên hướng dẫn học phần *Cảm biến và Cơ cấu chấp hành*, đã tận tình giảng dạy, định hướng và truyền đạt các kiến thức nền tảng về cảm biến, cơ cấu chấp hành, hệ thống đo lường, điều khiển chuyển động và ứng dụng của chúng trong các hệ thống robot hiện đại.

Học phần đã giúp chúng em hiểu rõ hơn vai trò của cảm biến trong việc thu nhận thông tin từ môi trường, vai trò của cơ cấu chấp hành trong biến đổi tín hiệu điều khiển thành chuyển động vật lý, cũng như mối liên hệ chặt chẽ giữa nhận thức, định vị, lập kế hoạch và điều khiển trong một hệ robot tự hành. Những kiến thức này là cơ sở quan trọng để chúng em triển khai đề tài nghiên cứu hệ thống robot tự hành TurtleBot4 trong môi trường mô phỏng sử dụng ROS2, SLAM, Nav2 và YOLOv8.

Chúng em cũng xin cảm ơn Khoa Kỹ thuật Điện tử 1, Học viện Công nghệ Bưu chính Viễn thông đã tạo điều kiện học tập, nghiên cứu và thực hành trong quá trình thực hiện báo cáo. Mặc dù đã cố gắng hoàn thiện báo cáo một cách nghiêm túc, bài làm vẫn khó tránh khỏi thiếu sót do giới hạn về thời gian và kinh nghiệm triển khai thực nghiệm. Chúng em kính mong nhận được góp ý của thầy để tiếp tục hoàn thiện kiến thức và năng lực nghiên cứu trong lĩnh vực robotics.

Hà Nội, ngày 04 tháng 06 năm 2026

Nhóm sinh viên thực hiện

LỜI MỞ ĐẦU

Robot tự hành đang trở thành một hướng phát triển quan trọng của kỹ thuật robot hiện đại, đặc biệt trong các bài toán kho vận thông minh, giám sát, tuần tra, dịch vụ và hỗ trợ con người. Một robot tự hành không chỉ cần cơ cấu chuyển động ổn định mà còn cần hệ cảm biến đủ tin cậy để nhận biết môi trường, định vị bản thân, phát hiện đối tượng và ra quyết định hành động. Vì vậy, việc nghiên cứu robot tự hành là một ví dụ điển hình cho sự kết hợp giữa cảm biến, cơ cấu chấp hành và thuật toán điều khiển.

Báo cáo này trình bày quá trình xây dựng hệ thống robot tự hành TurtleBot4 trong môi trường mô phỏng. Hệ thống sử dụng ROS2 làm nền tảng tích hợp phần mềm, Gazebo/Ignition làm môi trường mô phỏng, LiDAR và camera RGB-D làm nguồn dữ liệu cảm biến, SLAM Toolbox để xây dựng bản đồ, Nav2 để điều hướng, YOLOv8 để nhận diện mục tiêu và một Mission Manager để điều phối hành vi tuần tra, phát hiện và tiếp cận mục tiêu.

Dưới góc nhìn của học phần *Cảm biến và Cơ cấu chấp hành*, trọng tâm báo cáo không chỉ là phần mềm robot mà còn là phân tích mối quan hệ hệ thống: cảm biến cung cấp dữ liệu đo, thuật toán xử lý biến dữ liệu đo thành thông tin trạng thái, bộ lập kế hoạch tạo lệnh chuyển động, và cơ cấu chấp hành của robot thực hiện lệnh để tạo chuyển động trong môi trường. Cách tiếp cận này giúp đánh giá hệ robot như một hệ cơ điện tử hoàn chỉnh thay vì chỉ là một tập hợp các module rời rạc.

Nội dung báo cáo gồm tám chương: tổng quan đề tài; cơ sở lý thuyết; thiết kế kiến trúc hệ thống; hệ thống cảm biến; cơ cấu chấp hành và điều khiển chuyển động; triển khai nhiệm vụ tự hành; kết quả mô phỏng và đánh giá; kết luận và hướng phát triển.

MỤC LỤC

LỜI CẢM ƠN	i
LỜI MỞ ĐẦU	ii
1 TỔNG QUAN ĐỀ TÀI	1
1.1 Bối cảnh nghiên cứu	1
1.2 Lý do chọn đề tài	1
1.3 Bài toán nghiên cứu	3
1.4 Mục tiêu của đề tài	3
1.5 Yêu cầu kỹ thuật của hệ thống	4
1.6 Kiến trúc tổng quát của hệ thống	5
1.7 Phạm vi nghiên cứu	6
1.8 Phương pháp thực hiện	7
1.9 Đóng góp chính của báo cáo	8
1.10 Kết quả kỳ vọng	8
2 CƠ SỞ LÝ THUYẾT	9
2.1 Robot tự hành và vòng kín cảm biến - xử lý - điều khiển - chấp hành . .	9
2.2 LiDAR trong robot di động	10
2.2.1 Định nghĩa và chức năng	10
2.2.2 Cấu tạo và chức năng từng thành phần	11
2.2.3 Cách hoạt động và tính toán dữ liệu	11
2.2.4 Input, output và ứng dụng trong project	12
2.3 Camera RGB-D OAK-D	13
2.3.1 Định nghĩa và chức năng	13
2.3.2 Thành phần và chức năng	13
2.3.3 Cách hoạt động và tính toán dữ liệu	14
2.3.4 Input, output và ứng dụng trong project	15

2.4	Odometry và encoder	15
2.4.1	Định nghĩa và chức năng	15
2.4.2	Thành phần và chức năng	16
2.4.3	Cách tính toán dữ liệu odometry	16
2.4.4	Input, output và ứng dụng trong project	16
2.5	Hệ tọa độ và TF/TF2 trong ROS2	17
2.5.1	Định nghĩa và chức năng	17
2.5.2	Công thức biến đổi tọa độ	18
2.5.3	Input, output và ứng dụng trong project	18
2.6	SLAM	18
2.6.1	Định nghĩa và chức năng	18
2.6.2	Nguyên lý tính toán trong SLAM	19
2.6.3	Input, output và ứng dụng trong project	20
2.7	Nav2 và điều hướng robot	20
2.7.1	Định nghĩa và chức năng	20
2.7.2	Cách hoạt động của Nav2	20
2.7.3	Input, output và ứng dụng trong project	21
2.8	YOLOv8 trong nhận diện đối tượng	21
2.8.1	Định nghĩa và chức năng	21
2.8.2	Dữ liệu đầu ra của YOLOv8	22
2.8.3	Input, output và ứng dụng trong project	22
2.9	Định vị mục tiêu từ RGB-D	23
2.9.1	Vai trò của định vị mục tiêu	23
2.9.2	Quy trình tính toán	23
2.9.3	Input, output và ứng dụng trong project	24
2.10	Cơ cấu chấp hành robot vi sai	24
2.10.1	Định nghĩa và chức năng	24
2.10.2	Mô hình động học	25
2.10.3	Ý nghĩa các trường hợp chuyển động	26

2.11	Mission Manager và thuật toán tiếp cận an toàn	26
2.11.1	Chức năng của Mission Manager	26
2.11.2	Công thức tính safe goal	27
2.11.3	Input, output và ứng dụng trong project	28
2.12	Luồng dữ liệu tổng hợp trong project TurtleBot4	29
2.13	Kết luận chương	30
3	THIẾT KẾ KIẾN TRÚC HỆ THỐNG	31
3.1	Tổng quan kiến trúc hệ thống	31
3.2	Các package chính trong hệ thống	32
3.2.1	Package tb4_bringup	32
3.2.2	Package tb4_vision_oak	33
3.2.3	Package tb4_object_localization	34
3.2.4	Package tb4_nav_patrol	35
3.2.5	Package tb4_mission_manager	36
3.3	Luồng dữ liệu chính của hệ thống	38
3.4	Sơ đồ khối kiến trúc hệ thống	39
3.5	Môi trường mô phỏng	40
3.6	Cấu hình launch tổng hợp	40
3.7	Đánh giá kiến trúc hệ thống	42
3.8	Kết luận chương	43
4	HỆ THỐNG CẢM BIẾN VÀ XỬ LÝ DỮ LIỆU	44
4.1	Vai trò của cảm biến trong đề tài	44
4.2	LiDAR và dữ liệu /scan	45
4.2.1	Chức năng của LiDAR trong hệ thống	45
4.2.2	Cách hoạt động và cách tính dữ liệu LiDAR	46
4.2.3	Kiểm tra và lỗi thường gặp của dữ liệu /scan	47
4.3	Camera RGB-D OAK-D	48
4.3.1	Chức năng của camera RGB-D trong project	48

4.3.2	Thành phần dữ liệu của camera RGB-D	49
4.3.3	Cách tính tọa độ 3D từ ảnh RGB-D	49
4.4	Nhận diện đối tượng bằng YOLOv8n	50
4.4.1	Vai trò của YOLOv8n	50
4.4.2	Dữ liệu detection và tiêu chí chọn mục tiêu	51
4.5	Định vị mục tiêu từ RGB-D	52
4.5.1	Quy trình định vị	52
4.5.2	Lọc depth tại vùng bbox	52
4.5.3	Input, output của node object_localization_node	54
4.6	TF và hợp nhất thông tin cảm biến	54
4.6.1	Vai trò của TF/TF2	54
4.6.2	Công thức biến đổi pose giữa các frame	55
4.7	Đánh giá kỹ thuật hệ cảm biến	56
4.8	Luồng dữ liệu cảm biến trong project	56
4.9	Gợi ý kiểm tra khi chạy hệ thống	57
4.10	Kết luận chương	58
5	CƠ CẤU CHẤP HÀNH VÀ ĐIỀU KHIỂN CHUYỂN ĐỘNG	59
5.1	Cấu trúc chấp hành của TurtleBot4	59
5.1.1	Khái niệm cơ cấu chấp hành trong robot tự hành	59
5.1.2	Thành phần và chức năng	60
5.1.3	Input và output của lớp chấp hành	60
5.2	Mô hình động học robot vi sai	61
5.2.1	Trạng thái và giả thiết chuyển động	61
5.2.2	Phương trình động học thuận	61
5.2.3	Quan hệ giữa vận tốc robot và vận tốc bánh xe	62
5.3	Lệnh vận tốc /cmd_vel và quá trình thực thi chuyển động	63
5.4	Nav2 như lớp điều khiển chuyển động cấp cao	64
5.4.1	Vai trò của Nav2	64
5.4.2	Thành phần chính của Nav2 trong project	65

5.4.3	Input và output của Nav2	65
5.5	Patrol Node	66
5.5.1	Chức năng	66
5.5.2	Thuật toán tuần tra	66
5.6	Mission Manager và điều khiển hành vi	67
5.6.1	Vai trò của Mission Manager	67
5.6.2	Công thức tính điểm tiếp cận an toàn	68
5.7	Quan hệ giữa cảm biến và cơ cấu chấp hành	69
5.8	Luồng dữ liệu chính trong Chương 5	70
5.9	Đánh giá kỹ thuật lớp chấp hành và điều khiển	70
5.10	Kết luận chương	71
6	TRIỂN KHAI NHIỆM VỤ TỰ HÀNH	72
6.1	Mục tiêu nhiệm vụ	72
6.1.1	Yêu cầu chức năng của nhiệm vụ	73
6.1.2	Input và output của toàn nhiệm vụ	73
6.2	Quy trình khởi động hệ thống	74
6.2.1	Bước 1: Khởi động mô phỏng TurtleBot4	74
6.2.2	Bước 2: Chạy full mission	74
6.2.3	Input, output và điểm cần kiểm tra khi khởi động	75
6.3	SLAM và bản đồ môi trường	76
6.3.1	Dữ liệu vào và dữ liệu ra của quá trình SLAM	77
6.3.2	Cách quy đổi vị trí thực sang ô bản đồ	77
6.3.3	Các tham số SLAM quan trọng	78
6.4	Navigation và tuần tra	78
6.4.1	Mô hình waypoint tuần tra	79
6.4.2	Điều kiện hoàn thành waypoint	79
6.4.3	Trạng thái của Patrol Node	80
6.5	Nhận diện và định vị mục tiêu	80
6.5.1	Luồng xử lý nhận diện	81

6.5.2	Công thức tính tọa độ 3D từ RGB-D	81
6.6	Mission Manager	82
6.6.1	Trạng thái chính của Mission Manager	83
6.6.2	Tính điểm tiếp cận an toàn	83
6.6.3	Input và output của Mission Manager	85
6.7	Các topic kiểm tra quan trọng	85
6.7.1	Cách đọc lỗi theo luồng dữ liệu	86
6.8	Sơ đồ hoạt động tổng hợp của nhiệm vụ	87
6.9	Đánh giá quy trình triển khai	88
6.10	Kết luận chương	89
7	KẾT QUẢ MÔ PHỎNG VÀ ĐÁNH GIÁ	90
7.1	Mục đích và phạm vi đánh giá	90
7.2	Kết quả đạt được	91
7.3	Phương pháp đánh giá và chỉ tiêu định lượng đề xuất	92
7.4	Đánh giá hệ cảm biến	94
7.4.1	Đánh giá LiDAR	95
7.4.2	Đánh giá camera RGB-D	95
7.4.3	Đánh giá odometry và TF	95
7.5	Đánh giá cơ cấu chấp hành và điều khiển	97
7.6	Đánh giá nhận diện và định vị mục tiêu	99
7.7	Đánh giá kiến trúc phần mềm	100
7.8	Hạn chế của hệ thống	101
7.9	Hướng phát triển	102
7.10	Kết luận chương	103
8	KẾT LUẬN	105
8.1	Kết luận chung	105
8.2	Các nội dung đã hoàn thành	106
8.3	Ý nghĩa đối với học phần Cảm biến và Cơ cấu chấp hành	107

8.4	Hạn chế của đề tài	109
8.5	Định hướng phát triển tiếp theo	109
8.6	Tổng hợp giá trị đạt được của đề tài	110
8.7	Kết luận cuối cùng	111
PHỤ LỤC A: CẤU TRÚC SOURCE CODE		112
PHỤ LỤC B: CÁC LỆNH CHẠY CHÍNH		113
PHỤ LỤC C: MỘT SỐ LỖI VÀ CÁCH KHẮC PHỤC		115
TÀI LIỆU THAM KHẢO		116

1. TỔNG QUAN ĐỀ TÀI

1.1 Bối cảnh nghiên cứu

Trong những năm gần đây, robot tự hành đã được ứng dụng ngày càng rộng rãi trong nhiều lĩnh vực như kho vận, sản xuất thông minh, giám sát an ninh, dịch vụ trong nhà, chăm sóc y tế và nghiên cứu giáo dục. Khác với các cơ cấu tự động truyền thống chỉ thực hiện một chuỗi thao tác được lập trình trước, robot tự hành cần có khả năng thu nhận thông tin từ môi trường, xác định trạng thái hiện tại, lập kế hoạch di chuyển và tự điều chỉnh hành vi khi điều kiện hoạt động thay đổi.

Một hệ robot di động tự hành hoàn chỉnh thường bao gồm ba lớp chức năng chính:

- **Lớp cảm biến:** thu nhận thông tin từ môi trường và trạng thái nội tại của robot thông qua các thiết bị như LiDAR, camera, encoder bánh xe và cảm biến quán tính IMU.
- **Lớp xử lý và ra quyết định:** thực hiện các chức năng định vị, lập bản đồ, nhận diện đối tượng, lập kế hoạch đường đi, tránh vật cản và quản lý nhiệm vụ.
- **Lớp cơ cấu chấp hành:** chuyển đổi lệnh điều khiển thành chuyển động thực tế thông qua động cơ điện, bộ truyền động và hệ thống bánh xe.

Ba lớp chức năng này liên kết với nhau theo một vòng kín. Cảm biến cung cấp dữ liệu phản hồi về môi trường và trạng thái robot. Khối xử lý sử dụng dữ liệu này để xác định hành động phù hợp. Sau đó, cơ cấu chấp hành tạo ra chuyển động mới, làm thay đổi trạng thái của robot và tạo ra dữ liệu phản hồi cho chu kỳ xử lý tiếp theo.

Trong học phần *Cảm biến và Cơ cấu chấp hành*, việc khảo sát một hệ robot tự hành có ý nghĩa thực tiễn cao vì bài toán thể hiện đầy đủ mối quan hệ giữa cảm biến, xử lý tín hiệu, thuật toán điều khiển và cơ cấu chấp hành. Thông qua mô hình robot TurtleBot4, dữ liệu từ LiDAR, camera, encoder, odometry và hệ thống biến đổi tọa độ TF được khai thác để phục vụ các nhiệm vụ lập bản đồ, điều hướng và tiếp cận đối tượng mục tiêu.

1.2 Lý do chọn đề tài

Đề tài lựa chọn nền tảng TurtleBot4 Lite vì đây là một robot di động phù hợp cho hoạt động đào tạo và nghiên cứu robotics trên hệ sinh thái ROS2. Robot sử dụng cơ cấu truyền động vi sai, gồm hai bánh chủ động được điều khiển độc lập. Cấu trúc này tương đối đơn giản về mặt cơ khí nhưng vẫn cho phép nghiên cứu đầy đủ các bài toán quan

trọng của robot tự hành như điều khiển vận tốc, định vị, lập bản đồ và điều hướng trong môi trường có vật cản.

Cấu trúc tổng thể của nền tảng TurtleBot4 Lite được minh họa trong Hình 1.1. Đây là phiên bản rút gọn của TurtleBot4, nhưng vẫn tích hợp các thành phần cần thiết để triển khai các bài toán lập bản đồ, điều hướng và xử lý dữ liệu cảm biến.



Hình 1.1 Nền tảng robot TurtleBot4 Lite sử dụng trong đề tài

Nguồn: TurtleBot 4 User Manual

TurtleBot4 Lite hỗ trợ các thành phần cảm biến và thông tin trạng thái quan trọng:

- LiDAR hai chiều phục vụ phát hiện vật cản và xây dựng bản đồ môi trường.
- Camera RGB hoặc camera RGB-D phục vụ quan sát và nhận diện đối tượng.
- Encoder bánh xe phục vụ ước lượng quãng đường và vận tốc chuyển động.
- IMU phục vụ đo vận tốc góc và hỗ trợ ước lượng trạng thái.
- Odometry biểu diễn vị trí và hướng tương đối của robot theo thời gian.
- Hệ thống TF quản lý quan hệ hình học giữa các hệ tọa độ của robot, cảm biến, bản đồ và mục tiêu.

Bên cạnh đó, TurtleBot4 được hỗ trợ bởi các công cụ phổ biến trong ROS2 như SLAM Toolbox, Nav2, RViz2 và Gazebo/Ignition. Điều này cho phép xây dựng một quy trình phát triển theo từng bước: kiểm thử thuật toán trong mô phỏng, đánh giá tính ổn định của hệ thống và chuẩn bị cho quá trình triển khai trên phần cứng thật.

Việc sử dụng môi trường mô phỏng Gazebo/Ignition giúp giảm chi phí thử nghiệm, hạn chế rủi ro va chạm cơ khí và tạo điều kiện quan sát trực quan dữ liệu cảm biến. Ngoài

ra, mô phỏng cho phép thay đổi môi trường, vị trí vật cản và mục tiêu một cách linh hoạt mà không cần thiết lập lại không gian thử nghiệm thực tế.

1.3 Bài toán nghiên cứu

Bài toán đặt ra là xây dựng một hệ thống robot TurtleBot4 Lite có khả năng thực hiện nhiệm vụ tuần tra trong môi trường mô phỏng, phát hiện một đối tượng mục tiêu từ dữ liệu camera và chủ động di chuyển đến vị trí phù hợp gần đối tượng.

Trong quá trình hoạt động, robot cần thực hiện tuần tự các chức năng sau:

1. Thu nhận dữ liệu từ LiDAR, camera, odometry và TF.
2. Xây dựng hoặc sử dụng bản đồ môi trường đã được thiết lập trước.
3. Xác định vị trí hiện tại của robot trên bản đồ.
4. Di chuyển theo các điểm tuần tra được cấu hình trước.
5. Nhận diện đối tượng mục tiêu từ ảnh RGB bằng mô hình YOLOv8n.
6. Ước lượng vị trí tương đối của mục tiêu so với camera và robot.
7. Chuyển đổi vị trí mục tiêu sang hệ tọa độ bản đồ thông qua TF.
8. Tạo điểm đích phù hợp để robot tiếp cận mục tiêu mà không gây va chạm.
9. Gửi điểm đích đến Nav2 và theo dõi trạng thái thực hiện nhiệm vụ.
10. Sau khi hoàn thành nhiệm vụ tiếp cận, robot có thể quay lại chu trình tuần tra.

Luồng xử lý tổng quát của hệ thống được mô tả như sau:

Cảm biến → Xử lý dữ liệu → Định vị và lập bản đồ → Nhận diện mục tiêu → Lập kế hoạch đường đi → Điều khiển vận tốc → Cơ cấu chấp hành

1.4 Mục tiêu của đề tài

Mục tiêu tổng quát của đề tài là xây dựng và phân tích một hệ thống robot tự hành TurtleBot4 Lite có khả năng tuần tra, nhận diện và tiếp cận mục tiêu trong môi trường mô phỏng.

Các mục tiêu cụ thể gồm:

- Thiết lập môi trường mô phỏng TurtleBot4 Lite trên ROS2 Humble và Gazebo/Ignition.
- Khảo sát các nguồn dữ liệu cảm biến và thông tin trạng thái quan trọng như LiDAR, camera, encoder, odometry và TF.
- Sử dụng SLAM Toolbox để xây dựng bản đồ hai chiều của môi trường hoạt động.
- Sử dụng Nav2 để định vị, lập kế hoạch đường đi và điều hướng robot đến vị trí mong muốn.
- Tích hợp mô hình YOLOv8n để phát hiện đối tượng mục tiêu từ ảnh RGB.
- Ước lượng vị trí tương đối của mục tiêu từ dữ liệu ảnh và thông tin camera.
- Chuyển đổi tọa độ mục tiêu từ hệ tọa độ camera sang hệ tọa độ robot và hệ tọa độ bản đồ.
- Xây dựng Mission Manager để điều phối các trạng thái hoạt động như tuần tra, phát hiện mục tiêu, tiếp cận mục tiêu và quay lại nhiệm vụ ban đầu.
- Phân tích vai trò của cơ cấu chấp hành vi sai trong việc biến đổi lệnh vận tốc thành chuyển động thực tế của robot.

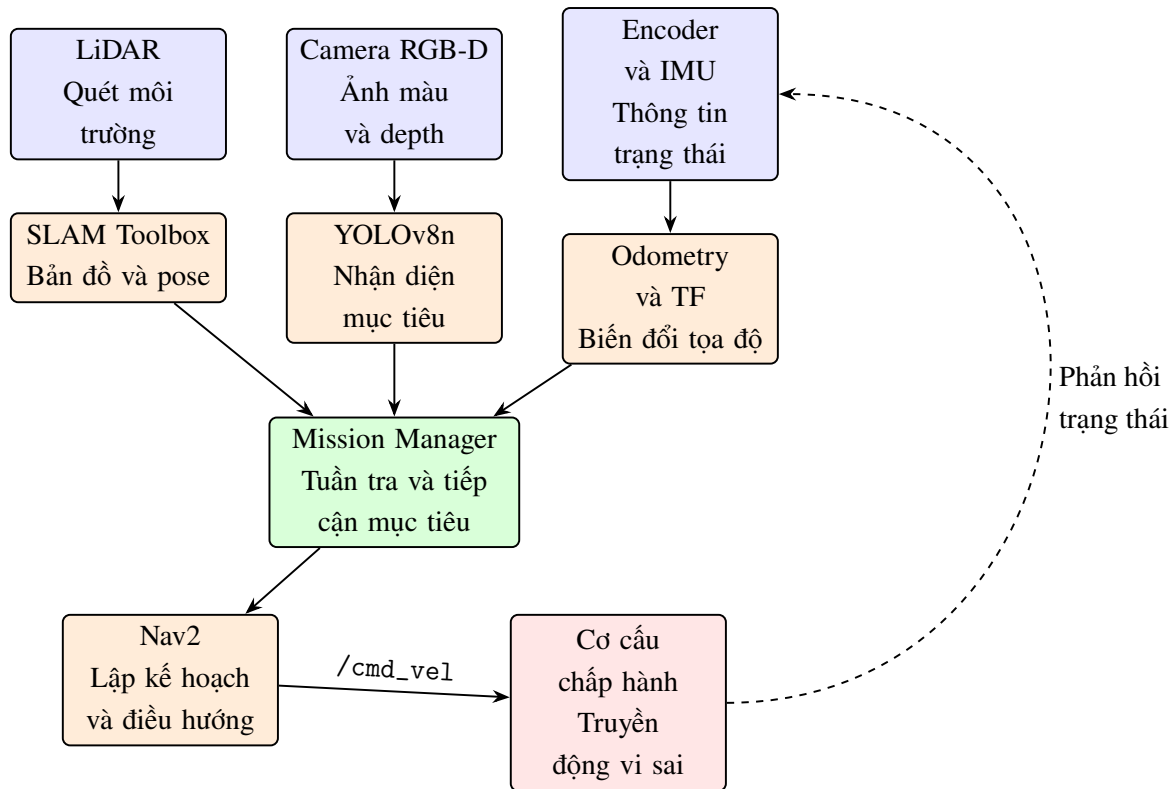
1.5 Yêu cầu kỹ thuật của hệ thống

Để hoàn thành nhiệm vụ, hệ thống cần đáp ứng các yêu cầu kỹ thuật cơ bản sau:

- Robot có thể khởi chạy ổn định trong môi trường mô phỏng Gazebo/Ignition.
- Các topic cảm biến cần thiết được publish đúng định dạng và có thể quan sát trên RViz2.
- Dữ liệu LiDAR được sử dụng để xây dựng bản đồ và hỗ trợ tránh vật cản.
- Dữ liệu odometry và TF được cập nhật liên tục để xác định trạng thái chuyển động của robot.
- Nav2 có thể nhận goal pose, lập kế hoạch và phát lệnh vận tốc điều khiển robot.
- Mô hình nhận diện có thể phát hiện đối tượng mục tiêu từ luồng ảnh RGB.
- Khi sử dụng dữ liệu depth, hệ thống có thể ước lượng khoảng cách từ camera đến mục tiêu.
- Mission Manager có thể chuyển đổi hợp lý giữa các trạng thái hoạt động của robot.
- Các module được thiết kế tương đối độc lập để thuận tiện cho kiểm thử và mở rộng.

1.6 Kiến trúc tổng quát của hệ thống

Kiến trúc tổng quát của hệ thống được minh họa trong Hình 1.2. Hệ thống được tổ chức theo chuỗi xử lý khép kín: dữ liệu cảm biến được thu nhận và xử lý để xác định trạng thái của robot và môi trường; Mission Manager lựa chọn hành vi phù hợp; Nav2 sinh lệnh vận tốc; cơ cấu truyền động vi sai tạo ra chuyển động thực tế và cung cấp thông tin phản hồi cho chu kỳ tiếp theo.



Hình 1.2 Kiến trúc tổng quát của hệ thống robot tự hành TurtleBot4 Lite

Hệ thống được chia thành các nhóm module chính như sau:

1. Module mô phỏng và bringup:

Khởi tạo robot TurtleBot4 Lite, môi trường Gazebo/Ignition, mô hình URDF và các node ROS2 cần thiết.

2. Module cảm biến và thông tin trạng thái:

Thu nhận dữ liệu LiDAR, camera, odometry và TF. Module này đóng vai trò cung cấp dữ liệu đầu vào cho các khối xử lý phía sau.

3. Module SLAM và localization:

Xây dựng bản đồ môi trường hoặc xác định pose hiện tại của robot trên bản đồ đã có.

4. Module navigation:

Sử dụng Nav2 để lập kế hoạch đường đi, tránh vật cản và sinh lệnh vận tốc tuyến tính, vận tốc góc cho robot.

5. Module perception:

Sử dụng mô hình YOLOv8n để nhận diện đối tượng mục tiêu từ ảnh RGB. Khi có dữ liệu depth, module thực hiện bổ sung bước ước lượng vị trí tương đối của mục tiêu.

6. Module Mission Manager:

Quản lý trạng thái hoạt động của hệ thống, điều phối quá trình tuần tra, tạm dừng, tiếp cận mục tiêu và tiếp tục nhiệm vụ.

1.7 Phạm vi nghiên cứu

Đề tài tập trung vào việc mô phỏng và tích hợp các thành phần cơ bản của một hệ robot tự hành. Robot sử dụng là TurtleBot4 Lite hoạt động trong môi trường warehouse của Gazebo/Ignition. Phần mềm được triển khai trên ROS2 Humble.

Môi trường mô phỏng TurtleBot4 Lite trong Ignition Gazebo được minh họa trong Hình 1.3. Việc sử dụng mô phỏng cho phép kiểm tra hoạt động của robot, luồng dữ liệu cảm biến và thuật toán điều hướng trước khi triển khai trên phần cứng thật.



Hình 1.3 TurtleBot4 Lite trong môi trường mô phỏng Ignition Gazebo

Nguồn: TurtleBot 4 User Manual

Trong phạm vi báo cáo, hệ thống tập trung vào các nội dung sau:

- Khởi tạo robot và môi trường mô phỏng.
- Quan sát và phân tích dữ liệu LiDAR, camera, odometry và TF.
- Lập bản đồ hai chiều bằng SLAM Toolbox.
- Điều hướng robot bằng Nav2.
- Nhận diện đối tượng mục tiêu bằng YOLOv8n pretrained trên tập dữ liệu COCO.
- Xây dựng logic quản lý nhiệm vụ cơ bản.

Một số nội dung chưa được triển khai chuyên sâu trong phạm vi đề tài gồm:

- Tối ưu hóa tham số Nav2 cho từng loại môi trường cụ thể.
- Theo dõi đồng thời nhiều đối tượng chuyển động.
- Xử lý tình huống mục tiêu bị che khuất hoặc mất khỏi trường nhìn camera.
- Huấn luyện lại mô hình nhận diện trên tập dữ liệu chuyên biệt.
- Tối ưu hiệu năng xử lý ảnh trên phần cứng nhúng.
- Đánh giá đầy đủ độ chính xác khi triển khai trên robot TurtleBot4 thật.

1.8 Phương pháp thực hiện

Quy trình thực hiện đề tài được chia thành các giai đoạn:

1. Nghiên cứu cấu trúc phần cứng và phần mềm của TurtleBot4 Lite.
2. Thiết lập ROS2 Humble và môi trường mô phỏng Gazebo/Ignition.
3. Kiểm tra các topic, frame và node liên quan đến cảm biến, cơ cấu chấp hành và trạng thái robot.
4. Chạy thử SLAM Toolbox để xây dựng bản đồ môi trường.
5. Chạy thử Nav2 để điều hướng robot đến các goal pose định trước.
6. Tích hợp mô hình YOLOv8n để nhận diện đối tượng từ camera.
7. Xây dựng module ước lượng vị trí và module quản lý nhiệm vụ.
8. Đánh giá kết quả, phân tích hạn chế và đề xuất hướng phát triển.

1.9 Đóng góp chính của báo cáo

Báo cáo không chỉ trình bày quy trình khởi chạy một hệ thống ROS2 mà còn phân tích vai trò kỹ thuật của từng thành phần dưới góc nhìn cảm biến và cơ cấu chấp hành.

Các đóng góp chính gồm:

- Đề xuất kiến trúc module cho hệ robot TurtleBot4 Lite thực hiện nhiệm vụ tuần tra và tiếp cận mục tiêu.
- Phân tích vai trò của LiDAR, camera, encoder, odometry và TF trong hệ robot tự hành.
- Trình bày chuỗi xử lý dữ liệu từ cảm biến đến bản đồ, pose robot và vị trí mục tiêu.
- Phân tích cơ cấu truyền động vi sai và quá trình Nav2 sinh lệnh vận tốc điều khiển robot.
- Xây dựng luồng nhiệm vụ kết hợp patrol, perception, localization và navigation.
- Đánh giá các kết quả đạt được, những hạn chế còn tồn tại và khả năng triển khai trên robot thật.

1.10 Kết quả kỳ vọng

Sau khi hoàn thành đề tài, hệ thống kỳ vọng đạt được các kết quả sau:

- Robot có thể hoạt động ổn định trong môi trường mô phỏng.
- Các dữ liệu cảm biến và thông tin trạng thái được hiển thị trực quan trên RViz2.
- Robot có thể xây dựng bản đồ môi trường và xác định vị trí tương đối trên bản đồ.
- Robot có thể nhận goal pose và tự động di chuyển đến vị trí được chỉ định.
- Hệ thống nhận diện có thể phát hiện đối tượng mục tiêu trong ảnh RGB.
- Mission Manager có thể điều phối luồng nhiệm vụ cơ bản từ tuần tra đến tiếp cận mục tiêu.

2. CƠ SỞ LÝ THUYẾT

2.1 Robot tự hành và vòng kín cảm biến - xử lý - điều khiển - chấp hành

Robot tự hành là một hệ cơ điện tử có khả năng di chuyển và thực hiện nhiệm vụ trong môi trường mà không cần con người điều khiển trực tiếp liên tục. Để tự hành, robot phải thu nhận dữ liệu từ môi trường, ước lượng trạng thái của bản thân, xây dựng hoặc sử dụng bản đồ, lập kế hoạch đường đi, tạo lệnh điều khiển và biến lệnh điều khiển thành chuyển động vật lý thông qua cơ cấu chấp hành.

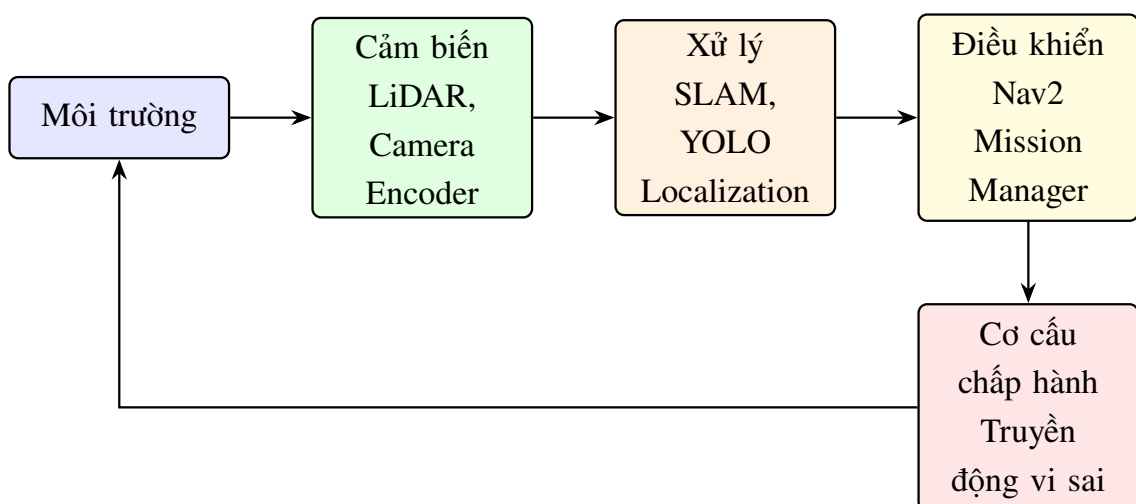
Trong đề tài này, robot TurtleBot4 được triển khai trong môi trường mô phỏng Gazebo/Ignition. Hệ thống sử dụng LiDAR để đo khoảng cách phục vụ SLAM và tránh vật cản, camera RGB-D OAK-D để phát hiện và định vị mục tiêu, odometry/encoder để ước lượng chuyển động, TF/TF2 để quản lý hệ tọa độ, Nav2 để điều hướng, YOLOv8 để nhận diện đối tượng và Mission Manager để điều phối hành vi.

Vòng kín hoạt động của robot tự hành có thể mô tả như sau:

Môi trường → Cảm biến → Xử lý/ước lượng

→ Điều khiển → Cơ cấu chấp hành → Môi trường.

Trong vòng kín này, cảm biến tạo dữ liệu đầu vào; các thuật toán xử lý biến dữ liệu đo thành thông tin trạng thái; bộ điều khiển và bộ lập kế hoạch tạo lệnh vận tốc hoặc lệnh vị trí; cơ cấu chấp hành biến lệnh điều khiển thành chuyển động thực tế. Nếu một khâu có sai số lớn, độ trễ cao hoặc mất dữ liệu, toàn bộ hệ thống có thể hoạt động không ổn định.



Hình 2.1 Sơ đồ vòng kín cảm biến – xử lý – điều khiển – chấp hành

Bảng 2.1 trình bày vai trò của các lớp chức năng trong hệ thống robot tự hành.

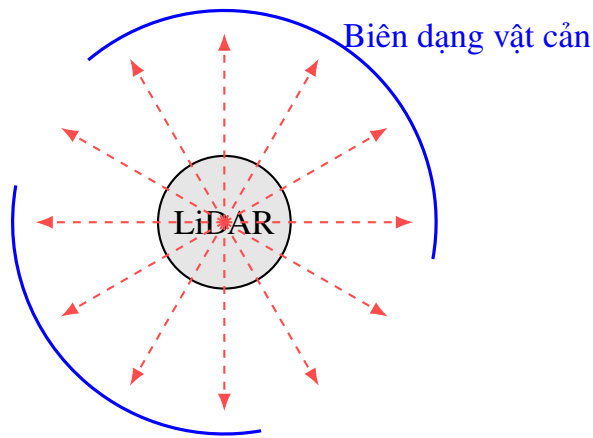
Lớp hệ thống	Dữ liệu hoặc lệnh chính	Chức năng trong robot	Thành phần trong project
Môi trường	Vật cản, mục tiêu, bản đồ, không gian di chuyển	Cung cấp điều kiện bên ngoài mà robot phải nhận biết và tương tác	Warehouse trong Gazebo/Ignition
Cảm biến	LaserScan, ảnh RGB, ảnh depth, odometry, TF	Thu nhận thông tin môi trường và trạng thái robot	LiDAR, OAK-D, encoder/odometry
Xử lý và ước lượng	Map, pose robot, detection, target pose	Biến dữ liệu thô thành thông tin có ý nghĩa	SLAM Toolbox, Object Localization, TF2
Điều khiển	Goal, path, velocity command	Lập kế hoạch và tạo lệnh chuyển động	Nav2, Patrol Node, Mission Manager
Chấp hành	Vận tốc bánh trái/phải, chuyển động thân robot	Thực thi lệnh điều khiển thành chuyển động	Differential drive actuator

2.2 LiDAR trong robot di động

2.2.1 Định nghĩa và chức năng

LiDAR là cảm biến đo khoảng cách bằng ánh sáng laser. Tên gọi LiDAR thường được hiểu là Light Detection and Ranging. Cảm biến phát chùm tia laser ra môi trường, thu tín hiệu phản xạ từ vật cản và tính khoảng cách dựa trên thời gian bay của tia hoặc dựa trên nguyên lý đo pha, đo cường độ phản xạ hay tam giác lượng tùy loại cảm biến.

Trong robot di động, LiDAR 2D thường quét theo mặt phẳng ngang quanh robot và tạo ra một dãy khoảng cách tương ứng với các góc quét khác nhau. Dữ liệu này giúp robot nhận biết tường, vật cản, hành lang, khoảng trống và biên dạng môi trường. Trong project TurtleBot4, LiDAR phát dữ liệu qua topic /scan để SLAM Toolbox, Nav2 costmap và RViz sử dụng.



Hình 2.2 Nguyên lý LiDAR 2D quét laser quanh robot và tạo tập điểm khoảng cách

2.2.2 Cấu tạo và chức năng từng thành phần

Thành phần	Chức năng
Bộ phát laser	Tạo chùm tia laser hẹp để chiếu ra môi trường theo từng hướng quét.
Bộ thu quang	Thu tín hiệu laser phản xạ từ vật cản quay lại cảm biến.
Cơ cấu quay hoặc gương quét	Thay đổi hướng tia laser để tạo vùng quét hai chiều quanh robot.
Mạch đo thời gian/cường độ	Đo thời gian bay, độ lệch pha hoặc cường độ phản xạ để suy ra khoảng cách.
Bộ xử lý tín hiệu	Lọc nhiễu, loại bỏ giá trị không hợp lệ và đóng gói dữ liệu thành mảng khoảng cách.
Giao tiếp ROS2	Chuyển dữ liệu đo thành bản tin <code>sensor_msgs/LaserScan</code> trên topic <code>/scan</code> .

2.2.3 Cách hoạt động và tính toán dữ liệu

Với nguyên lý thời gian bay, cảm biến phát tia laser đến vật cản và đo thời gian t để tia đi từ cảm biến đến vật cản rồi phản xạ quay lại. Vì tia laser đi cả chiều đi và chiều về, khoảng cách đến vật cản được tính bởi:

$$d = \frac{ct}{2}$$

Trong đó, d là khoảng cách từ LiDAR đến vật cản, c là vận tốc ánh sáng và t là thời gian bay hai chiều của tia laser.

LiDAR 2D không chỉ trả về một khoảng cách duy nhất mà trả về một mảng khoảng cách:

$$r = [r_0, r_1, \dots, r_{N-1}]$$

Mỗi phần tử r_i ứng với một góc quét θ_i :

$$\theta_i = \theta_{\min} + i\Delta\theta, \quad i = 0, 1, \dots, N-1$$

Khi cần biểu diễn điểm đo trong hệ tọa độ của cảm biến, ta đổi từ tọa độ cực sang tọa độ Descartes:

$$x_i = r_i \cos \theta_i$$

$$y_i = r_i \sin \theta_i$$

Như vậy, một bản tin LaserScan có thể được xem như tập hợp nhiều điểm (x_i, y_i) mô tả biên dạng vật cản xung quanh robot.

2.2.4 Input, output và ứng dụng trong project

Loại dữ liệu	Nội dung	Ý nghĩa trong project
Input vật lý	Tia laser phản xạ từ tường, kệ hàng, vật cản	Cung cấp thông tin khoảng cách thực tế xung quanh robot.
Output ROS2	Topic /scan kiểu sensor_msgs/LaserScan	Dữ liệu chính cho SLAM, costmap và kiểm tra trong RViz.
Tham số quan trọng	range_min, range_max, angle_min, angle_max, angle_increment	Quy định giới hạn đo, vùng quét và độ phân giải góc.
Ứng dụng trong project	SLAM Toolbox, Nav2, RViz	Xây dựng bản đồ, tránh vật cản và đánh giá dữ liệu cảm biến.

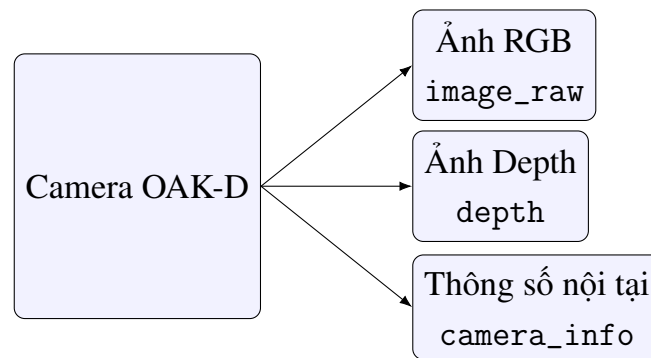
Trong project TurtleBot4, LiDAR được dùng chủ yếu cho ba nhiệm vụ: xây dựng bản đồ bằng SLAM, định vị robot trên bản đồ và cập nhật costmap để Nav2 tránh vật cản. Nếu dữ liệu /scan sai, cố định hoặc mất frame, robot có thể tạo bản đồ sai, không tránh được vật cản hoặc không thể điều hướng ổn định.

2.3 Camera RGB-D OAK-D

2.3.1 Định nghĩa và chức năng

Camera RGB-D là loại camera cung cấp đồng thời ảnh màu RGB và ảnh chiều sâu. Ảnh RGB chứa thông tin màu sắc và hình dạng của vật thể, phù hợp cho các thuật toán nhận diện đối tượng như YOLOv8. Ảnh depth cho biết khoảng cách từ camera đến các điểm trong ảnh, nhờ đó có thể ước lượng vị trí 3D của vật thể.

Trong project TurtleBot4, camera OAK-D được dùng để phát hiện mục tiêu bằng YOLOv8n và định vị mục tiêu bằng ảnh depth. Đây là cảm biến quan trọng giúp robot không chỉ biết có vật thể xuất hiện trong ảnh mà còn biết vật thể đó nằm cách robot bao xa.



Hình 2.3 Camera RGB-D OAK-D gồm luồng ảnh RGB, ảnh depth và thông số nội tại camera

2.3.2 Thành phần và chức năng

Thành phần	Chức năng
Camera RGB	Thu ảnh màu của môi trường để nhận diện đối tượng bằng YOLOv8.
Cụm camera stereo hoặc cảm biến depth	Tạo ảnh chiều sâu, trong đó mỗi pixel chứa giá trị khoảng cách.
Bộ xử lý ảnh	Đồng bộ dữ liệu RGB và depth, tiền xử lý ảnh và tạo luồng dữ liệu đầu ra.
Thông số nội tại camera	Cung cấp các tham số f_x , f_y , c_x , c_y phục vụ chuyển đổi 2D sang 3D.

Thành phần	Chức năng
Giao tiếp ROS2	Publish ảnh RGB, ảnh depth và CameraInfo cho các node xử lý.

2.3.3 Cách hoạt động và tính toán dữ liệu

Ảnh RGB được đưa vào mô hình YOLOv8 để phát hiện bounding box của đối tượng. Giả sử bounding box có tâm tại điểm ảnh (u, v) và ảnh depth cho biết độ sâu tại điểm đó là Z . Dựa trên mô hình camera lỗ kim, tọa độ 3D của điểm trong hệ camera được tính như sau:

$$X_c = \frac{(u - c_x)Z}{f_x}$$

$$Y_c = \frac{(v - c_y)Z}{f_y}$$

$$Z_c = Z$$

Trong đó, f_x và f_y là tiêu cự theo đơn vị pixel; c_x và c_y là tọa độ tâm ảnh; u và v là tọa độ pixel của tâm bounding box; Z là giá trị depth tại vùng chứa mục tiêu.

Có thể viết dưới dạng ma trận nội tại camera:

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

Quan hệ chiếu phối cảnh từ điểm 3D sang điểm ảnh là:

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix}$$

Vì ảnh depth thường có nhiễu hoặc điểm mất dữ liệu, project không nên lấy một pixel đơn lẻ tại tâm bbox. Cách ổn định hơn là lấy trung vị trong một cửa sổ nhỏ quanh tâm bbox:

$$Z = \text{median} \{D(u+i, v+j) \mid (i, j) \in \Omega\}$$

Trong đó, D là ảnh depth và Ω là vùng lân cận quanh tâm bounding box.

2.3.4 Input, output và ứng dụng trong project

Loại dữ liệu	Topic hoặc nội dung	Vai trò trong project
Input ảnh RGB	/oakd/rgb/preview/ image_raw	Ảnh đầu vào cho node YOLOv8.
Input ảnh depth	/oakd/rgb/preview/ depth	Dùng để lấy khoảng cách từ camera đến mục tiêu.
Input camera info	/oakd/rgb/preview/ camera_info	Cung cấp f_x, f_y, c_x, c_y để tính tọa độ 3D.
Output trung gian	Bounding box và class id từ YOLOv8	Xác định vùng ảnh chứa mục tiêu.
Output cuối	/target_object_ pose_map	Vị trí mục tiêu sau định vị, dùng cho Mission Manager.

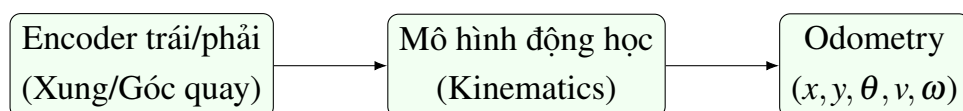
Trong project, camera RGB-D biến thông tin thị giác thành vị trí mục tiêu trong không gian. Nếu chỉ có ảnh RGB, robot chỉ biết mục tiêu nằm ở đâu trên ảnh; nếu chỉ có depth, robot không biết điểm nào là mục tiêu. Khi kết hợp RGB, YOLOv8 và depth, robot có thể chuyển thông tin ngữ nghĩa thành tọa độ 3D để điều hướng.

2.4 Odometry và encoder

2.4.1 Định nghĩa và chức năng

Odometry là phương pháp ước lượng chuyển động của robot theo thời gian dựa trên dữ liệu từ encoder bánh xe và mô hình động học. Encoder đo góc quay hoặc số xung quay của từng bánh xe. Từ đó, robot tính được quãng đường mỗi bánh đã đi, suy ra độ dịch chuyển và độ quay của thân robot.

Odometry có ưu điểm là cập nhật nhanh, liên tục và không phụ thuộc trực tiếp vào môi trường bên ngoài. Tuy nhiên, odometry có sai số tích lũy theo thời gian do trượt bánh, sai số bán kính bánh xe, sai số khoảng cách giữa hai bánh và nhiễu encoder. Vì vậy, odometry thường được kết hợp với LiDAR và SLAM để tăng độ chính xác.



Hình 2.4 Encoder đo góc quay hai bánh và suy ra chuyển động của robot vì sai

2.4.2 Thành phần và chức năng

Thành phần	Chức năng
Encoder bánh trái	Đo góc quay hoặc số xung của bánh trái.
Encoder bánh phải	Đo góc quay hoặc số xung của bánh phải.
Bánh xe và động cơ	Tạo chuyển động vật lý cho robot.
Bộ điều khiển thấp	Đọc encoder, tính vận tốc bánh và xuất dữ liệu odometry.
Topic /odom	Cung cấp pose tương đối và vận tốc của robot cho các node ROS2.

2.4.3 Cách tính toán dữ liệu odometry

Gọi $\Delta\phi_r$ và $\Delta\phi_l$ lần lượt là độ thay đổi góc quay của bánh phải và bánh trái trong một chu kỳ lấy mẫu. Nếu bán kính bánh xe là r , quãng đường mỗi bánh đi được là:

$$\Delta s_r = r\Delta\phi_r$$

$$\Delta s_l = r\Delta\phi_l$$

Độ dịch chuyển trung bình của robot là:

$$\Delta s = \frac{\Delta s_r + \Delta s_l}{2}$$

Độ thay đổi góc hướng của robot là:

$$\Delta\theta = \frac{\Delta s_r - \Delta s_l}{L}$$

Trong đó, L là khoảng cách giữa hai bánh chủ động. Nếu trạng thái robot tại thời điểm k là (x_k, y_k, θ_k) , trạng thái tại thời điểm tiếp theo có thể xấp xỉ bởi:

$$x_{k+1} = x_k + \Delta s \cos\left(\theta_k + \frac{\Delta\theta}{2}\right)$$

$$y_{k+1} = y_k + \Delta s \sin\left(\theta_k + \frac{\Delta\theta}{2}\right)$$

$$\theta_{k+1} = \theta_k + \Delta\theta$$

2.4.4 Input, output và ứng dụng trong project

Loại dữ liệu	Nội dung	Vai trò trong project
Input	Xung encoder hoặc vận tốc bánh xe	Cơ sở để tính quãng đường bánh trái/phải.
Output	Topic /odom	Cung cấp pose tương đối và vận tốc robot.
Dữ liệu sử dụng	x, y, θ, v, ω	Đưa vào SLAM, Nav2 và TF.
Ứng dụng	Hỗ trợ scan matching, điều hướng và ước lượng trạng thái	Giúp robot có thông tin chuyển động liên tục.

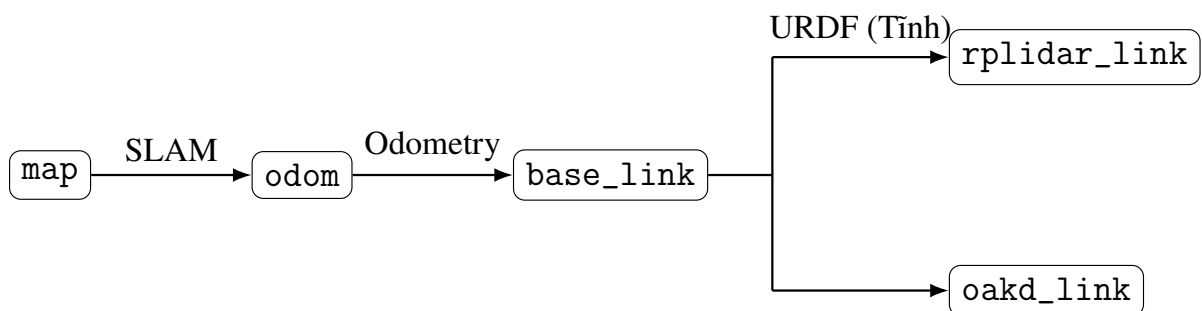
Trong project TurtleBot4, odometry giúp robot biết chuyển động tương đối của mình giữa các lần đo LiDAR. Dữ liệu này đặc biệt quan trọng đối với SLAM vì nó giúp giới hạn vùng tìm kiếm khi so khớp các scan liên tiếp.

2.5 Hệ tọa độ và TF/TF2 trong ROS2

2.5.1 Định nghĩa và chức năng

Trong robot, các cảm biến thường được lắp tại các vị trí khác nhau. LiDAR, camera, thân robot và bản đồ không cùng một hệ tọa độ. TF/TF2 là cơ chế của ROS2 dùng để quản lý và biến đổi dữ liệu giữa các hệ tọa độ đó.

Các frame thường gặp trong project gồm map, odom, base_link, oakd_link và rplidar_link. Frame map là hệ tọa độ toàn cục, ổn định và dùng cho navigation. Frame odom là hệ tọa độ cục bộ liên tục nhưng có thể trôi. Frame base_link gắn với thân robot. Các frame cảm biến gắn với vị trí lắp đặt LiDAR và camera.



Hình 2.5 Cây TF của hệ thống TurtleBot4

2.5.2 Công thức biến đổi tọa độ

Một điểm p_B trong hệ tọa độ B có thể được chuyển sang hệ tọa độ A bằng công thức:

$$p_A = R_A^B p_B + t_A^B$$

Trong đó, R_A^B là ma trận quay từ hệ B sang hệ A , còn t_A^B là vector tịnh tiến giữa hai hệ tọa độ.

Dưới dạng tọa độ thuần nhất, phép biến đổi được viết là:

$$\begin{bmatrix} p_A \\ 1 \end{bmatrix} = \begin{bmatrix} R_A^B & t_A^B \\ 0^T & 1 \end{bmatrix} \begin{bmatrix} p_B \\ 1 \end{bmatrix}$$

Nếu mục tiêu được phát hiện trong frame camera, TF cho phép chuyển pose mục tiêu từ oakd_link sang map. Nhờ đó, Mission Manager có thể tạo goal trong hệ tọa độ bản đồ để gửi cho Nav2.

2.5.3 Input, output và ứng dụng trong project

Loại dữ liệu	Nội dung	Vai trò trong project
Input	Quan hệ hình học giữa các frame	Xác định vị trí tương đối giữa robot, cảm biến và bản đồ.
Output	Transform giữa các frame	Cho phép chuyển đổi dữ liệu cảm biến sang frame cần dùng.
Topic liên quan	/tf, /tf_static	Cung cấp transform động và transform tĩnh.
Ứng dụng	Chuyển pose mục tiêu từ camera sang map	Điều kiện bắt buộc để Mission Manager gửi goal chính xác.

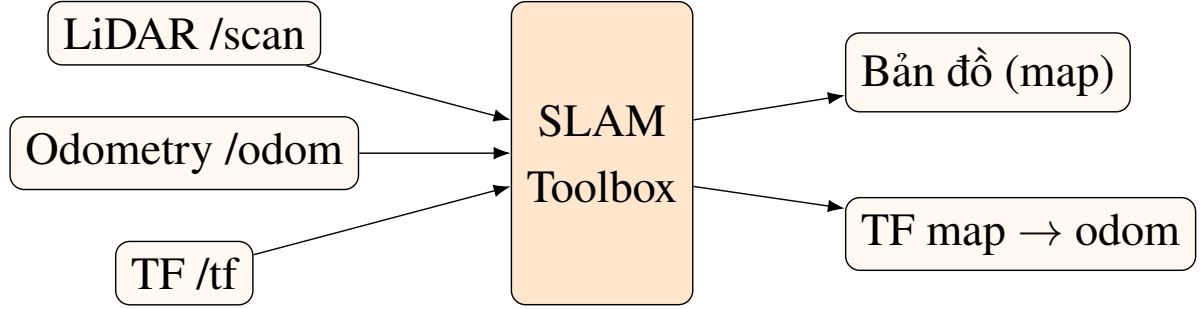
2.6 SLAM

2.6.1 Định nghĩa và chức năng

SLAM là viết tắt của Simultaneous Localization and Mapping, nghĩa là đồng thời định vị và xây dựng bản đồ. Bài toán SLAM đặt ra khi robot chưa có bản đồ hoàn chỉnh

của môi trường. Robot phải vừa ước lượng vị trí của chính nó, vừa cập nhật bản đồ dựa trên dữ liệu cảm biến.

Trong project TurtleBot4, SLAM Toolbox sử dụng dữ liệu LiDAR /scan, odometry và TF để xây dựng bản đồ occupancy grid. Bản đồ này sau đó được dùng cho Nav2 để lập kế hoạch đường đi.



Hình 2.6 SLAM sử dụng LiDAR, odometry và TF để tạo bản đồ và pose robot

2.6.2 Nguyên lý tính toán trong SLAM

Một mô hình SLAM tổng quát có thể mô tả bằng hai phương trình: mô hình chuyển động và mô hình đo.

Mô hình chuyển động:

$$x_t = f(x_{t-1}, u_t) + w_t$$

Mô hình đo:

$$z_t = h(x_t, m) + v_t$$

Trong đó, x_t là trạng thái robot tại thời điểm t , u_t là lệnh điều khiển hoặc odometry, z_t là dữ liệu đo từ cảm biến, m là bản đồ, w_t là nhiễu chuyển động và v_t là nhiễu đo.

Với SLAM dựa trên đồ thị pose, robot tạo các nút pose và các ràng buộc giữa chúng. Bài toán tối ưu có thể biểu diễn dưới dạng:

$$X^* = \arg \min_X \sum_{(i,j) \in \mathcal{C}} \|e_{ij}(x_i, x_j, z_{ij})\|_{\Omega_{ij}}^2$$

Trong đó, X^* là tập pose tối ưu, $\mathcal{C}$ là tập các ràng buộc giữa các pose, e_{ij} là sai số giữa đo thực tế và dự đoán, Ω_{ij} là ma trận trọng số thông tin.

Bản đồ occupancy grid thường biểu diễn môi trường bằng các ô lưới. Mỗi ô có xác suất bị chiếm dụng. Cập nhật theo log-odds có dạng:

$$L_t(m_i) = L_{t-1}(m_i) + \log \frac{p(m_i | z_t)}{1 - p(m_i | z_t)} - L_0$$

Trong đó, $L_t(m_i)$ là log-odds của ô m_i tại thời điểm t , z_t là dữ liệu đo hiện tại và L_0 là log-odds ban đầu.

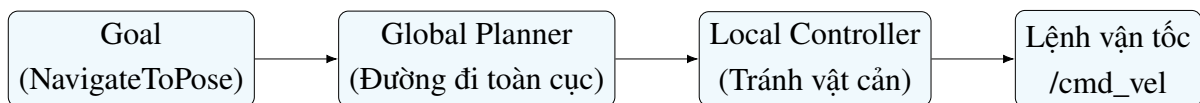
2.6.3 Input, output và ứng dụng trong project

Loại dữ liệu	Nội dung	Vai trò trong project
Input	/scan, /odom, /tf	Dữ liệu phục vụ scan matching và tối ưu pose.
Output	Occupancy grid map, transform map -> odom	Bản đồ và quan hệ frame cho navigation.
Thuật toán liên quan	Scan matching, pose graph, loop closure	Giảm sai số tích lũy và cải thiện bản đồ.
Ứng dụng	Tạo bản đồ warehouse	Là nền tảng để Nav2 lập kế hoạch đường đi.

2.7 Nav2 và điều hướng robot

2.7.1 Định nghĩa và chức năng

Nav2 là framework điều hướng cho ROS2. Nav2 nhận mục tiêu vị trí, lập đường đi toàn cục, tạo lệnh điều khiển cục bộ, tránh vật cản và gửi lệnh vận tốc cho robot. Trong project TurtleBot4, Patrol Node và Mission Manager không điều khiển trực tiếp bánh xe mà gửi goal đến Nav2 qua action NavigateToPose.



Hình 2.7 Luồng điều hướng Nav2 từ goal đến global planner, local controller và /cmd_vel

2.7.2 Cách hoạt động của Nav2

Nav2 thường gồm các khối chính: map server, localization, global planner, local controller, costmap, behavior tree và recovery behaviors. Khi nhận goal, Nav2 tìm một đường đi từ pose hiện tại đến goal. Một cách khái quát, bài toán lập đường đi có thể viết

như sau:

$$\mathcal{P}^* = \arg \min_{\mathcal{P}} \sum_{i=0}^{n-1} J(p_i, p_{i+1})$$

Trong đó, $\mathcal{P}^*$ là đường đi tối ưu, $\mathcal{P} = \{p_0, p_1, \dots, p_n\}$ là một đường đi ứng viên và J là hàm chi phí giữa hai điểm liên tiếp. Chi phí này thường phụ thuộc vào độ dài đường đi, khoảng cách đến vật cản và vùng chi phí trên costmap.

Sau khi có đường đi toàn cục, local controller tạo lệnh vận tốc:

$$u = \begin{bmatrix} v \\ \omega \end{bmatrix}$$

Trong đó, v là vận tốc tuyến tính và ω là vận tốc góc. Lệnh này được publish dưới dạng geometry_msgs/Twist trên topic /cmd_vel.

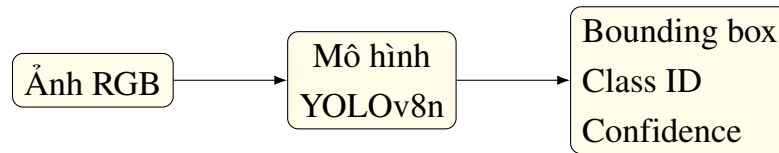
2.7.3 Input, output và ứng dụng trong project

Loại dữ liệu	Nội dung	Vai trò trong project
Input	Map, robot pose, costmap, goal	Cơ sở để lập đường đi và tránh vật cản.
Output	Topic /cmd_vel	Lệnh vận tốc gửi đến cơ cấu chấp hành.
Action	NavigateToPose	Giao tiếp goal từ Patrol Node hoặc Mission Manager đến Nav2.
Ứng dụng	Tuần tra waypoint, tiếp cận mục tiêu	Điều khiển robot di chuyển an toàn trong môi trường.

2.8 YOLOv8 trong nhận diện đối tượng

2.8.1 Định nghĩa và chức năng

YOLO là họ mô hình phát hiện đối tượng một giai đoạn. YOLOv8 nhận ảnh đầu vào và dự đoán trực tiếp bounding box, nhãn lớp và độ tin cậy của đối tượng. Trong project TurtleBot4, YOLOv8n được dùng vì mô hình nhỏ, tốc độ nhanh và phù hợp với mô phỏng thời gian thực.



Hình 2.8 YOLOv8 nhận ảnh RGB và xuất bounding box, class id, confidence

2.8.2 Dữ liệu đầu ra của YOLOv8

Mỗi đối tượng được phát hiện có thể biểu diễn bởi:

$$b = (x_c, y_c, w, h, c, s)$$

Trong đó, x_c và y_c là tọa độ tâm bounding box, w là chiều rộng bbox, h là chiều cao bbox, c là nhãn lớp và s là độ tin cậy.

Độ tin cậy có thể hiểu khái quát là:

$$s = P(\text{object}) P(c \mid \text{object})$$

Khi có nhiều bounding box chồng lấn, thuật toán Non-Maximum Suppression sử dụng chỉ số IoU để loại bỏ bbox dư thừa:

$$IoU(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

BBox có độ tin cậy cao hơn được giữ lại, còn bbox có IoU lớn hơn ngưỡng và confidence thấp hơn sẽ bị loại bỏ.

2.8.3 Input, output và ứng dụng trong project

Loại dữ liệu	Nội dung	Vai trò trong project
Input	Ảnh RGB từ /oakd/rgb/preview/image_raw	Dữ liệu đầu vào cho mô hình YOLOv8n.
Output	/vision/detected_objects	Chứa bounding box, class id và confidence.
Tham số	Confidence threshold, model path	Quy định ngưỡng nhận diện và mô hình sử dụng.

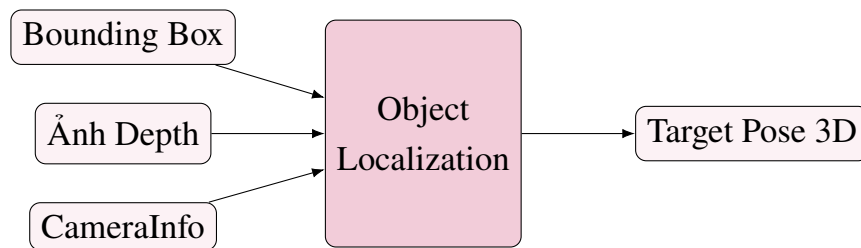
Loại dữ liệu	Nội dung	Vai trò trong project
Ứng dụng	Phát hiện mục tiêu, ưu tiên lớp person	Cung cấp bbox cho Object Localization.

2.9 Định vị mục tiêu từ RGB-D

2.9.1 Vai trò của định vị mục tiêu

Nhận diện đối tượng chỉ cho biết mục tiêu nằm ở vùng nào trên ảnh. Để robot di chuyển đến gần mục tiêu, hệ thống cần chuyển vùng ảnh đó thành vị trí 3D trong không gian. Đây là nhiệm vụ của Object Localization.

Object Localization kết hợp ba nguồn dữ liệu: bounding box từ YOLOv8, ảnh depth từ camera RGB-D và thông số nội tại camera. Kết quả cuối cùng là pose mục tiêu để Mission Manager tính điểm tiếp cận an toàn.



Hình 2.9 Quy trình bbox, depth và CameraInfo tạo target pose 3D

2.9.2 Quy trình tính toán

Bước 1: Lấy tâm bounding box:

$$u = \frac{x_{\min} + x_{\max}}{2}$$

$$v = \frac{y_{\min} + y_{\max}}{2}$$

Bước 2: Lấy độ sâu tại vùng lân cận tâm bbox:

$$Z = \text{median}(D_{\Omega})$$

Bước 3: Chuyển điểm ảnh sang tọa độ 3D trong hệ camera:

$$\begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix} = \begin{bmatrix} \frac{(u-c_x)Z}{f_x} \\ \frac{(v-c_y)Z}{f_y} \\ Z \end{bmatrix}$$

Bước 4: Đổi từ quy ước trục camera sang quy ước trục ROS:

$$x_{ros} = Z_c$$

$$y_{ros} = -X_c$$

$$z_{ros} = -Y_c$$

Bước 5: Dùng TF để chuyển pose mục tiêu sang frame map nếu cần:

$$p_{map} = T_{map}^{camera} p_{camera}$$

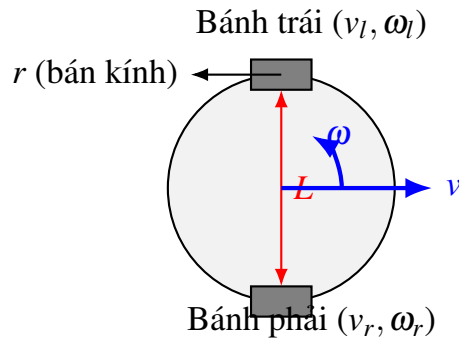
2.9.3 Input, output và ứng dụng trong project

Loại dữ liệu	Nội dung	Vai trò trong project
Input 1	Detection từ YOLOv8	Xác định vùng ảnh chứa mục tiêu.
Input 2	Ảnh depth	Lấy khoảng cách từ camera đến mục tiêu.
Input 3	CameraInfo	Cung cấp ma trận nội tại camera.
Output	/target_object_pose_map, /target_object_marker	Pose mục tiêu và marker hiển thị trong RViz.
Ứng dụng	Cung cấp vị trí mục tiêu cho Mission Manager	Giúp robot tiếp cận mục tiêu thay vì chỉ phát hiện trên ảnh.

2.10 Cơ cấu chấp hành robot vi sai

2.10.1 Định nghĩa và chức năng

Robot vi sai là robot di động có hai bánh chủ động được điều khiển độc lập. Khi hai bánh quay cùng tốc độ, robot đi thẳng. Khi hai bánh quay khác tốc độ, robot quay. Khi hai bánh quay ngược chiều, robot có thể quay gần như tại chỗ. Cấu trúc này đơn giản, dễ điều khiển và phù hợp với robot trong nhà như TurtleBot4.



Hình 2.10 Mô hình robot vi sai với hai bánh chủ động, bán kính bánh r và khoảng cách bánh L

2.10.2 Mô hình động học

Giả sử robot chuyển động trên mặt phẳng và trạng thái robot là:

$$x = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix}$$

Phương trình động học của robot vi sai là:

$$\dot{x} = v \cos \theta$$

$$\dot{y} = v \sin \theta$$

$$\dot{\theta} = \omega$$

Trong đó, v là vận tốc tuyến tính của robot và ω là vận tốc góc quanh trục thẳng đứng.

Quan hệ giữa vận tốc robot và vận tốc góc hai bánh là:

$$v = \frac{r}{2} (\omega_r + \omega_l)$$

$$\omega = \frac{r}{L} (\omega_r - \omega_l)$$

Trong đó, ω_r là vận tốc góc bánh phải, ω_l là vận tốc góc bánh trái, r là bán kính bánh xe và L là khoảng cách giữa hai bánh chủ động.

Từ hai công thức trên, vận tốc từng bánh được tính theo lệnh v và ω :

$$\omega_r = \frac{v + \frac{L}{2}\omega}{r}$$

$$\omega_l = \frac{v - \frac{L}{2}\omega}{r}$$

2.10.3 Ý nghĩa các trường hợp chuyển động

Điều kiện vận tốc bánh	Chuyển động của robot	Ý nghĩa điều khiển
$\omega_r = \omega_l$	Đi thẳng	Hai bánh tạo cùng vận tốc tiến.
$\omega_r > \omega_l$	Quay về phía trái	Bánh phải đi nhanh hơn bánh trái.
$\omega_r < \omega_l$	Quay về phía phải	Bánh trái đi nhanh hơn bánh phải.
$\omega_r = -\omega_l$	Quay tại chỗ	Hai bánh quay ngược chiều nhau.
$\omega_r = \omega_l = 0$	Đứng yên	Robot không di chuyển.

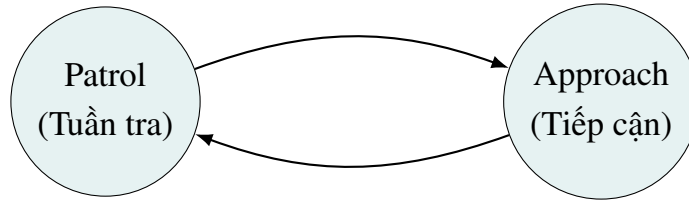
Trong project TurtleBot4, Nav2 không trực tiếp điều khiển từng bánh xe mà phát lệnh `/cmd_vel` gồm v và ω . Bộ điều khiển cấp thấp sẽ chuyển lệnh này thành vận tốc bánh trái và bánh phải theo mô hình động học vi sai.

2.11 Mission Manager và thuật toán tiếp cận an toàn

2.11.1 Chức năng của Mission Manager

Mission Manager là lớp điều phối hành vi của robot. Khi robot đang tuần tra, Mission Manager theo dõi pose mục tiêu được publish từ Object Localization. Nếu mục tiêu hợp lệ và khoảng cách đến mục tiêu lớn hơn khoảng cách an toàn, Mission Manager tạm dừng Patrol Node, tính một goal an toàn và gửi goal đó đến Nav2.

Phát hiện mục tiêu



Tiếp cận xong / Mất dấu

Hình 2.11 Mission Manager chuyển từ trạng thái tuần tra sang tiếp cận mục tiêu rồi quay lại tuần tra

2.11.2 Công thức tính safe goal

Gọi vị trí robot trong frame map là:

$$p_r = \begin{bmatrix} x_r \\ y_r \end{bmatrix}$$

Vị trí mục tiêu là:

$$p_t = \begin{bmatrix} x_t \\ y_t \end{bmatrix}$$

Vector từ robot đến mục tiêu là:

$$\Delta p = p_t - p_r = \begin{bmatrix} x_t - x_r \\ y_t - y_r \end{bmatrix} = \begin{bmatrix} d_x \\ d_y \end{bmatrix}$$

Khoảng cách phẳng giữa robot và mục tiêu là:

$$d = \sqrt{d_x^2 + d_y^2}$$

Gọi d_s là khoảng cách an toàn cần giữ lại giữa robot và mục tiêu. Hệ số tiến đến mục tiêu được tính bởi:

$$\alpha = \frac{d - d_s}{d}, \quad d > d_s$$

Khi đó, goal an toàn là:

$$p_g = p_r + \alpha \Delta p$$

Viết theo từng thành phần:

$$x_g = x_r + \alpha d_x$$

$$y_g = y_r + \alpha d_y$$

Góc hướng để robot quay mặt về phía mục tiêu có thể tính bởi:

$$\theta_g = \text{atan2}(d_y, d_x)$$

Công thức này giúp robot không đi đến đúng vị trí mục tiêu mà dừng trước mục tiêu một khoảng d_s , giảm nguy cơ va chạm hoặc đứng quá sát đối tượng.

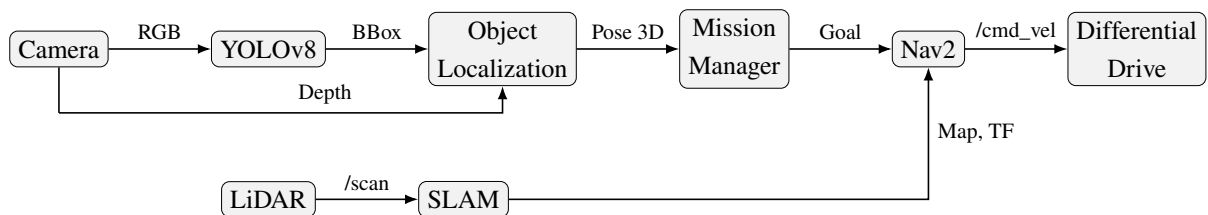
2.11.3 Input, output và ứng dụng trong project

Loại dữ liệu	Nội dung	Vai trò trong project
Input	Target pose, robot pose, TF	Xác định vị trí tương đối giữa robot và mục tiêu.
Tham số	safe_distance, cooldown_time	Quy định khoảng cách dừng và thời gian tránh lặp phản ứng.
Output	Goal an toàn gửi đến Nav2	Điều khiển robot tiếp cận mục tiêu.
Ứng dụng	Tuần tra - phát hiện - tiếp cận - quay lại tuần tra	Tạo hành vi tự hành ở mức nhiệm vụ.

2.12 Luồng dữ liệu tổng hợp trong project TurtleBot4

Toàn bộ hệ thống có thể được mô tả bằng chuỗi dữ liệu từ cảm biến đến điều khiển như sau:

RGB Image $\rightarrow$ YOLOv8 $\rightarrow$ 2D Detection,
 2D Detection + Depth + CameraInfo $\rightarrow$ 3D Target Pose,
 Target Pose + TF + Robot Pose $\rightarrow$ Safe Goal,
 Safe Goal + Map + Costmap $\rightarrow$ Nav2 Path $\rightarrow$ /cmd_vel,
 /cmd_vel $\rightarrow$ Differential Drive $\rightarrow$ Wheel Motion.



Hình 2.12 Luồng dữ liệu tổng hợp từ camera, LiDAR, SLAM, YOLOv8, Mission Manager đến Nav2 và cơ cấu chấp hành

Bảng 2.12 tóm tắt input/output của các module chính trong project.

Module	Input	Output	Vai trò chính
LiDAR	Môi trường vật lý	/scan	Đo khoảng cách phục vụ SLAM và tránh vật cản.
OAK-D RGB-D	Ảnh RGB, ảnh depth	Image, depth, CameraInfo	Cung cấp dữ liệu cho nhận diện và định vị mục tiêu.
YOLOv8	Ảnh RGB	/vision/ detected_objects	Phát hiện đối tượng và tạo bounding box.
Object Localization	Detection, depth, CameraInfo, TF	/target_object_ pose_map	Tính vị trí 3D của mục tiêu.
SLAM Toolbox	/scan, /odom, TF	Map, transform map -> odom	Xây dựng bản đồ và hỗ trợ định vị.
Nav2	Map, pose, goal, costmap	/cmd_vel	Lập đường đi và tạo lệnh vận tốc.

Module	Input	Output	Vai trò chính
Mission Manager	Target pose, robot pose	Safe goal, patrol enable/disable	Điều phối hành vi tuần tra và tiếp cận.
Differential Drive	v, ω	Vận tốc hai bánh	Biến lệnh điều khiển thành chuyển động.

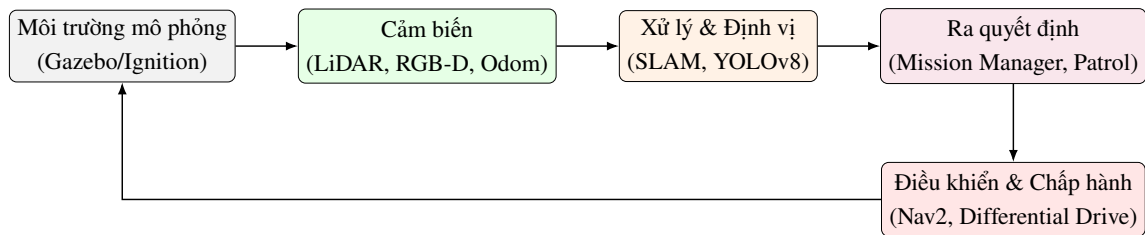
2.13 Kết luận chương

Chương này đã trình bày các cơ sở lý thuyết quan trọng phục vụ xây dựng hệ thống robot tự hành TurtleBot4. Các nội dung bao gồm vòng kín cảm biến - xử lý - điều khiển - chấp hành, LiDAR, camera RGB-D, odometry/encoder, TF/TF2, SLAM, Nav2, YOLOv8, định vị mục tiêu RGB-D, cơ cấu truyền động vi sai và thuật toán tiếp cận mục tiêu an toàn.

Các công thức được trình bày trong chương cho thấy mối liên hệ giữa dữ liệu cảm biến và quyết định điều khiển. LiDAR biến thời gian bay của tia laser thành khoảng cách; camera RGB-D biến điểm ảnh và depth thành tọa độ 3D; encoder biến góc quay bánh xe thành odometry; TF biến đổi dữ liệu giữa các hệ tọa độ; Nav2 biến goal thành lệnh vận tốc; cơ cấu vi sai biến lệnh vận tốc thành chuyển động bánh xe. Đây là nền tảng để triển khai các chương tiếp theo về kiến trúc hệ thống, cảm biến, điều khiển chuyển động và nhiệm vụ tự hành.

3. THIẾT KẾ KIẾN TRÚC HỆ THỐNG

Chương này trình bày kiến trúc tổng thể của hệ thống robot tự hành TurtleBot4 trong môi trường mô phỏng. Khác với Chương 2, nội dung ở đây không đi sâu lại nguyên lý của từng cảm biến mà tập trung vào cách các thành phần phần mềm và dữ liệu được tổ chức thành một hệ thống hoàn chỉnh. Mục tiêu của chương là làm rõ hệ thống gồm những package nào, mỗi package nhận dữ liệu gì, xử lý gì, xuất dữ liệu gì và phối hợp với nhau như thế nào để robot có thể tuần tra, phát hiện mục tiêu và tiếp cận mục tiêu an toàn.



Hình 3.1 Kiến trúc tổng quát của hệ thống TurtleBot4 sử dụng ROS2, SLAM, Nav2 và YOLOv8

3.1 Tổng quan kiến trúc hệ thống

Hệ thống được thiết kế theo hướng module hóa trên ROS2. Mỗi chức năng chính được tách thành một package riêng, giao tiếp với nhau thông qua topic, action, parameter và TF. Cách tổ chức này giúp hệ thống dễ mở rộng, dễ kiểm thử và dễ thay thế từng phần mà không làm ảnh hưởng đến toàn bộ chương trình.

Về tổng thể, hệ thống có thể chia thành bốn lớp chính: lớp mô phỏng và cảm biến, lớp xử lý dữ liệu, lớp ra quyết định và lớp điều khiển chuyển động. Các lớp này tương ứng với chuỗi hoạt động từ thu nhận dữ liệu, xử lý thông tin, quyết định hành vi đến phát lệnh điều khiển cho robot.

Bảng 3.1 Các lớp chức năng chính trong kiến trúc hệ thống

Lớp hệ thống	Thành phần chính	Vai trò trong project
Mô phỏng và cảm biến	Gazebo/Ignition, TurtleBot4 Lite, LiDAR, camera RGB-D, odometry	Tạo môi trường thử nghiệm và sinh dữ liệu đầu vào cho hệ thống.
Xử lý dữ liệu	SLAM/localization, YOLOv8, object localization, TF	Biến dữ liệu cảm biến thành bản đồ, pose robot và pose mục tiêu.
Ra quyết định	Patrol Node, Mission Manager	Quyết định robot tiếp tục tuần tra hay chuyển sang tiếp cận mục tiêu.
Điều khiển chuyển động	Nav2, /cmd_vel, differential drive	Lập kế hoạch đường đi và tạo lệnh vận tốc cho robot.

Có thể mô tả tập các package chính của hệ thống như sau:

$$\mathcal{P} = \{P_b, P_v, P_o, P_p, P_m\} \quad (3.1)$$

Trong đó, P_b là package bringup, P_v là package vision, P_o là package object localization, P_p là package patrol và P_m là package mission manager. Biểu thức trên nhấn mạnh hệ thống được tổ chức thành nhiều module độc lập, không phải một chương trình đơn lẻ.

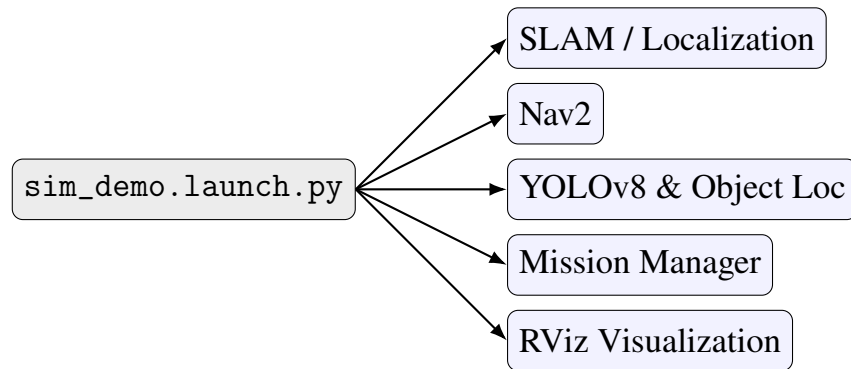
3.2 Các package chính trong hệ thống

3.2.1 Package *tb4_bringup*

tb4_bringup là package khởi chạy và cấu hình tổng thể. Package này gom các launch file, file cấu hình và công cụ hiển thị vào một điểm khởi động chung. Nó không trực tiếp nhận diện đối tượng hay điều khiển mục tiêu, nhưng có vai trò quan trọng vì bảo đảm các node được khởi động đúng thứ tự.

Bảng 3.2 Vai trò của package `tb4_bringup`

Mục	Nội dung
Chức năng chính	Khởi chạy các node cần thiết cho toàn bộ hệ thống.
Input	Tham số launch, đường dẫn bản đồ, file cấu hình Nav2, file cấu hình SLAM, tùy chọn mở RViz.
Output	Các node được chạy đồng bộ, hệ thống sẵn sàng nhận dữ liệu cảm biến và thực hiện nhiệm vụ.
File tiêu biểu	<code>sim_demo.launch.py</code> , <code>slam_demo.launch.py</code> , <code>slam_toolbox_tb4.yaml</code> , <code>tb4_debug.rviz</code> .
Vai trò	Giảm thao tác thủ công và bảo đảm hệ thống khởi động ổn định.



Hình 3.2 Vai trò của package `tb4_bringup` trong quá trình khởi động hệ thống

3.2.2 Package `tb4_vision_oak`

`tb4_vision_oak` là package nhận diện đối tượng từ ảnh RGB của camera OAK-D. Package này nhận ảnh màu, đưa ảnh vào mô hình YOLOv8n và xuất ra danh sách đối tượng được phát hiện dưới dạng detection 2D.

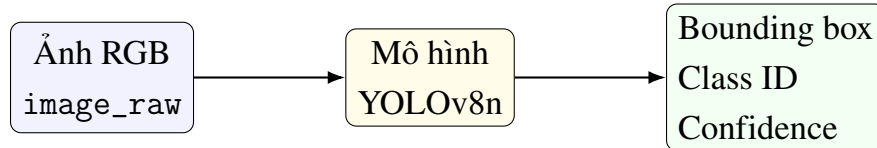
Bảng 3.3 Input, output và vai trò của package `tb4_vision_oak`

Mục	Nội dung
Chức năng chính	Nhận ảnh RGB, chạy YOLOv8n và publish kết quả detection 2D.
Input	Topic ảnh RGB <code>/oakd/rgb/preview/image_raw</code> .
Output	Topic <code>/vision/detected_objects</code> ; có thể có ảnh debug <code>/vision/debug_image</code> .
Dữ liệu xử lý	Bounding box, class id, confidence score.
Vai trò	Cung cấp thông tin ngữ nghĩa cho hệ thống, giúp robot biết mục tiêu là đối tượng nào.

Một detection thứ i có thể biểu diễn bởi:

$$D_i = (u_i, v_i, w_i, h_i, c_i, s_i) \quad (3.2)$$

Trong đó, (u_i, v_i) là tâm bounding box, w_i và h_i là kích thước bounding box, c_i là nhãn lớp, còn s_i là độ tin cậy của detection. Dữ liệu này vẫn ở dạng 2D nên cần package định vị mục tiêu để chuyển sang vị trí 3D.



Hình 3.3 Luồng xử lý ảnh RGB qua YOLOv8n trong package `tb4_vision_oak`

3.2.3 Package `tb4_object_localization`

`tb4_object_localization` là package chuyển kết quả nhận diện 2D thành vị trí mục tiêu trong không gian 3D. Package này kết hợp bounding box từ YOLOv8, ảnh depth và thông số nội tại camera.

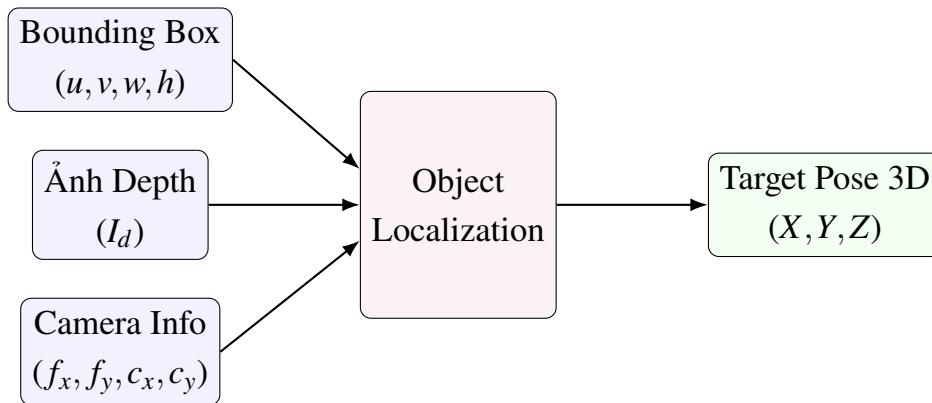
Bảng 3.4 Input, output và vai trò của package `tb4_object_localization`

Mục	Nội dung
Chức năng chính	Tính vị trí 3D của mục tiêu từ detection 2D và ảnh depth.
Input	<code>/vision/detected_objects</code> , <code>/oakd/rgb/preview/depth</code> , <code>/oakd/rgb/preview/camera_info</code> .
Output	<code>/target_object_pose_map</code> , <code>/target_object_marker</code> .
Dữ liệu xử lý	Tâm bbox, giá trị depth, ma trận nội tại camera, frame cảm biến.
Vai trò	Chuyển thông tin “thấy đối tượng trên ảnh” thành “mục tiêu nằm ở vị trí nào”.

Nếu tâm bounding box là (u, v) và độ sâu tại vùng lân cận tâm bbox là Z , tọa độ 3D trong hệ camera được tính theo mô hình camera lỗ kim:

$$X_c = \frac{(u - c_x)Z}{f_x}, \quad Y_c = \frac{(v - c_y)Z}{f_y}, \quad Z_c = Z \quad (3.3)$$

Trong đó, f_x , f_y , c_x và c_y được lấy từ topic camera info. Kết quả (X_c, Y_c, Z_c) biểu diễn vị trí mục tiêu so với camera. Sau đó, hệ thống dùng TF để chuyển pose này sang frame phù hợp cho navigation.

**Hình 3.4 Quá trình chuyển bounding box và ảnh depth thành pose 3D của mục tiêu**

3.2.4 Package `tb4_nav_patrol`

`tb4_nav_patrol` là package tạo hành vi tuần tra. Package này gửi lần lượt các waypoint đến Nav2 thông qua action `NavigateToPose`. Khi robot đến một waypoint, node tiếp tục gửi waypoint tiếp theo để tạo thành chu trình tuần tra.

Bảng 3.5 Input, output và vai trò của package `tb4_nav_patrol`

Mục	Nội dung
Chức năng chính	Gửi các điểm tuần tra cho Nav2.
Input	Danh sách waypoint, trạng thái bật/tắt patrol từ Mission Manager.
Output	Goal điều hướng gửi tới action <code>NavigateToPose</code> .
Dữ liệu xử lý	Tọa độ waypoint trong frame map.
Vai trò	Tạo hành vi mặc định cho robot khi chưa có mục tiêu cần tiếp cận.

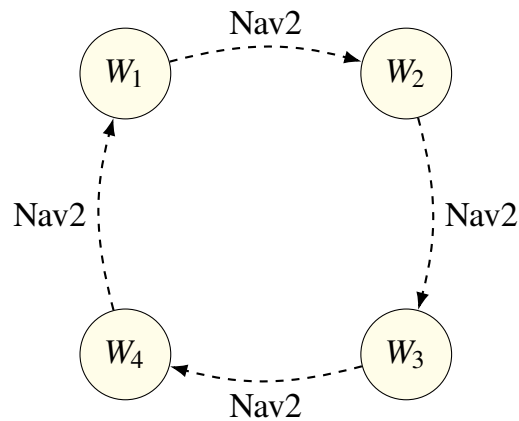
Một waypoint có thể biểu diễn bởi:

$$W_i = (x_i, y_i, \theta_i) \quad (3.4)$$

Danh sách các waypoint tuần tra được biểu diễn bởi:

$$\mathcal{W} = \{W_1, W_2, \dots, W_n\} \quad (3.5)$$

Trong đó, x_i và y_i là tọa độ waypoint trên bản đồ, còn θ_i là hướng mong muốn của robot tại điểm đó. Khi Mission Manager phát hiện mục tiêu hợp lệ, Patrol Node có thể bị tạm dừng để nhường quyền điều khiển cho nhiệm vụ tiếp cận mục tiêu.



Hình 3.5 Sơ đồ tuần tra waypoint của package `tb4_nav_patrol`

3.2.5 Package `tb4_mission_manager`

`tb4_mission_manager` là package điều phối hành vi cấp nhiệm vụ. Package này không xử lý ảnh trực tiếp và không điều khiển động cơ trực tiếp. Vai trò chính của nó

là quyết định thời điểm robot tiếp tục tuần tra, thời điểm tạm dừng tuần tra và thời điểm gửi goal tiếp cận mục tiêu cho Nav2.

Bảng 3.6 Input, output và vai trò của package tb4_mission_manager

Mục	Nội dung
Chức năng chính	Điều phối giữa trạng thái tuần tra và trạng thái tiếp cận mục tiêu.
Input	Pose mục tiêu, TF giữa camera và map, pose robot, trạng thái Nav2.
Output	Tín hiệu bật/tắt patrol; goal tiếp cận an toàn gửi đến Nav2.
Trạng thái chính	patrolling, approaching_target.
Vai trò	Liên kết perception với navigation ở mức hành vi.

Giả sử vị trí robot và mục tiêu trong frame map lần lượt là:

$$\mathbf{p}_r = \begin{bmatrix} x_r \\ y_r \end{bmatrix}, \quad \mathbf{p}_t = \begin{bmatrix} x_t \\ y_t \end{bmatrix} \quad (3.6)$$

Vector từ robot đến mục tiêu được tính bởi:

$$\Delta \mathbf{p} = \mathbf{p}_t - \mathbf{p}_r = \begin{bmatrix} x_t - x_r \\ y_t - y_r \end{bmatrix} \quad (3.7)$$

Khoảng cách phẳng giữa robot và mục tiêu là:

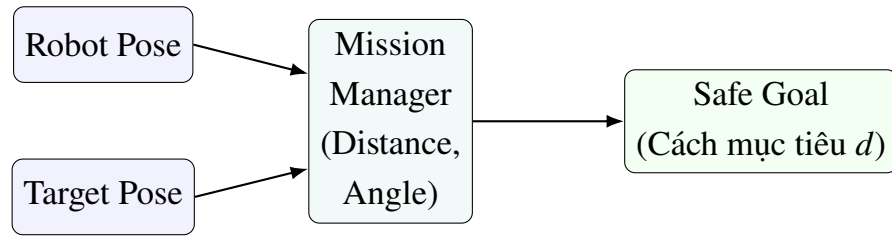
$$d = \|\Delta \mathbf{p}\|_2 = \sqrt{(x_t - x_r)^2 + (y_t - y_r)^2} \quad (3.8)$$

Nếu d_s là khoảng cách an toàn, điểm goal tiếp cận an toàn được chọn trên đoạn thẳng từ robot đến mục tiêu:

$$\mathbf{p}_g = \mathbf{p}_r + \frac{d - d_s}{d} \Delta \mathbf{p}, \quad d > d_s \quad (3.9)$$

Khi $d \leq d_s$, robot đã đủ gần mục tiêu nên Mission Manager không cần gửi goal tiếp cận mới. Góc quay tại điểm goal có thể chọn sao cho robot hướng về mục tiêu:

$$\theta_g = \text{atan2}(y_t - y_g, x_t - x_g) \quad (3.10)$$



Hình 3.6 Sơ đồ Mission Manager tính safe goal từ pose robot và pose mục tiêu

3.3 Luồng dữ liệu chính của hệ thống

Luồng dữ liệu của hệ thống có thể chia thành ba nhánh: nhánh định vị và bản đồ, nhánh nhận diện và định vị mục tiêu, nhánh điều khiển chuyển động. Ba nhánh này gặp nhau tại Mission Manager và Nav2.

Bảng 3.7 Các bước xử lý trong luồng dữ liệu chính của hệ thống

Bước	Khối xử lý	Dữ liệu/chức năng
1	Gazebo/Ignition	Sinh dữ liệu mô phỏng của robot, LiDAR, camera RGB-D và odometry.
2	LiDAR, odometry, TF	Cung cấp dữ liệu cho SLAM/localization và Nav2.
3	Camera RGB-D	Cung cấp ảnh RGB, ảnh depth và camera info cho nhận diện và định vị mục tiêu.
4	YOLOv8	Tạo detection 2D từ ảnh RGB.
5	Object Localization	Kết hợp detection, depth và camera info để tạo pose mục tiêu.
6	Mission Manager	Quyết định dừng patrol hay gửi goal tiếp cận mục tiêu.
7	Nav2	Lập kế hoạch đường đi và tạo lệnh vận tốc /cmd_vel.
8	Differential Drive	Thực thi chuyển động của robot trong môi trường.

Có thể biểu diễn dữ liệu đi qua hệ thống theo chuỗi:

$$I_{\text{rgb}}, I_d, \mathbf{K} \longrightarrow D_i \longrightarrow \mathbf{p}_{\text{target}} \longrightarrow \mathbf{p}_{\text{goal}} \longrightarrow \mathbf{u} \quad (3.11)$$

Trong đó, I_{rgb} là ảnh màu, I_d là ảnh depth, $\mathbf{K}$ là ma trận nội tại camera, D_i là detection, $\mathbf{p}_{\text{target}}$ là pose mục tiêu, $\mathbf{p}_{\text{goal}}$ là goal gửi cho Nav2 và $\mathbf{u}$ là lệnh điều khiển vận tốc.



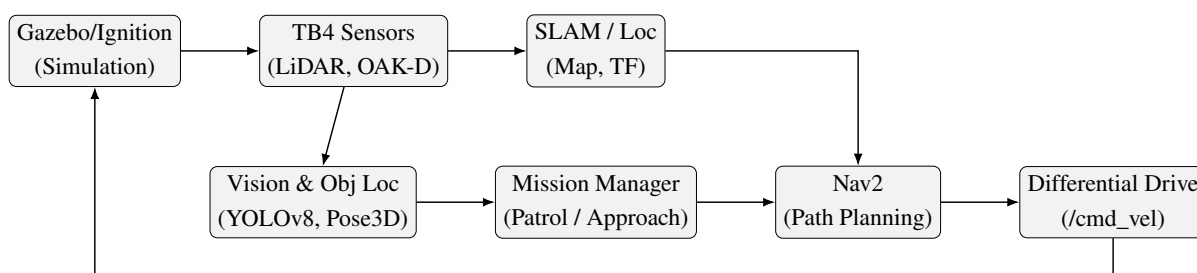
Hình 3.7 Luồng dữ liệu chính từ cảm biến đến lệnh điều khiển robot

3.4 Sơ đồ khối kiến trúc hệ thống

Sơ đồ khối của hệ thống nên thể hiện các module và hướng truyền dữ liệu giữa chúng. Không cần đưa quá nhiều chi tiết nội bộ của từng thuật toán trong sơ đồ này vì các nguyên lý cảm biến đã được trình bày ở Chương 2. Sơ đồ chỉ cần làm rõ package nào nhận dữ liệu nào và gửi kết quả đến đâu.

Bảng 3.8 Input và output của các khối trong hệ thống TurtleBot4

Khối	Dữ liệu vào	Dữ liệu ra
Gazebo/Ignition	Mô hình robot, world warehouse	Dữ liệu cảm biến mô phỏng, /clock.
TurtleBot4 Sensors	Môi trường mô phỏng	/scan, /odom, RGB image, depth image, camera info.
SLAM/Localization	/scan, /odom, TF	Map, pose robot, transform map → odom.
YOLOv8 Detection	RGB image	/vision/detected_objects.
Object Localization	Detection, depth, camera info	Pose mục tiêu, marker RViz.
Patrol Node	Waypoint	Goal tuần tra đến Nav2.
Mission Manager	Pose mục tiêu, pose robot, trạng thái patrol	Safe goal, tín hiệu bật/tắt patrol.
Nav2	Map, pose robot, goal	/cmd_vel.
Differential Drive	/cmd_vel	Chuyển động của robot.



Hình 3.8 Sơ đồ khối kiến trúc module của hệ thống TurtleBot4

3.5 Môi trường mô phỏng

Hệ thống sử dụng Gazebo/Ignition để mô phỏng robot và môi trường warehouse. Việc mô phỏng giúp kiểm tra pipeline cảm biến, nhận diện, định vị và điều hướng trước khi triển khai lên robot thật. Trong mô phỏng, TurtleBot4 Lite được spawn vào môi trường với vị trí ban đầu đã định nghĩa.

Bảng 3.9 Các thành phần trong môi trường mô phỏng

Thành phần mô phỏng	Vai trò
World warehouse	Mô phỏng không gian trong nhà, hành lang, kệ hàng hoặc vật cản.
TurtleBot4 Lite	Mô hình robot di động vi sai.
LiDAR mô phỏng	Tạo dữ liệu scan dùng cho SLAM và Nav2.
Camera RGB-D mô phỏng	Tạo ảnh RGB, depth và camera info.
Odometry mô phỏng	Cung cấp ước lượng chuyển động của robot.
/clock	Đồng bộ thời gian mô phỏng cho các node ROS2.



Hình 3.9 Môi trường mô phỏng warehouse và robot TurtleBot4 Lite trong Gazebo/Ignition

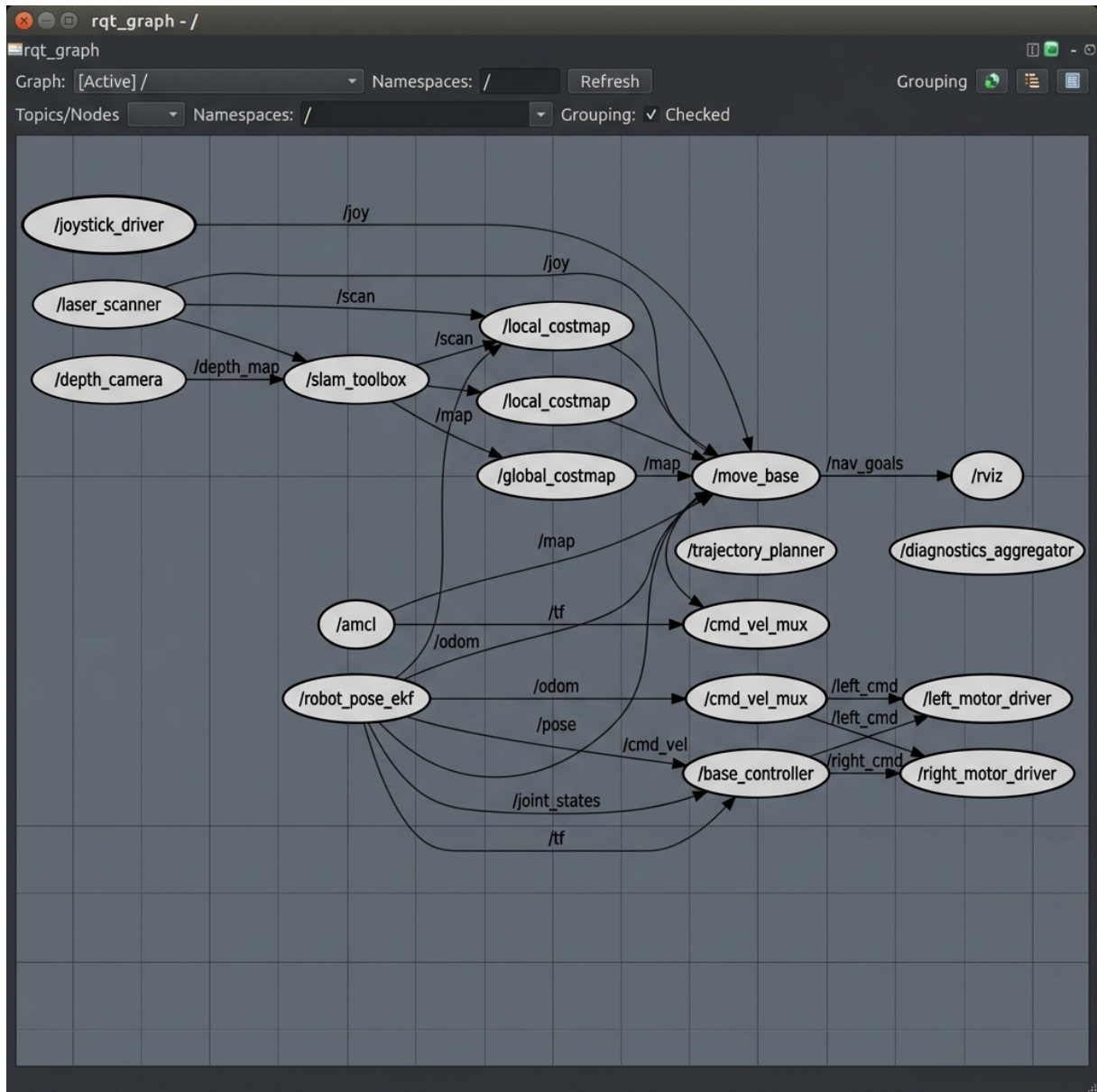
3.6 Cấu hình launch tổng hợp

Launch file tổng hợp giúp đưa toàn bộ hệ thống vào trạng thái hoạt động chỉ bằng một lệnh. Về mặt kiến trúc, đây là thành phần quan trọng vì nó giảm lỗi thao tác, thống

nhất tham số và bảo đảm các node phụ thuộc nhau được khởi động hợp lý.

Bảng 3.10 Các thành phần được khởi chạy bởi launch file tổng hợp

Thành phần được launch	Mục đích
Localization hoặc SLAM	Tạo bản đồ hoặc định vị robot trên bản đồ.
Nav2	Nhận goal và điều hướng robot.
YOLO detection	Nhận diện mục tiêu từ ảnh RGB.
Object localization	Tính pose mục tiêu từ detection và depth.
Patrol Node	Gửi waypoint tuần tra.
Mission Manager	Điều phối hành vi tuần tra và tiếp cận.
RViz	Hiển thị map, scan, TF, marker và trạng thái robot.



Hình 3.10 Quan hệ giữa launch file tổng hợp và các node được khởi chạy

3.7 Đánh giá kiến trúc hệ thống

Kiến trúc của hệ thống có ưu điểm là rõ ràng và dễ mở rộng. Mỗi package có một nhiệm vụ cụ thể, do đó khi cần thay đổi một chức năng, ta có thể thay package tương ứng mà không phải viết lại toàn bộ hệ thống. Ví dụ, có thể thay YOLOv8n bằng mô hình nhận diện khác mà vẫn giữ nguyên Nav2, Patrol Node và Mission Manager.

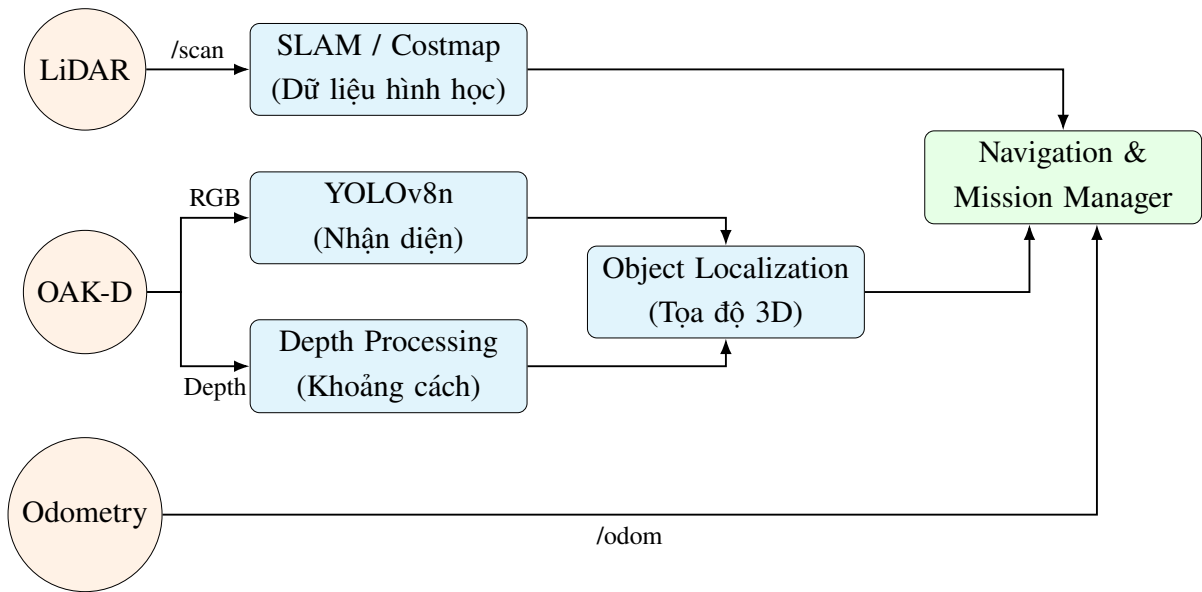
Tuy nhiên, kiến trúc này cũng phụ thuộc mạnh vào TF và đồng bộ thời gian. Nếu dữ liệu camera, depth hoặc TF không khớp thời điểm, pose mục tiêu có thể bị sai. Nếu Nav2 chưa sẵn sàng hoặc bản đồ chưa được load đúng, Mission Manager có thể gửi goal nhưng robot không di chuyển. Vì vậy, khi triển khai thực tế cần kiểm tra đầy đủ các topic, frame và trạng thái lifecycle của Nav2.

3.8 Kết luận chương

Chương này đã trình bày kiến trúc hệ thống robot tự hành TurtleBot4 theo hướng module hóa trên ROS2. Các package chính gồm tb4_bringup, tb4_vision_oak, tb4_object_localization, tb4_nav_patrol và tb4_mission_manager. Mỗi package đảm nhiệm một phần trong chuỗi xử lý từ cảm biến, nhận diện, định vị mục tiêu, điều phối hành vi đến điều hướng robot. Cách thiết kế này giúp hệ thống rõ ràng, dễ kiểm thử và phù hợp với bài toán mô phỏng robot tự hành sử dụng ROS2, SLAM, Nav2 và YOLOv8.

4. HỆ THỐNG CẢM BIẾN VÀ XỬ LÝ DỮ LIỆU

Chương này phân tích lớp cảm biến và lớp xử lý dữ liệu của hệ thống robot tự hành TurtleBot4. Khác với Chương 2, nội dung không đi sâu lại vào lý thuyết chung của từng cảm biến mà tập trung vào cách các nguồn dữ liệu được sử dụng trong project: LiDAR cung cấp dữ liệu hình học cho SLAM và costmap; camera RGB-D OAK-D cung cấp ảnh màu, ảnh chiều sâu và thông số nội tại; YOLOv8n xử lý ảnh RGB để phát hiện đối tượng; Object Localization kết hợp bounding box, depth và camera intrinsic để suy ra vị trí 3D; TF/TF2 dùng để đưa dữ liệu về cùng hệ tọa độ phục vụ điều hướng.



Hình 4.1 Sơ đồ tổng quan pipeline cảm biến và xử lý dữ liệu trong TurtleBot4

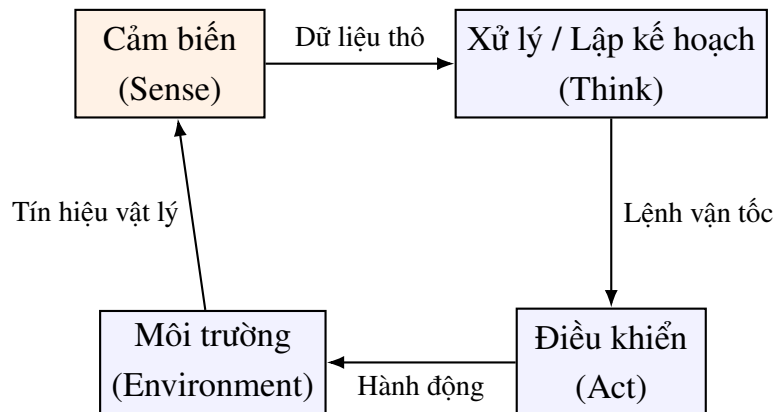
4.1 Vai trò của cảm biến trong đề tài

Trong hệ robot tự hành, cảm biến là nguồn thông tin đầu vào quyết định robot biết gì về môi trường và trạng thái của chính nó. Nếu thiếu cảm biến, robot chỉ có thể chạy theo lệnh cố định mà không biết vật cản, vị trí bản thân hay vị trí mục tiêu. Trong project TurtleBot4, cảm biến được dùng cho ba nhóm nhiệm vụ chính: nhận biết không gian xung quanh, nhận diện mục tiêu và hỗ trợ điều hướng.

Bảng 4.1 Các nhóm dữ liệu cảm biến sử dụng trong project

Nhóm dữ liệu	Nguồn dữ liệu	Mục đích sử dụng trong project
Dữ liệu hình học	LiDAR /scan	Xây dựng bản đồ, tránh vật cản và tạo costmap cho Nav2.
Dữ liệu chuyển động	Odometry /odom	Ước lượng chuyển động tương đối của robot theo thời gian.
Dữ liệu thị giác	Ảnh RGB từ OAK-D	Đưa vào YOLOv8n để phát hiện người hoặc đối tượng mục tiêu.
Dữ liệu chiều sâu	Depth image từ OAK-D	Tính khoảng cách từ camera đến vùng chứa mục tiêu.
Dữ liệu hình học hệ tọa độ	TF/TF2	Chuyển đổi pose giữa camera, base_link, odom và map.

Luồng xử lý cảm biến trong chương này có thể hiểu theo chuỗi: đo dữ liệu thô, kiểm tra dữ liệu, chuyển đổi dữ liệu sang dạng ROS2 topic, xử lý thành thông tin có ý nghĩa, rồi đưa thông tin này cho các module điều hướng và điều phối nhiệm vụ.



Hình 4.2 Vị trí của lớp cảm biến trong vòng kín cảm biến – xử lý – điều khiển – chấp hành

4.2 LiDAR và dữ liệu /scan

4.2.1 Chức năng của LiDAR trong hệ thống

LiDAR là cảm biến chính phục vụ nhận biết hình học xung quanh robot. Trong project, dữ liệu LiDAR được publish dưới dạng topic /scan. Topic này được SLAM Toolbox dùng để xây dựng bản đồ và được Nav2 dùng để cập nhật costmap, từ đó giúp robot tránh vật cản trong quá trình di chuyển.

Bảng 4.2 Vai trò của LiDAR trong hệ thống

Mục	Nội dung
Input của LiDAR	Môi trường xung quanh robot, gồm tường, kệ hàng, vật cản và bề mặt phản xạ laser.
Output của LiDAR	Bản tin LaserScan trên topic <code>/scan</code> .
Dạng dữ liệu	Mảng khoảng cách theo từng góc quét trong mặt phẳng ngang.
Module sử dụng	SLAM Toolbox, Nav2 costmap, RViz để hiển thị LaserScan.
Vai trò trong project	Tạo thông tin khoảng cách giúp robot lập bản đồ, định vị và tránh vật cản.

4.2.2 Cách hoạt động và cách tính dữ liệu LiDAR

Một phép đo LiDAR có thể được mô tả bằng cặp giá trị gồm góc quét và khoảng cách. Với mỗi hướng quét, cảm biến phát tia laser ra môi trường, nhận tín hiệu phản xạ quay lại, sau đó tính khoảng cách đến vật cản. Nếu dùng nguyên lý thời gian bay, khoảng cách có thể được tính theo công thức:

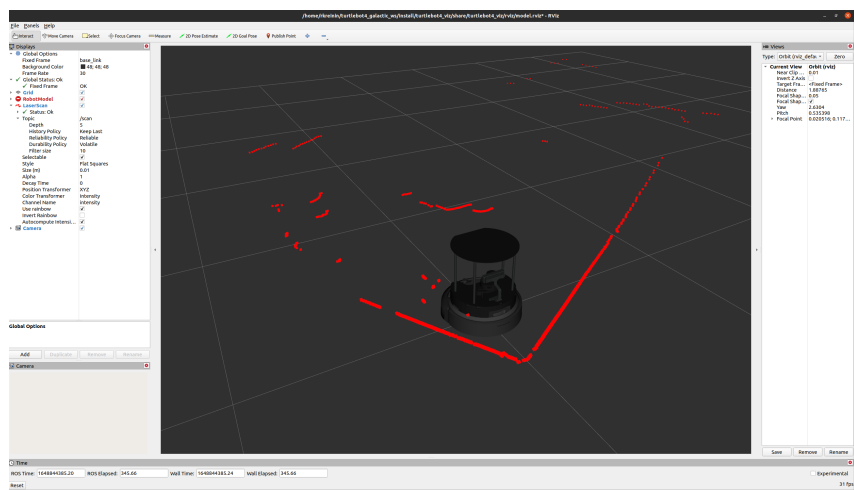
$$d = \frac{c \Delta t}{2} \quad (4.1)$$

Trong đó, d là khoảng cách từ cảm biến đến vật cản, c là tốc độ lan truyền của ánh sáng, Δt là thời gian từ lúc phát tia đến lúc nhận tín hiệu phản xạ. Hệ số 2 xuất hiện vì tia laser đi từ cảm biến đến vật cản rồi phản xạ quay lại cảm biến, tức là quãng đường đo được gấp đôi khoảng cách thực.

Trong bản tin LaserScan, mỗi phần tử khoảng cách tương ứng với một góc quét. Nếu gọi θ_i là góc của tia quét thứ i , khoảng cách đo được là r_i , tọa độ điểm vật cản trong hệ tọa độ LiDAR có thể biểu diễn là:

$$x_i = r_i \cos(\theta_i), \quad y_i = r_i \sin(\theta_i) \quad (4.2)$$

Công thức trên biến dữ liệu cực của LiDAR thành điểm trong mặt phẳng hai chiều. Các điểm này được dùng để tạo hình dạng tường, vật cản và vùng trống trong bản đồ occupancy grid. Trong cấu hình của đề tài, các tham số quan trọng gồm `resolution = 0.05 m`, `min_laser_range = 0.2 m` và `max_laser_range = 12.0 m`. Các giá trị nằm ngoài dải đo hợp lệ sẽ bị loại bỏ hoặc không được dùng trong SLAM và costmap.



Hình 4.3 Minh họa LiDAR 2D quét môi trường và tạo tập điểm khoảng cách

4.2.3 Kiểm tra và lỗi thường gặp của dữ liệu /scan

Trong mô phỏng, dữ liệu /scan có thể bất thường nếu frame LiDAR đặt sai, mô hình collision che tia laser, hoặc topic chưa được bridge đúng giữa Gazebo/Ignition và ROS2. Khi /scan có giá trị gần như cố định hoặc không thay đổi theo môi trường, robot có thể xây dựng bản đồ sai, costmap sai và điều hướng không ổn định.

Bảng 4.3 Một số lỗi thường gặp của dữ liệu /scan

Hiện tượng	Nguyên nhân có thể	Cách kiểm tra
Không có dữ liệu /scan	LiDAR chưa spawn, bridge chưa hoạt động hoặc sai namespace.	Chạy <code>ros2 topic echo /scan -once</code> và kiểm tra danh sách topic.
Khoảng cách gần như cố định	LiDAR bị che bởi mô hình robot hoặc collision không đúng.	Quan sát LaserScan trong RViz và kiểm tra frame <code>rplidar_link</code> .
Scan lệch hướng	Sai quan hệ TF giữa <code>rplidar_link</code> và <code>base_link</code> .	Hiển thị TF trong RViz và kiểm tra trục tọa độ.
SLAM tạo bản đồ méo	Odometry, scan hoặc TF không đồng bộ.	Kiểm tra <code>/odom</code> , <code>/scan</code> , <code>/clock</code> và transform map <code>-> odom -> base_link</code> .

4.3 Camera RGB-D OAK-D

4.3.1 Chức năng của camera RGB-D trong project

Camera RGB-D OAK-D cung cấp hai loại thông tin bổ sung cho nhau. Ảnh RGB chứa thông tin màu sắc và hình dạng, phù hợp cho nhận diện đối tượng bằng YOLOv8n. Ảnh depth chứa khoảng cách của từng điểm ảnh đến camera, giúp hệ thống xác định mục tiêu ở gần hay xa. Ngoài ra, bản tin camera_info cung cấp các tham số nội tại của camera để chuyển đổi từ điểm ảnh 2D sang tọa độ 3D.

Bảng 4.4 Các topic liên quan đến camera RGB-D và định vị mục tiêu

Topic	Loại dữ liệu	Ý nghĩa trong project
/oakd/rgb/preview/ image_raw	Ảnh RGB	Đầu vào cho node YOLOv8n để phát hiện đối tượng.
/oakd/rgb/preview/ depth	Ảnh chiều sâu	Cung cấp khoảng cách Z tại vùng chứa mục tiêu.
/oakd/rgb/preview/ camera_info	Thông số nội tại camera	Cung cấp f_x , f_y , c_x , c_y để tính tọa độ 3D.
/vision/ detected_objects	Detection- 2DArray	Output của YOLOv8n, gồm bounding box, class id và confidence.
/target_object_pose_map	PoseStamped	Output của bước định vị, biểu diễn pose mục tiêu để Mission Manager sử dụng.
/target_object_marker	Marker	Dùng để hiển thị vị trí mục tiêu trong RViz.

4.3.2 Thành phần dữ liệu của camera RGB-D

Bảng 4.5 Thành phần dữ liệu của camera RGB-D

Thành phần	Chức năng
Camera RGB	Thu ảnh màu của môi trường, giúp nhận diện người hoặc đối tượng bằng mô hình học sâu.
Cảm biến depth hoặc mô phỏng depth	Tạo ảnh chiều sâu, trong đó mỗi pixel biểu diễn khoảng cách từ camera đến bề mặt quan sát được.
CameraInfo	Lưu thông số nội tại như f_x , f_y , c_x , c_y và kích thước ảnh.
Node xử lý ảnh	Nhận ảnh, chuyển đổi định dạng và đưa vào mô hình nhận diện.
Giao tiếp ROS2	Publish/subscribe dữ liệu camera qua các topic ROS2 để các node khác sử dụng.

4.3.3 Cách tính tọa độ 3D từ ảnh RGB-D

Sau khi YOLOv8n phát hiện đối tượng, hệ thống lấy tâm bounding box trên ảnh RGB. Giả sử tâm bounding box là điểm ảnh có tọa độ (u, v) . Từ ảnh depth, lấy giá trị độ sâu tại vùng lân cận điểm này. Để giảm nhiễu, hệ thống không nên lấy một pixel duy nhất mà nên lấy trung vị của một vùng nhỏ xung quanh tâm bbox. Gọi giá trị depth hợp lệ là Z .

Theo mô hình camera lỗ kim, điểm ảnh 2D có thể được chuyển sang tọa độ 3D trong hệ camera bằng các công thức:

$$X_c = \frac{(u - c_x)Z}{f_x}, \quad (4.3)$$

$$Y_c = \frac{(v - c_y)Z}{f_y}, \quad (4.4)$$

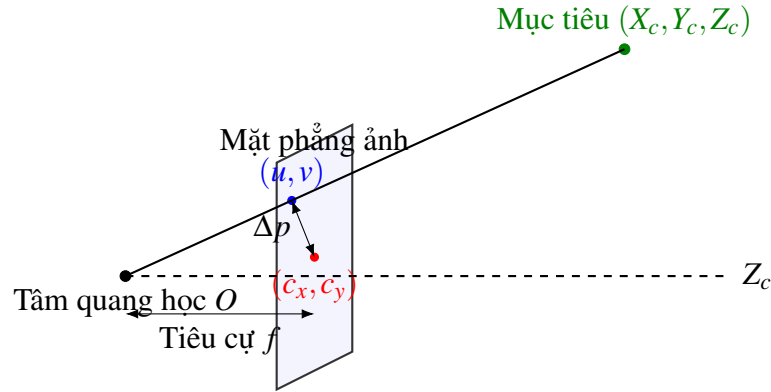
$$Z_c = Z. \quad (4.5)$$

Trong đó, u , v là tọa độ pixel của tâm bbox; Z là độ sâu tại vùng tâm bbox; f_x và f_y là tiêu cự theo đơn vị pixel; c_x và c_y là tọa độ tâm ảnh. Nếu $u = c_x$ và $v = c_y$, mục tiêu nằm gần trục quang học của camera. Nếu u lệch khỏi c_x , mục tiêu lệch ngang trong ảnh; nếu v lệch khỏi c_y , mục tiêu lệch theo phương dọc ảnh.

Do quy ước trục của camera khác quy ước trục của robot trong ROS, tọa độ trong hệ camera cần được ánh xạ sang quy ước ROS như sau:

$$x_{\text{ros}} = Z_c, \quad y_{\text{ros}} = -X_c, \quad z_{\text{ros}} = -Y_c \quad (4.6)$$

Cách ánh xạ này giúp hướng phía trước camera tương ứng với trục x của robot trong ROS. Kết quả cuối cùng là một pose mục tiêu tương đối so với camera hoặc so với frame đã được biến đổi bằng TF.



Hình 4.4 Mô hình camera pinhole và cách chuyển điểm ảnh RGB-D sang tọa độ 3D

4.4 Nhận diện đối tượng bằng YOLOv8n

4.4.1 Vai trò của YOLOv8n

YOLOv8n là module nhận diện đối tượng trong project. Node `tb4_vision_oak` nhận ảnh RGB từ camera OAK-D, chạy mô hình YOLOv8n, sau đó xuất ra danh sách đối tượng phát hiện được. Kết quả này không trực tiếp điều khiển robot mà được chuyển cho Object Localization để kết hợp với depth.

Bảng 4.6 Input, output và vai trò của YOLOv8n

Mục	Nội dung
Input	Ảnh RGB từ <code>/oakd/rgb/preview/image_raw</code> .
Xử lý	Chạy inference bằng mô hình YOLOv8n, lọc detection theo confidence.
Output	Detection2DArray trên <code>/vision/detected_objects</code> .
Thông tin trong output	Tâm bbox, kích thước bbox, class id, confidence.
Ứng dụng trong project	Phát hiện mục tiêu, ưu tiên class person nếu xuất hiện.

4.4.2 Dữ liệu detection và tiêu chí chọn mục tiêu

Mỗi detection có thể được biểu diễn bởi một bounding box và thông tin phân loại. Gọi bounding box có tâm là (u, v) , chiều rộng w và chiều cao h . Vùng bbox được mô tả bởi:

$$B = (u, v, w, h, \text{class_id}, \text{conf}) \quad (4.7)$$

Trong đó, class_id cho biết lớp đối tượng, còn conf là độ tin cậy của dự đoán. Nếu có nhiều detection, hệ thống có thể áp dụng quy tắc chọn mục tiêu như sau: ưu tiên đối tượng thuộc lớp person; nếu không có person, chọn detection có confidence cao nhất; nếu confidence thấp hơn ngưỡng, không tạo mục tiêu.

$$B_{\text{target}} = \arg \max_i \text{conf}_i, \quad \text{conf}_i \geq \tau \quad (4.8)$$

Trong công thức trên, τ là ngưỡng confidence. Trong project, ngưỡng mặc định là 0.5. Việc dùng ngưỡng giúp loại bỏ các phát hiện không chắc chắn, tránh việc Mission Manager phản ứng sai với nhiễu hoặc vật thể không mong muốn.



Hình 4.5 Minh họa YOLOv8n phát hiện bounding box trên ảnh RGB

4.5 Định vị mục tiêu từ RGB-D

4.5.1 Quy trình định vị

Định vị mục tiêu là bước biến thông tin nhận diện 2D thành vị trí 3D có thể dùng cho điều hướng. Nếu chỉ có YOLOv8n, robot chỉ biết mục tiêu nằm ở đâu trong ảnh. Nếu chỉ có depth, robot không biết vùng nào là mục tiêu. Vì vậy, cần kết hợp bbox từ YOLOv8n với ảnh depth và thông số camera.

Bảng 4.7 Quy trình định vị mục tiêu từ dữ liệu RGB-D

Bước	Dữ liệu sử dụng	Kết quả
1	Detection2DArray	Chọn bbox mục tiêu.
2	Ảnh depth	Lấy độ sâu Z tại vùng tâm bbox.
3	CameraInfo	Lấy f_x, f_y, c_x, c_y .
4	Công thức pinhole	Tính tọa độ 3D trong hệ camera.
5	TF/TF2	Chuyển pose sang frame phù hợp như map hoặc base_link.
6	ROS2 topic	Publish /target_object_pose_map và /target_object_marker.

4.5.2 Lọc depth tại vùng bbox

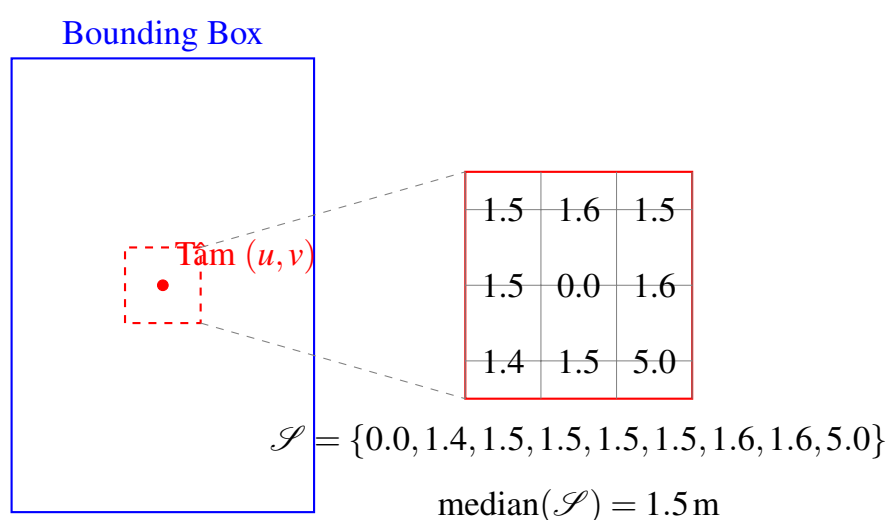
Ảnh depth thường có nhiễu, điểm mất dữ liệu hoặc giá trị không hợp lệ tại biên vật thể. Vì vậy, thay vì lấy đúng một pixel tại tâm bbox, node định vị nên lấy một cửa sổ nhỏ quanh tâm bbox và tính trung vị. Gọi tập giá trị depth hợp lệ trong cửa sổ là $\mathcal{S}$. Giá trị depth dùng để định vị được tính là:

$$Z = \text{median}(\mathcal{S}) \quad (4.9)$$

Cách dùng trung vị giúp giảm ảnh hưởng của các điểm bất thường. Ví dụ, nếu trong cửa sổ có một vài pixel lỗi bằng 0 hoặc quá lớn, giá trị trung vị vẫn thường ổn định hơn giá trị trung bình hoặc một pixel đơn lẻ.

Bảng 4.8 Cách xử lý một số trường hợp depth bất thường

Loại giá trị depth	Cách xử lý đề xuất
$Z = 0$ hoặc không hợp lệ	Bỏ qua detection, không publish pose mục tiêu.
Z quá nhỏ	Có thể là vật thể quá gần hoặc lỗi đo; cần kiểm tra ngưỡng an toàn.
Z quá lớn	Có thể nằm ngoài tầm quan sát hữu ích; không nên dùng để tiếp cận.
Z ổn định trong nhiều frame	Có thể dùng để tạo pose mục tiêu tin cậy hơn.



Hình 4.6 Minh họa lấy depth trung vị trong cửa sổ quanh tâm bounding box

4.5.3 Input, output của node *object_localization_node*

Bảng 4.9 Input và output của node *object_localization_node*

Loại	Topic/Dữ liệu	Chức năng
Input	/vision/detected_objects	Nhận bbox, class id và confidence từ YOLOv8n.
Input	/oakd/rgb/preview/depth	Lấy giá trị khoảng cách đến vùng mục tiêu.
Input	/oakd/rgb/preview/camera_info	Lấy thông số nội tại camera.
Output	/target_object_pose_map	Publish pose mục tiêu để Mission Manager sử dụng.
Output	/target_object_marker	Publish marker để hiển thị mục tiêu trong RViz.

4.6 TF và hợp nhất thông tin cảm biến

4.6.1 Vai trò của TF/TF2

TF/TF2 là lớp liên kết hình học giữa các frame trong ROS2. Mỗi cảm biến có thể đo dữ liệu trong hệ tọa độ riêng. Camera đo mục tiêu trong frame camera, LiDAR đo vật cản trong frame *rplidar_link*, odometry mô tả robot trong frame *odom*, còn navigation thường cần pose trong frame *map*. Vì vậy, TF giúp đưa dữ liệu từ nhiều nguồn về cùng hệ quy chiếu.

Bảng 4.10 Các frame chính trong hệ thống

Frame	Ý nghĩa
map	Hệ tọa độ bản đồ toàn cục, dùng cho navigation goal.
odom	Hệ tọa độ odometry liên tục nhưng có thể trôi theo thời gian.
base_link	Hệ tọa độ gắn với thân robot.
rplidar_link	Hệ tọa độ gắn với LiDAR.
oakd_link	Hệ tọa độ gắn với camera OAK-D.

4.6.2 Công thức biến đổi pose giữa các frame

Một điểm trong hệ tọa độ camera có thể được chuyển sang hệ tọa độ map bằng ma trận biến đổi đồng nhất. Gọi điểm mục tiêu trong frame camera là $\mathbf{p}_{\text{camera}}$, điểm tương ứng trong frame map là $\mathbf{p}_{\text{map}}$, ta có:

$$\mathbf{p}_{\text{map}} = \mathbf{T}_{\text{map} \leftarrow \text{camera}} \mathbf{p}_{\text{camera}} \quad (4.10)$$

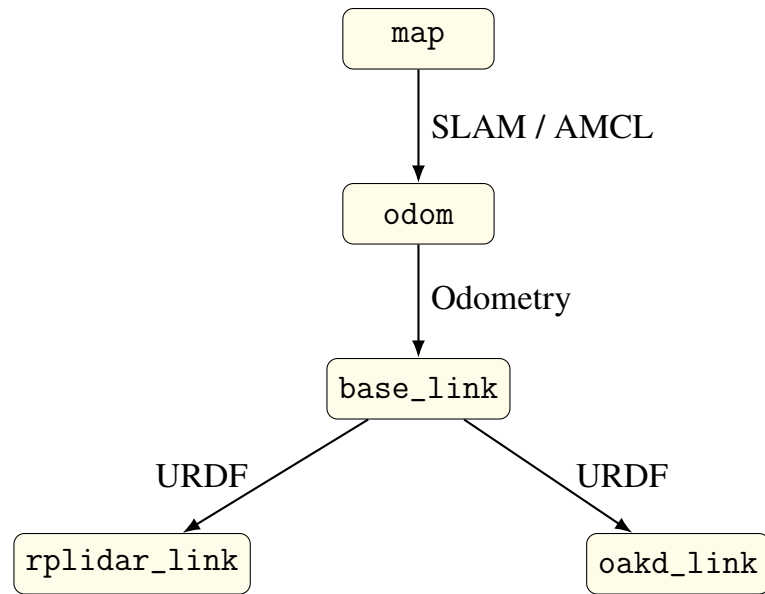
Với dạng tọa độ đồng nhất, điểm 3D được viết là:

$$\mathbf{p} = \begin{bmatrix} x & y & z & 1 \end{bmatrix}^T \quad (4.11)$$

Ma trận biến đổi gồm phần quay $\mathbf{R}$ và phần tịnh tiến $\mathbf{t}$:

$$\mathbf{T} = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0}_{1 \times 3} & 1 \end{bmatrix} \quad (4.12)$$

Trong thực tế triển khai ROS2, người lập trình không cần tự nhân ma trận nếu dùng `tf2_ros::Buffer` và hàm `lookup_transform`. Tuy nhiên, công thức trên cho thấy bản chất của TF là biến đổi hình học giữa các hệ tọa độ khác nhau.



Hình 4.7 Cây TF của TurtleBot4 gồm map, odom, base_link, rplidar_link và oakd_link

4.7 Đánh giá kỹ thuật hệ cảm biến

Hệ cảm biến của project có tính bổ sung rõ ràng. LiDAR ổn định cho thông tin hình học nhưng không biết ý nghĩa vật thể. Camera RGB có khả năng cung cấp thông tin ngữ nghĩa nhưng không đủ để tính khoảng cách nếu thiếu depth. Depth giúp xác định vị trí 3D nhưng dễ nhiễu ở biên vật thể hoặc bề mặt phản xạ kém. Odometry có tần số cao nhưng sai số tích lũy theo thời gian. TF không phải cảm biến vật lý nhưng là thành phần bắt buộc để hợp nhất dữ liệu từ nhiều nguồn.

Bảng 4.11 Đánh giá kỹ thuật các thành phần cảm biến và dữ liệu

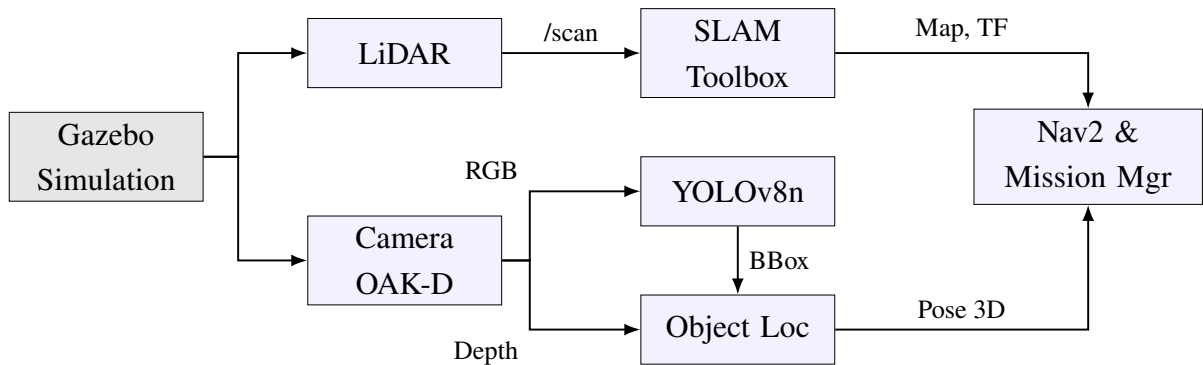
Thành phần	Điểm mạnh	Hạn chế	Vai trò trong project
LiDAR	Ổn định cho khoảng cách và vật cản.	Không cung cấp thông tin ngữ nghĩa.	SLAM, costmap và tránh vật cản.
Camera RGB	Cung cấp hình ảnh giàu thông tin.	Phụ thuộc ánh sáng và mô hình nhận diện.	Đầu vào cho YOLOv8n.
Depth image	Cung cấp khoảng cách đến mục tiêu.	Dễ nhiễu tại biên vật thể.	Tính vị trí 3D mục tiêu.
Odometry	Nhanh, liên tục.	Sai số tích lũy.	Hỗ trợ định vị và điều hướng.
TF/TF2	Liên kết các hệ tọa độ.	Phụ thuộc đồng bộ thời gian và cấu hình frame.	Chuyển pose mục tiêu sang frame map.

4.8 Luồng dữ liệu cảm biến trong project

Từ góc nhìn triển khai, luồng cảm biến có thể chia thành hai nhánh chính. Nhánh LiDAR và odometry phục vụ bản đồ, định vị và costmap. Nhánh camera RGB-D phục vụ nhận diện và định vị mục tiêu. Hai nhánh này gặp nhau ở TF và Mission Manager, nơi dữ liệu mục tiêu được chuyển thành quyết định điều hướng.

Bảng 4.12 Luồng dữ liệu cảm biến trong project

Bước	Khối xử lý	Input	Output
1	Gazebo/Ignition	Mô hình robot và môi trường warehouse.	Dữ liệu mô phỏng của LiDAR, camera và odometry.
2	LiDAR	Vật cản xung quanh robot.	Topic /scan.
3	Camera OAK-D	Cảnh quan sát phía trước robot.	Ảnh RGB, ảnh depth, camera_info.
4	YOLOv8n	Ảnh RGB.	/vision/detected_objects.
5	Object Localization	Detection, depth, camera_info.	/target_object_pose_map và marker RViz.
6	TF/TF2	Pose trong frame camera và cây TF.	Pose mục tiêu trong frame map.
7	Mission Manager/Nav2	Pose mục tiêu và bản đồ.	Goal tiếp cận và lệnh điều hướng.



Hình 4.8 Luồng dữ liệu cảm biến từ Gazebo đến Mission Manager và Nav2

4.9 Gợi ý kiểm tra khi chạy hệ thống

Để xác nhận pipeline cảm biến hoạt động đúng, có thể kiểm tra lần lượt các topic quan trọng. Nếu một topic ở đầu pipeline không có dữ liệu, các bước phía sau sẽ không thể hoạt động ổn định.

Bảng 4.13 Các topic cần kiểm tra khi chạy pipeline cảm biến

Topic cần kiểm tra	Ý nghĩa khi có dữ liệu ổn định
/scan	LiDAR hoạt động, có dữ liệu cho SLAM và costmap.
/odom	Robot có dữ liệu chuyển động tương đối.
/clock	Mô phỏng đang phát thời gian cho ROS2.
/oakd/rgb/preview/image_raw	Camera RGB có ảnh đầu vào cho YOLOv8n.
/oakd/rgb/preview/depth	Camera có ảnh chiều sâu để tính khoảng cách mục tiêu.
/oakd/rgb/preview/camera_info	Có thông số nội tại để chuyển đổi 2D sang 3D.
/vision/detected_objects	YOLOv8n phát hiện được đối tượng.
/target_object_pose_map	Object Localization đã tạo pose mục tiêu.
/target_object_marker	Có marker để quan sát mục tiêu trong RViz.

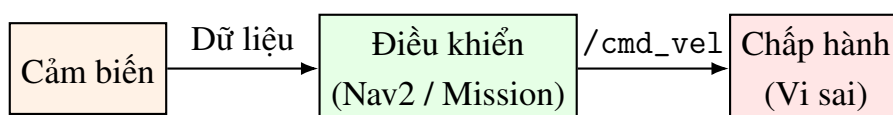
Nếu /vision/detected_objects có dữ liệu nhưng /target_object_pose_map không có, lỗi thường nằm ở depth hoặc camera_info. Nếu /target_object_pose_map có dữ liệu nhưng Mission Manager không gửi goal, cần kiểm tra TF giữa camera và map.

4.10 Kết luận chương

Chương này đã phân tích hệ thống cảm biến và xử lý dữ liệu của TurtleBot4 trong project. LiDAR cung cấp dữ liệu hình học cho SLAM và Nav2; camera RGB-D cung cấp ảnh màu, chiều sâu và thông số nội tại; YOLOv8n phát hiện đối tượng; Object Localization chuyển detection 2D thành pose 3D; TF hợp nhất dữ liệu theo các frame; các topic ROS2 đóng vai trò cầu nối giữa các node. Nhờ sự kết hợp này, robot có thể không chỉ di chuyển trong bản đồ mà còn nhận biết và tiếp cận mục tiêu trong môi trường mô phỏng.

5. CƠ CẤU CHẤP HÀNH VÀ ĐIỀU KHIỂN CHUYỂN ĐỘNG

Chương này phân tích lớp chấp hành và điều khiển chuyển động của hệ thống robot tự hành TurtleBot4. Chương 5 tập trung vào cách hệ thống biến thông tin nhận thức thành chuyển động vật lý. Nội dung gồm cấu trúc truyền động vi sai, mô hình động học, lệnh vận tốc `/cmd_vel`, vai trò của Nav2, Patrol Node, Mission Manager và mối liên hệ giữa cảm biến với cơ cấu chấp hành.



Hình 5.1 Tổng quan chuỗi cảm biến – điều khiển – cơ cấu chấp hành của TurtleBot4

5.1 Cấu trúc chấp hành của TurtleBot4

5.1.1 Khái niệm cơ cấu chấp hành trong robot tự hành

Cơ cấu chấp hành là phần tử biến tín hiệu điều khiển thành tác động vật lý lên môi trường. Trong robot di động, tác động vật lý quan trọng nhất là chuyển động của thân robot. Với TurtleBot4, cơ cấu chấp hành chính là hệ truyền động vi sai gồm hai bánh chủ động được điều khiển độc lập. Khi hai bánh quay cùng chiều và cùng tốc độ, robot đi thẳng; khi hai bánh quay khác tốc độ, robot quay; khi hai bánh quay ngược chiều, robot có thể quay gần như tại chỗ.

Trong project, các node cấp cao như Patrol Node hoặc Mission Manager không điều khiển trực tiếp động cơ. Các node này chỉ gửi mục tiêu hoặc yêu cầu di chuyển cho Nav2. Nav2 tạo lệnh vận tốc dạng `geometry_msgs/Twist` trên topic `/cmd_vel`, sau đó bộ điều khiển cấp thấp chuyển lệnh vận tốc thành vận tốc quay của từng bánh xe.

5.1.2 Thành phần và chức năng

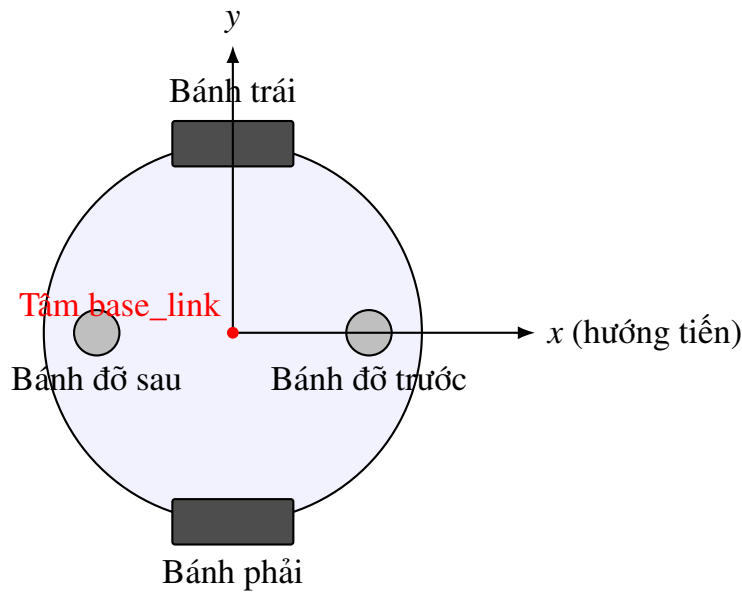
Bảng 5.1 Thành phần và chức năng của lớp chấp hành

Thành phần	Chức năng trong hệ chấp hành
Hai bánh chủ động trái/phải	Tạo lực kéo và quyết định hướng chuyển động của robot. Mỗi bánh có thể được điều khiển với tốc độ riêng.
Bánh tự do hoặc bánh đỡ	Giữ cân bằng cơ học cho thân robot, không trực tiếp tạo lực kéo.
Động cơ bánh xe	Biến tín hiệu điều khiển thành chuyển động quay của bánh.
Encoder bánh xe	Đo số xung quay của bánh, giúp ước lượng vận tốc và quãng đường di chuyển.
Bộ điều khiển cấp thấp	Nhận lệnh vận tốc từ <code>/cmd_vel</code> và chuyển thành tốc độ bánh trái, bánh phải.
Topic <code>/cmd_vel</code>	Kênh điều khiển vận tốc chính, chứa vận tốc tuyến tính và vận tốc góc của robot.

5.1.3 Input và output của lớp chấp hành

Bảng 5.2 Input và output của lớp chấp hành

Mục	Nội dung
Input	Lệnh <code>geometry_msgs/Twist</code> trên topic <code>/cmd_vel</code> . Thành phần thường dùng là <code>linear.x = v</code> và <code>angular.z = omega</code> .
Xử lý	Bộ điều khiển cấp thấp chuyển đổi vận tốc robot thành vận tốc từng bánh theo mô hình robot vi sai.
Output vật lý	Robot di chuyển tịnh tiến, quay, hoặc vừa tiến vừa quay trong mặt phẳng.
Output dữ liệu phản hồi	Odometry <code>/odom</code> , TF <code>odom -> base_link</code> , vận tốc thực tế hoặc trạng thái chuyển động của robot.



Hình 5.2 Cấu tạo truyền động vi sai của TurtleBot4 gồm hai bánh chủ động và bánh đỡ

5.2 Mô hình động học robot vi sai

5.2.1 Trạng thái và giả thiết chuyển động

TurtleBot4 được mô hình hóa là robot vi sai chuyển động trên mặt phẳng. Trạng thái của robot tại thời điểm bất kỳ được mô tả bởi ba đại lượng: vị trí theo trục x , vị trí theo trục y và góc hướng của thân robot so với hệ tọa độ bản đồ.

$$\mathbf{q} = \begin{bmatrix} x & y & \theta \end{bmatrix}^T \quad (5.1)$$

Trong đó, x và y là tọa độ của robot trên mặt phẳng, θ là góc hướng của robot. Mô hình này giả thiết robot chuyển động trên mặt phẳng, hai bánh chủ động tiếp xúc với mặt sàn, không xét trượt bánh lớn và không xét chuyển động theo phương thẳng đứng.

5.2.2 Phương trình động học thuận

Khi robot nhận vận tốc tuyến tính v và vận tốc góc ω , tốc độ thay đổi trạng thái của robot được mô tả bởi hệ phương trình động học:

$$\dot{x} = v \cos(\theta), \quad (5.2)$$

$$\dot{y} = v \sin(\theta), \quad (5.3)$$

$$\dot{\theta} = \omega. \quad (5.4)$$

Ba công thức trên cho thấy hướng chuyển động của robot phụ thuộc vào góc θ . Nếu robot đang hướng theo trục x của bản đồ thì phần lớn vận tốc tuyến tính làm thay đổi x . Nếu robot quay sang hướng khác, cùng một giá trị v sẽ tạo ra thành phần chuyển động theo cả x và y .

5.2.3 Quan hệ giữa vận tốc robot và vận tốc bánh xe

Gọi r là bán kính bánh xe, L là khoảng cách giữa hai bánh chủ động, ω_r là vận tốc góc của bánh phải và ω_l là vận tốc góc của bánh trái. Vận tốc tuyến tính và vận tốc góc của robot được tính như sau:

$$v = \frac{r}{2} (\omega_r + \omega_l), \quad (5.5)$$

$$\omega = \frac{r}{L} (\omega_r - \omega_l). \quad (5.6)$$

Công thức (5.5) cho biết vận tốc tiến của robot bằng trung bình vận tốc dài của hai bánh. Công thức (5.6) cho biết robot quay khi hai bánh có vận tốc khác nhau. Chênh lệch vận tốc giữa hai bánh càng lớn thì robot quay càng nhanh.

Khi cần điều khiển robot theo lệnh v và ω , bộ điều khiển phải tính ngược lại vận tốc góc của từng bánh:

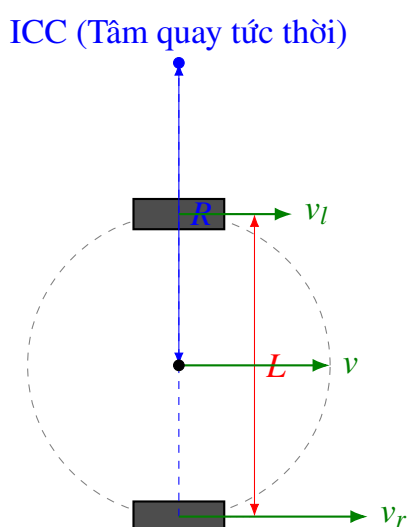
$$\omega_r = \frac{v + \frac{L}{2}\omega}{r}, \quad (5.7)$$

$$\omega_l = \frac{v - \frac{L}{2}\omega}{r}. \quad (5.8)$$

Nếu $\omega = 0$ thì $\omega_r = \omega_l$, robot đi thẳng. Nếu $v = 0$ và $\omega \neq 0$, hai bánh quay ngược chiều, robot quay tại chỗ. Nếu v và ω cùng khác 0, robot vừa tiến vừa quay theo một quỹ đạo cong.

Bảng 5.3 Các trường hợp chuyển động của robot vi sai

Trường hợp	Điều kiện	Chuyển động của robot
Đi thẳng	$\omega_r = \omega_l$	Robot chuyển động theo hướng hiện tại.
Quay trái	$\omega_r > \omega_l$	Bánh phải đi nhanh hơn bánh trái, robot quay sang trái.
Quay phải	$\omega_r < \omega_l$	Bánh trái đi nhanh hơn bánh phải, robot quay sang phải.
Quay tại chỗ	$\omega_r = -\omega_l$	Robot quay quanh tâm gần giữa hai bánh.
Đứng yên	$\omega_r = \omega_l = 0$	Robot không di chuyển.



Hình 5.3 Mô hình động học robot vi sai và quan hệ giữa vận tốc hai bánh

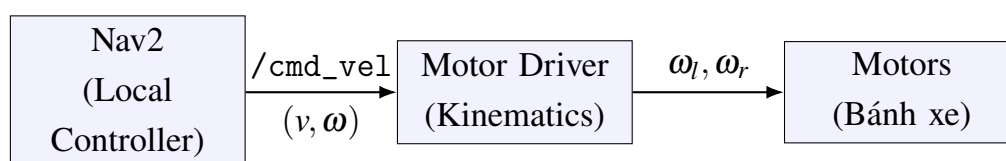
5.3 Lệnh vận tốc /cmd_vel và quá trình thực thi chuyển động

Trong ROS2, lệnh điều khiển vận tốc cho robot di động thường được biểu diễn bằng bản tin `geometry_msgs/Twist`. Đối với robot vi sai di chuyển trên mặt phẳng, hai thành phần quan trọng nhất là `linear.x` và `angular.z`.

Bảng 5.4 Các thành phần chính của bản tin /cmd_vel

Thành phần trong /cmd_vel	Ký hiệu	Ý nghĩa
linear.x	v	Vận tốc tuyến tính theo hướng tiến của robot.
angular.z	ω	Vận tốc góc quay quanh trục thẳng đứng của robot.
linear.y, linear.z	–	Thường không dùng đối với robot vì sai mặt đất.
angular.x, angular.y	–	Thường không dùng vì robot không lắc quanh các trục này trong mô hình phẳng.

Quá trình thực thi chuyển động có thể mô tả theo chuỗi: Nav2 sinh lệnh /cmd_vel, bộ điều khiển cấp thấp chuyển đổi thành tốc độ bánh, động cơ làm quay bánh, robot di chuyển, encoder và odometry phản hồi trạng thái mới. Đây là vòng kín điều khiển ở mức chuyển động.



Hình 5.4 Luồng chuyển đổi từ /cmd_vel sang vận tốc bánh và chuyển động thực tế

5.4 Nav2 như lớp điều khiển chuyển động cấp cao

5.4.1 Vai trò của Nav2

Nav2 là lớp điều khiển chuyển động cấp cao trong project. Thay vì tự viết toàn bộ thuật toán lập đường, tránh vật cản và điều khiển cục bộ, hệ thống sử dụng Nav2 để nhận goal vị trí và chuyển goal đó thành chuỗi lệnh vận tốc phù hợp.

Nav2 không điều khiển trực tiếp động cơ. Nó làm việc ở mức cao hơn: nhận mục tiêu, dùng bản đồ và costmap để lập đường đi, dùng controller để bám đường, sau đó xuất lệnh /cmd_vel cho lớp chấp hành.

5.4.2 Thành phần chính của Nav2 trong project

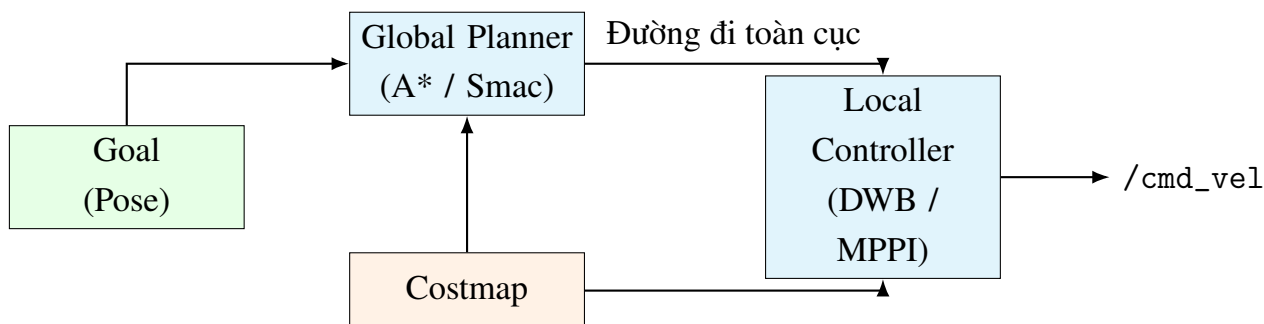
Bảng 5.5 Các thành phần chính của Nav2 trong project

Thành phần	Chức năng
BT Navigator	Quản lý tiến trình điều hướng theo behavior tree hoặc logic điều hướng đã cấu hình.
Planner Server	Tính đường đi toàn cục từ vị trí hiện tại đến goal.
Controller Server	Sinh lệnh vận tốc cục bộ để robot bám theo đường đi.
Costmap	Biểu diễn vùng trống, vật cản và vùng nguy hiểm xung quanh robot.
Recovery behavior	Thực hiện hành vi khôi phục khi robot bị kẹt hoặc không đến được goal.
Action NavigateToPose	Giao diện nhận goal từ Patrol Node hoặc Mission Manager.

5.4.3 Input và output của Nav2

Bảng 5.6 Input và output của Nav2

Mục	Dữ liệu
Input	Goal NavigateToPose, map, costmap, pose robot, odometry, TF, dữ liệu vật cản từ LiDAR.
Xử lý	Lập đường toàn cục, điều khiển cục bộ, tránh vật cản và kiểm tra trạng thái goal.
Output	Lệnh /cmd_vel và trạng thái action như accepted, executing, succeeded, canceled hoặc failed.
Vai trò trong project	Điều khiển robot đến waypoint tuần tra hoặc điểm tiếp cận mục tiêu do Mission Manager tạo ra.



Hình 5.5 Sơ đồ Nav2 nhận goal và sinh lệnh vận tốc /cmd_vel

5.5 Patrol Node

5.5.1 Chức năng

Patrol Node tạo hành vi tuần tra tự động bằng cách gửi lần lượt các waypoint cho Nav2. Waypoint là các điểm mục tiêu trong frame map. Sau khi robot đến một waypoint, node chuyển sang waypoint tiếp theo. Cách làm này đơn giản nhưng phù hợp để tạo hành vi tuần tra trong môi trường mô phỏng.

Trong báo cáo gốc, danh sách waypoint gồm bốn điểm tạo thành một chu trình: (1.0, 0.0), (1.0, 1.0), (0.0, 1.0), (0.0, 0.0). Khi Mission Manager phát hiện mục tiêu và cần tiếp cận, Patrol Node nhận tín hiệu tạm dừng rồi hủy goal đang chạy.

Bảng 5.7 Input, output và vai trò của Patrol Node

Mục	Nội dung
Input	Danh sách waypoint, trạng thái Nav2, tín hiệu /mission_manager/patrol_enabled.
Output	Goal NavigateToPose gửi đến Nav2 hoặc lệnh hủy goal khi cần nhường quyền điều khiển.
Trạng thái cơ bản	Đang gửi waypoint, đang chờ Nav2 hoàn thành, tạm dừng do Mission Manager.
Vai trò	Tạo chuyển động tuần tra nền để robot liên tục quan sát môi trường.

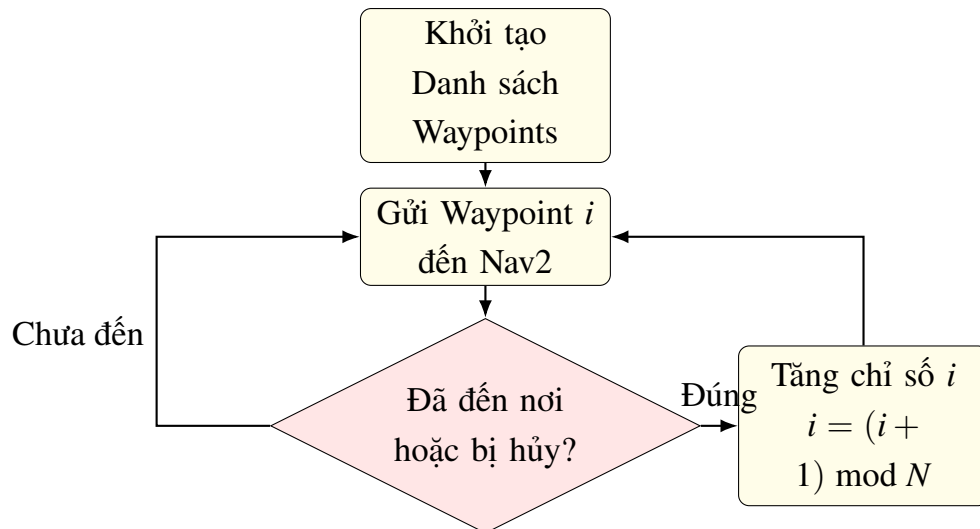
5.5.2 Thuật toán tuần tra

Thuật toán tuần tra có thể mô tả ngắn gọn như sau: nạp danh sách waypoint, đặt chỉ số waypoint ban đầu bằng 0, gửi waypoint hiện tại đến Nav2, chờ kết quả, sau đó tăng chỉ số và lặp lại. Nếu nhận tín hiệu tạm dừng từ Mission Manager, goal hiện tại bị

hủy để robot chuyển sang nhiệm vụ tiếp cận mục tiêu.

$$i_{\text{next}} = (i + 1) \bmod N \quad (5.9)$$

Trong đó, i là chỉ số waypoint hiện tại, N là số waypoint trong chu trình. Phép toán modulo giúp robot quay lại waypoint đầu tiên sau khi hoàn thành waypoint cuối cùng.



Hình 5.6 Sơ đồ thuật toán Patrol Node gửi waypoint tuần tự cho Nav2

5.6 Mission Manager và điều khiển hành vi

5.6.1 Vai trò của Mission Manager

Mission Manager là lớp điều phối hành vi giữa tuần tra và tiếp cận mục tiêu. Khi robot đang tuần tra, nếu hệ thống nhận được pose mục tiêu hợp lệ và mục tiêu nằm xa hơn khoảng cách an toàn, Mission Manager tạm dừng Patrol Node, tính điểm tiếp cận an toàn và gửi goal mới cho Nav2. Khi tiếp cận xong hoặc hết thời gian chờ, robot được chuyển về chế độ tuần tra.

Bảng 5.8 Input, output và vai trò của Mission Manager

Mục	Nội dung
Input	Pose mục tiêu, TF giữa camera và map, pose robot hiện tại, trạng thái Nav2.
Output	Tín hiệu bật/tắt tuần tra, goal tiếp cận an toàn gửi đến Nav2.
Trạng thái chính	patrolling và approaching_target.
Vai trò	Quyết định robot đang tuần tra hay chuyển sang tiếp cận mục tiêu.

5.6.2 Công thức tính điểm tiếp cận an toàn

Điểm tiếp cận an toàn không trùng với vị trí mục tiêu. Điểm này nằm trên đường thẳng nối robot và mục tiêu, nhưng cách mục tiêu một khoảng d_s . Cách tính này giúp robot không va chạm hoặc đứng quá sát đối tượng.

Gọi vị trí robot và vị trí mục tiêu trong frame map lần lượt là:

$$\mathbf{p}_r = \begin{bmatrix} x_r & y_r \end{bmatrix}^T, \quad \mathbf{p}_t = \begin{bmatrix} x_t & y_t \end{bmatrix}^T \quad (5.10)$$

Vector từ robot đến mục tiêu là:

$$\Delta \mathbf{p} = \mathbf{p}_t - \mathbf{p}_r = \begin{bmatrix} x_t - x_r & y_t - y_r \end{bmatrix}^T \quad (5.11)$$

Khoảng cách phẳng giữa robot và mục tiêu được tính bằng chuẩn Euclid:

$$d = \|\Delta \mathbf{p}\|_2 = \sqrt{(x_t - x_r)^2 + (y_t - y_r)^2} \quad (5.12)$$

Với d_s là khoảng cách an toàn, hệ số tiến tới mục tiêu được tính như sau:

$$\alpha = \frac{d - d_s}{d}, \quad d > d_s \quad (5.13)$$

Điểm goal an toàn được xác định bởi:

$$\mathbf{p}_g = \mathbf{p}_r + \alpha \Delta \mathbf{p} \quad (5.14)$$

Viết theo từng tọa độ:

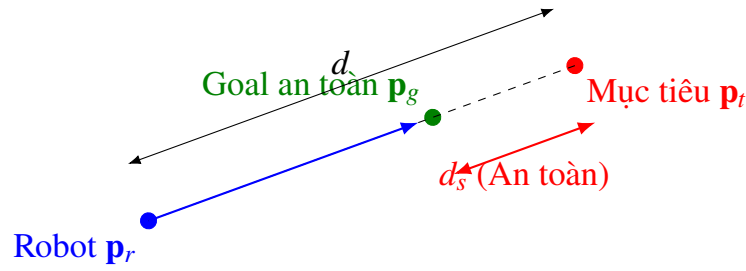
$$x_g = x_r + \alpha(x_t - x_r), \quad (5.15)$$

$$y_g = y_r + \alpha(y_t - y_r). \quad (5.16)$$

Góc quay tại goal có thể chọn sao cho robot hướng về phía mục tiêu:

$$\theta_g = \text{atan2}(y_t - y_g, x_t - x_g) \quad (5.17)$$

Nếu $d \leq d_s$, robot đã ở đủ gần mục tiêu nên không cần gửi goal tiếp cận mới. Điều kiện này tránh việc robot liên tục gửi goal ngắn và gây dao động gần mục tiêu.



Hình 5.7 Cách Mission Manager tính điểm tiếp cận an toàn từ vị trí robot và vị trí mục tiêu

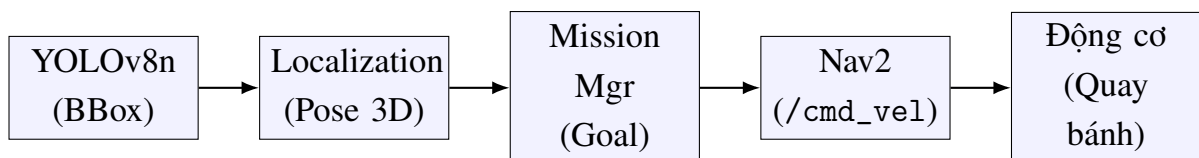
5.7 Quan hệ giữa cảm biến và cơ cấu chấp hành

Cơ cấu chấp hành không hoạt động độc lập. Nó nhận quyết định từ lớp điều khiển, còn lớp điều khiển lại phụ thuộc vào dữ liệu cảm biến và kết quả xử lý. Trong project, camera RGB-D và YOLO phát hiện mục tiêu, Object Localization tính pose mục tiêu, TF chuyển pose sang frame map, Mission Manager tạo goal, Nav2 sinh lệnh `/cmd_vel`, sau đó cơ cấu vi sai thực hiện chuyển động.

$$\text{Detection} \rightarrow 3\text{D Localization} \rightarrow \text{Goal} \rightarrow /cmd_vel \rightarrow \text{Wheel Motion} \quad (5.18)$$

Bảng 5.9 Quan hệ giữa cảm biến, xử lý và cơ cấu chấp hành

Giai đoạn	Dữ liệu chính	Vai trò
Nhận diện	Ảnh RGB, detection YOLO	Xác định có mục tiêu trong ảnh.
Định vị mục tiêu	Detection, depth, CameraInfo, TF	Tính vị trí mục tiêu trong không gian.
Quản lý nhiệm vụ	Pose mục tiêu, pose robot, safe distance	Chọn tuần tra hoặc tiếp cận mục tiêu.
Điều hướng	Goal, map, costmap, odometry	Tạo lệnh vận tốc phù hợp.
Chấp hành	<code>/cmd_vel</code> , vận tốc bánh	Biến lệnh điều khiển thành chuyển động thực tế.



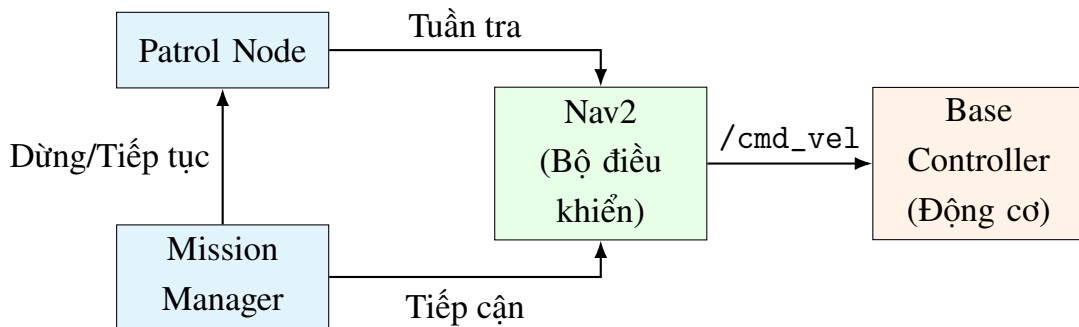
Hình 5.8 Chuỗi xử lý từ phát hiện mục tiêu đến chuyển động bánh xe

5.8 Luồng dữ liệu chính trong Chương 5

Bảng 5.10 Luồng dữ liệu chính trong Chương 5

Bước	Khối xử lý	Input	Output
1	Patrol Node	Danh sách waypoint	Goal tuần tra cho Nav2.
2	Mission Manager	Pose mục tiêu và trạng thái robot	Tín hiệu tạm dừng tuần tra, safe goal.
3	Nav2	Goal, map, costmap, odometry, TF	Lệnh /cmd_vel.
4	Bộ điều khiển cấp thấp	/cmd_vel	Vận tốc bánh phải và bánh trái.
5	Cơ cấu bánh xe	Vận tốc bánh	Chuyển động robot trong môi trường.
6	Encoder/Odometry	Chuyển động bánh	Thông tin phản hồi /odom và TF.

Luồng dữ liệu trên cho thấy Chương 5 là phần nối giữa thuật toán điều hướng và phần chuyển động vật lý của robot. Các công thức động học giúp giải thích vì sao lệnh /cmd_vel có thể được chuyển thành chuyển động của hai bánh.



Hình 5.9 Sơ đồ khối luồng điều khiển chuyển động từ Patrol/Mission Manager đến cơ cấu bánh xe

5.9 Đánh giá kỹ thuật lớp chấp hành và điều khiển

Cấu trúc truyền động vi sai phù hợp với robot trong nhà vì đơn giản, dễ điều khiển và có khả năng quay với bán kính nhỏ. Việc sử dụng Nav2 giúp giảm đáng kể độ phức tạp của hệ thống vì nhóm phát triển không cần tự xây dựng toàn bộ local planner và global planner từ đầu.

Tuy nhiên, chất lượng chuyển động phụ thuộc vào nhiều yếu tố như tham số controller, costmap, độ chính xác của odometry, tốc độ cập nhật dữ liệu cảm biến và độ trễ giữa các node. Trong mô phỏng, các yếu tố như trượt bánh, sai số động cơ và nhiễu encoder chưa thể hiện đầy đủ như trên robot thật. Vì vậy, khi triển khai thực tế cần hiệu chỉnh lại tham số điều khiển và kiểm tra đáp ứng của robot trong từng môi trường cụ thể.

Bảng 5.11 Đánh giá kỹ thuật lớp chấp hành và điều khiển

Ưu điểm	Hạn chế cần lưu ý
Cơ cấu vi sai đơn giản, dễ mô hình hóa và dễ điều khiển.	Dễ bị sai số odometry nếu bánh xe trượt hoặc mặt sàn không đều.
Nav2 cung cấp sẵn lập đường, bám đường và tránh vật cản.	Cần tinh chỉnh controller, costmap và giới hạn vận tốc để robot chạy mượt.
Patrol Node và Mission Manager tách biệt rõ nhiệm vụ và điều khiển.	Mission Manager hiện còn đơn giản, chưa phải behavior tree đầy đủ.
Luồng /cmd_vel chuẩn ROS2, dễ mở rộng sang robot thật.	Khi robot thật hoạt động, cần kiểm tra độ trễ và giới hạn động cơ.

5.10 Kết luận chương

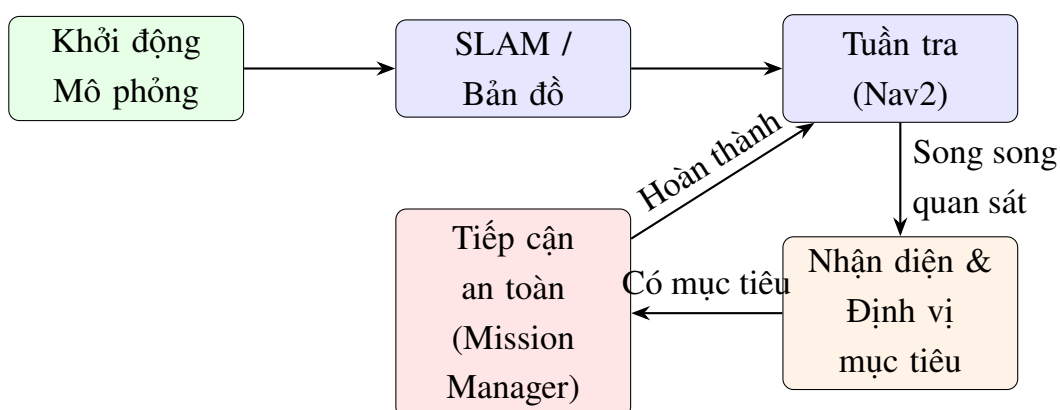
Chương này đã phân tích lớp cơ cấu chấp hành và điều khiển chuyển động của TurtleBot4. Hệ thống sử dụng truyền động vi sai, nhận lệnh vận tốc từ Nav2 và thực hiện chuyển động thông qua hai bánh chủ động. Nav2 đóng vai trò điều khiển cấp cao, Patrol Node tạo hành vi tuần tra, còn Mission Manager điều phối quá trình tạm dừng tuần tra và tiếp cận mục tiêu an toàn.

Phân tích cho thấy mối liên hệ giữa cảm biến và chấp hành là điểm cốt lõi của robot tự hành: cảm biến cung cấp dữ liệu, thuật toán tạo quyết định, Nav2 sinh lệnh vận tốc và cơ cấu truyền động biến lệnh đó thành chuyển động.

6. TRIỂN KHAI NHIỆM VỤ TỰ HÀNH

Chương này trình bày cách hệ thống TurtleBot4 được triển khai thành một nhiệm vụ tự hành hoàn chỉnh trong môi trường mô phỏng. Khác với các chương trước tập trung vào cơ sở lý thuyết, kiến trúc và từng khối kỹ thuật riêng lẻ, nội dung chương này đi theo trình tự vận hành thực tế của hệ thống: khởi động mô phỏng, tạo bản đồ, điều hướng, tuần tra, nhận diện mục tiêu, định vị mục tiêu, điều phối hành vi và kiểm tra các topic quan trọng.

Mục tiêu của chương là làm rõ một câu hỏi chính: từ các package ROS2 đã thiết kế, robot được chạy như thế nào để có thể tuần tra, phát hiện người hoặc vật thể mục tiêu, tính vị trí mục tiêu trong không gian và tiếp cận mục tiêu ở khoảng cách an toàn. Vì vậy, chương này tập trung nhiều vào quy trình, dữ liệu vào/ra, trạng thái hệ thống và cách các node phối hợp với nhau trong quá trình chạy.



Hình 6.1 Quy trình tổng quát triển khai nhiệm vụ tự hành TurtleBot4

6.1 Mục tiêu nhiệm vụ

Nhiệm vụ tự hành trong project được xây dựng theo kịch bản tuần tra và phản ứng với mục tiêu. Robot ban đầu di chuyển theo các waypoint đã định trước trong bản đồ. Trong quá trình tuần tra, camera RGB-D liên tục quan sát môi trường, YOLOv8n phát hiện đối tượng trong ảnh RGB, node định vị mục tiêu dùng ảnh depth để tính tọa độ 3D, sau đó Mission Manager quyết định có tạm dừng tuần tra và gửi goal tiếp cận mục tiêu hay không.

Kịch bản này thể hiện đầy đủ chuỗi hoạt động của robot tự hành: cảm biến thu dữ liệu, thuật toán xử lý biến dữ liệu thành thông tin trạng thái, bộ điều phối chuyển thông tin thành quyết định hành vi, Nav2 biến quyết định thành lệnh di chuyển và cơ cấu chấp hành thực thi chuyển động.

6.1.1 Yêu cầu chức năng của nhiệm vụ

Bảng 6.1 Yêu cầu chức năng của nhiệm vụ tự hành

Yêu cầu	Mô tả
Khởi động môi trường mô phỏng	TurtleBot4 Lite phải được spawn trong Gazebo/Ignition với pose ban đầu xác định.
Thu dữ liệu cảm biến	Hệ thống phải có dữ liệu /scan, /odom, TF, ảnh RGB, ảnh depth và camera_info.
Xây dựng hoặc sử dụng bản đồ	SLAM Toolbox dùng LiDAR và odometry để tạo bản đồ hoặc localization dùng bản đồ đã lưu.
Điều hướng theo waypoint	Patrol Node gửi tuần tự các điểm tuần tra đến Nav2 thông qua action NavigateToPose.
Nhận diện mục tiêu	YOLOv8n phát hiện đối tượng trên ảnh RGB và publish Detection2DArray.
Định vị mục tiêu	Object Localization dùng bbox, depth và camera intrinsic để tính pose 3D.
Tiếp cận an toàn	Mission Manager tính goal cách mục tiêu một khoảng an toàn rồi gửi đến Nav2.
Quay lại tuần tra	Sau khi tiếp cận hoặc hết thời gian chờ, robot bật lại chế độ tuần tra.

6.1.2 Input và output của toàn nhiệm vụ

Bảng 6.2 Input và output của toàn nhiệm vụ tự hành

Loại	Nội dung
Input chính	Mô hình robot TurtleBot4 Lite, môi trường warehouse, dữ liệu LiDAR, camera RGB-D, odometry, TF, bản đồ hoặc dữ liệu SLAM, danh sách waypoint, tham số khoảng cách an toàn.
Output trung gian	Bản đồ occupancy grid, pose robot, detection 2D, pose mục tiêu 3D, marker hiển thị RViz, trạng thái patrol_enabled.
Output cuối	Robot di chuyển theo waypoint, phát hiện mục tiêu, tiếp cận mục tiêu ở khoảng cách an toàn và quay lại tuần tra.

6.2 Quy trình khởi động hệ thống

Hệ thống được khởi động theo hai lớp. Lớp thứ nhất là mô phỏng robot và môi trường trong Gazebo/Ignition. Lớp thứ hai là các node ROS2 phục vụ navigation, perception, localization, patrol và mission management. Việc tách hai bước này giúp dễ kiểm tra lỗi: nếu robot chưa spawn hoặc cảm biến chưa có dữ liệu, cần xử lý ở lớp mô phỏng; nếu cảm biến đã có dữ liệu nhưng robot chưa thực hiện nhiệm vụ, cần kiểm tra ở lớp ROS2.

6.2.1 Bước 1: Khởi động mô phỏng TurtleBot4

Trước khi chạy mission, workspace cần được source đúng môi trường ROS2 và package của project. Các biến môi trường như ROS_DOMAIN_ID và RMW_FASTRTPS_USE_SHM giúp bảo đảm các node trong cùng hệ thống có thể phát hiện nhau và giảm lỗi shared memory trong một số môi trường máy ảo hoặc WSL.

```
1 cd ~/tb4_project_ab
2 export ROS_DOMAIN_ID=7
3 export RMW_FASTRTPS_USE_SHM=0
4 source /opt/ros/humble/setup.bash
5 source install/setup.bash
```

Sau đó khởi động mô phỏng TurtleBot4 Lite trong world warehouse:

```
1 ros2 launch turtlebot4_ignition_bringup
  turtlebot4_ignition.launch.py \
2   model:=lite world:=warehouse x:=2.0 y:=0.0 z:=0.01
   yaw:=0.0
```

Các tham số x, y, z và yaw xác định vị trí và hướng ban đầu của robot trong môi trường mô phỏng. Trong project này, robot được đặt tại vị trí $x = 2.0$, $y = 0.0$, $z = 0.01$ và $yaw = 0.0$ để bảo đảm robot xuất hiện ổn định trên mặt sàn warehouse.

6.2.2 Bước 2: Chạy full mission

Khi robot đã được spawn, launch file tổng hợp `sim_demo.launch.py` được dùng để chạy các thành phần còn lại của hệ thống. Launch file này giúp giảm thao tác thủ công vì người dùng không cần mở riêng từng node YOLO, object localization, patrol, mission manager và RViz.

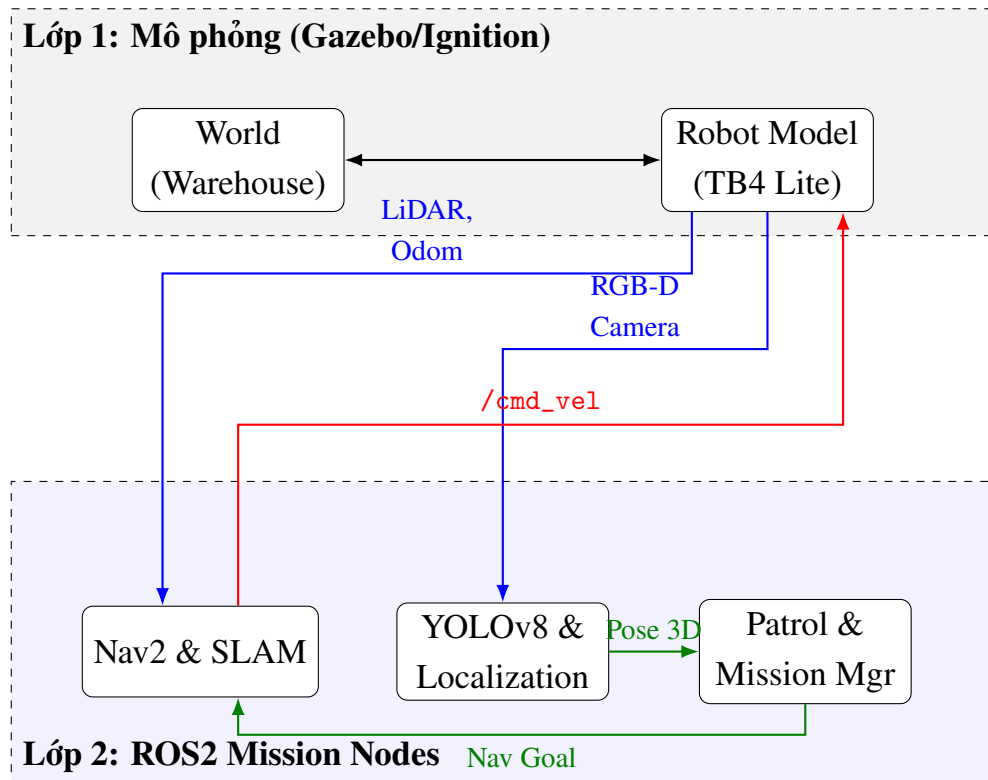
```
1 ros2 launch tb4_bringup sim_demo.launch.py use_rviz:=true
```


Nếu `use_rviz:=true`, RViz được mở để quan sát robot, map, LaserScan, TF, marker mục tiêu và trạng thái navigation. Đây là công cụ quan trọng để kiểm tra trực quan quá trình vận hành.

6.2.3 Input, output và điểm cần kiểm tra khi khởi động

Bảng 6.3 Input, output và điểm cần kiểm tra khi khởi động hệ thống

Khối	Input	Output cần có	Ý nghĩa kiểm tra
Gazebo/Ignition	World warehouse, model TurtleBot4 Lite, pose ban đầu.	Robot xuất hiện trong mô phỏng, có clock mô phỏng.	Xác nhận môi trường và robot đã chạy.
LiDAR	Mô hình cảm biến LiDAR trong robot.	Topic <code>/scan</code> có dữ liệu LaserScan.	Là điều kiện đầu vào cho SLAM và costmap.
Camera RGB-D	Mô hình camera OAK-D trong robot.	Ảnh RGB, depth và <code>camera_info</code> .	Là điều kiện đầu vào cho YOLO và định vị mục tiêu.
Nav2	Map, TF, odometry, scan, goal.	Action <code>NavigateToPose</code> , lệnh <code>/cmd_vel</code> .	Cho biết robot có thể nhận goal và điều hướng.
Mission Manager	Pose mục tiêu, trạng thái patrol, TF.	Tín hiệu tạm dừng tuần tra và goal tiếp cận.	Cho biết hệ thống có thể chuyển hành vi khi phát hiện mục tiêu.



Hình 6.2 Hai lớp khởi động hệ thống gồm mô phỏng Gazebo/Ignition và các node ROS2 mission

6.3 SLAM và bản đồ môi trường

Trong nhiệm vụ tự hành, robot cần biết vị trí của mình tương đối với môi trường. Nếu chưa có bản đồ, hệ thống sử dụng SLAM Toolbox để xây dựng occupancy grid map từ dữ liệu LiDAR, odometry và TF. Nếu đã có bản đồ, robot có thể chuyển sang chế độ localization để định vị trên bản đồ đã lưu và dùng Nav2 để điều hướng.

Occupancy grid map biểu diễn môi trường thành các ô lưới. Mỗi ô có thể được hiểu là một vùng nhỏ ngoài đời thực với kích thước phụ thuộc vào độ phân giải bản đồ. Trong cấu hình của project, độ phân giải thường dùng là 0.05 m, nghĩa là mỗi ô trên bản đồ tương ứng với 5 cm trong môi trường mô phỏng.

6.3.1 Dữ liệu vào và dữ liệu ra của quá trình SLAM

Bảng 6.4 Dữ liệu vào và dữ liệu ra của quá trình SLAM

Thành phần	Vai trò trong SLAM
/scan	Cung cấp khoảng cách từ robot đến vật cản theo nhiều góc quét. Đây là dữ liệu chính để tạo biên vật cản trong bản đồ.
/odom	Cung cấp ước lượng chuyển động tương đối của robot giữa các thời điểm.
TF	Liên kết các frame map, odom, base_link và rplidar_link để dữ liệu quét được đặt đúng vị trí trong không gian.
SLAM Toolbox	Thực hiện scan matching, cập nhật pose graph, phát hiện loop closure và publish map.
/map	Output của SLAM, biểu diễn môi trường dưới dạng occupancy grid.
map -> odom	Transform giúp liên hệ hệ tọa độ bản đồ với hệ tọa độ odometry.

6.3.2 Cách quy đổi vị trí thực sang ô bản đồ

Khi bản đồ được chia thành lưới, một điểm trong môi trường có tọa độ thực (x, y) sẽ được quy đổi sang chỉ số ô bản đồ. Gọi r là độ phân giải bản đồ, x_0 và y_0 là gốc bản đồ, chỉ số ô có thể viết:

$$i = \left\lfloor \frac{x - x_0}{r} \right\rfloor, \quad (6.1)$$

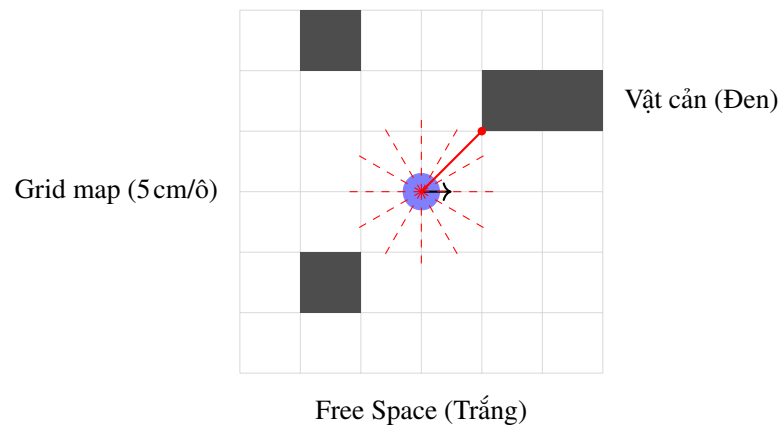
$$j = \left\lfloor \frac{y - y_0}{r} \right\rfloor. \quad (6.2)$$

Trong đó, i và j là chỉ số hàng/cột của ô bản đồ. Nếu $r = 0.05$ m, một đoạn 1 m trong môi trường thật sẽ tương ứng với 20 ô bản đồ. Nhờ cách biểu diễn này, thuật toán điều hướng có thể lập kế hoạch đường đi trên bản đồ số thay vì xử lý trực tiếp toàn bộ không gian liên tục.

6.3.3 Các tham số SLAM quan trọng

Bảng 6.5 Các tham số SLAM quan trọng

Tham số	Giá trị thường dùng	Ý nghĩa
resolution	0.05	Độ phân giải bản đồ, mỗi ô tương ứng 5 cm.
min_laser_range	0.2	Loại bỏ các điểm đo quá gần robot, thường dễ nhiễu hoặc nằm trên thân robot.
max_laser_range	12.0	Giới hạn khoảng cách đo xa nhất của LiDAR được dùng trong SLAM.
map_update_interval	2.0 s	Chu kỳ cập nhật bản đồ.
use_scan_matching	true	So khớp các lần quét LiDAR liên tiếp để cải thiện pose.
do_loop_closing	true	Phát hiện robot quay lại vùng đã đi qua để giảm sai số tích lũy.



Hình 6.3 Minh họa occupancy grid map và tia LiDAR quét trúng vật cản

6.4 Navigation và tuần tra

Sau khi có bản đồ và pose robot, Nav2 được dùng để điều hướng robot đến các điểm đích. Trong project, hành vi tuần tra được tạo bởi Patrol Node. Node này không tự điều khiển bánh xe, mà gửi từng waypoint đến Nav2 bằng action NavigateToPose. Nav2 chịu trách nhiệm lập đường, điều khiển cục bộ, tránh vật cản và phát lệnh vận tốc.

6.4.1 Mô hình waypoint tuần tra

Mỗi waypoint có thể mô tả bởi một pose trong frame map:

$$\mathbf{P}_k = (x_k, y_k, \theta_k) \quad (6.3)$$

Trong đó, x_k và y_k là tọa độ điểm tuần tra thứ k , còn θ_k là hướng quay mong muốn của robot tại điểm đó. Danh sách waypoint trong báo cáo gốc gồm bốn điểm tạo thành một chu trình đơn giản:

$$\mathbf{P}_1 = (1.0, 0.0), \quad \mathbf{P}_2 = (1.0, 1.0), \quad \mathbf{P}_3 = (0.0, 1.0), \quad \mathbf{P}_4 = (0.0, 0.0) \quad (6.4)$$

Khi Nav2 báo goal hiện tại đã hoàn thành, Patrol Node tăng chỉ số waypoint và gửi waypoint tiếp theo. Khi đến waypoint cuối cùng, chỉ số được đưa về điểm đầu để tạo vòng tuần tra lặp lại.

6.4.2 Điều kiện hoàn thành waypoint

Về mặt logic, một waypoint được xem là hoàn thành khi robot đến đủ gần vị trí đích và góc quay nằm trong ngưỡng cho phép. Khoảng cách giữa robot và waypoint có thể tính:

$$e_p = \sqrt{(x_g - x_r)^2 + (y_g - y_r)^2} \quad (6.5)$$

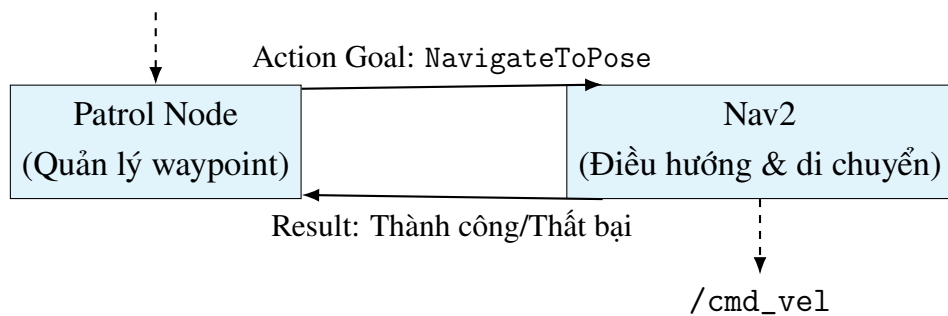
Trong đó, (x_r, y_r) là vị trí hiện tại của robot và (x_g, y_g) là vị trí goal. Nếu e_p nhỏ hơn một ngưỡng sai số cho phép, Nav2 có thể xem robot đã tới vùng đích. Phần kiểm tra chi tiết thường do controller và action server của Nav2 xử lý, Patrol Node chỉ nhận kết quả thành công, thất bại hoặc bị hủy.

6.4.3 Trạng thái của Patrol Node

Bảng 6.6 Các trạng thái chính của Patrol Node

Trạng thái	Mô tả
Chờ Nav2 sẵn sàng	Node kiểm tra action server NavigateToPose trước khi gửi goal.
Gửi waypoint	Waypoint hiện tại được đóng gói thành goal và gửi đến Nav2.
Đang tuần tra	Robot di chuyển đến waypoint, Patrol Node chờ feedback/result.
Hoàn thành waypoint	Node chuyển sang waypoint tiếp theo.
Bị Mission Manager tạm dừng	Nếu patrol_enabled = false, goal hiện tại bị hủy để nhường quyền cho Mission Manager.

Tín hiệu tạm dừng từ Mission Manager



Hình 6.4 Sơ đồ Patrol Node gửi tuần tự waypoint đến Nav2

6.5 Nhận diện và định vị mục tiêu

Trong khi robot tuần tra, hệ thống perception chạy song song. YOLOv8n xử lý ảnh RGB để phát hiện đối tượng, còn Object Localization dùng ảnh depth và thông số camera để biến bounding box 2D thành vị trí 3D. Đây là bước biến thông tin thị giác thành thông tin có thể dùng cho navigation.

6.5.1 Luồng xử lý nhận diện

Bảng 6.7 Luồng xử lý nhận diện và định vị mục tiêu

Bước	Input	Xử lý	Output
1	Ảnh RGB từ /oakd/rgb/preview/image_raw	YOLOv8n chạy inference trên từng frame ảnh.	Danh sách detection 2D.
2	Detection 2D	Lọc theo confidence và ưu tiên class person nếu có.	Bounding box mục tiêu.
3	Bounding box và ảnh depth	Lấy tâm bbox, đọc depth tại vùng lân cận và dùng median để giảm nhiễu.	Độ sâu Z hợp lệ.
4	Z và CameraInfo	Dùng mô hình pinhole để tính tọa độ 3D trong hệ camera.	Pose mục tiêu tương đối với camera.
5	Pose camera và TF	Chuyển pose mục tiêu sang frame map nếu cần.	Pose mục tiêu phục vụ Mission Manager.

6.5.2 Công thức tính tọa độ 3D từ RGB-D

Gọi (u, v) là tâm bounding box trên ảnh, Z là độ sâu tại vùng tâm bbox, f_x, f_y là tiêu cự theo pixel, c_x, c_y là tọa độ tâm ảnh. Tọa độ mục tiêu trong hệ camera được tính theo mô hình pinhole:

$$X_c = \frac{(u - c_x)Z}{f_x}, \quad (6.6)$$

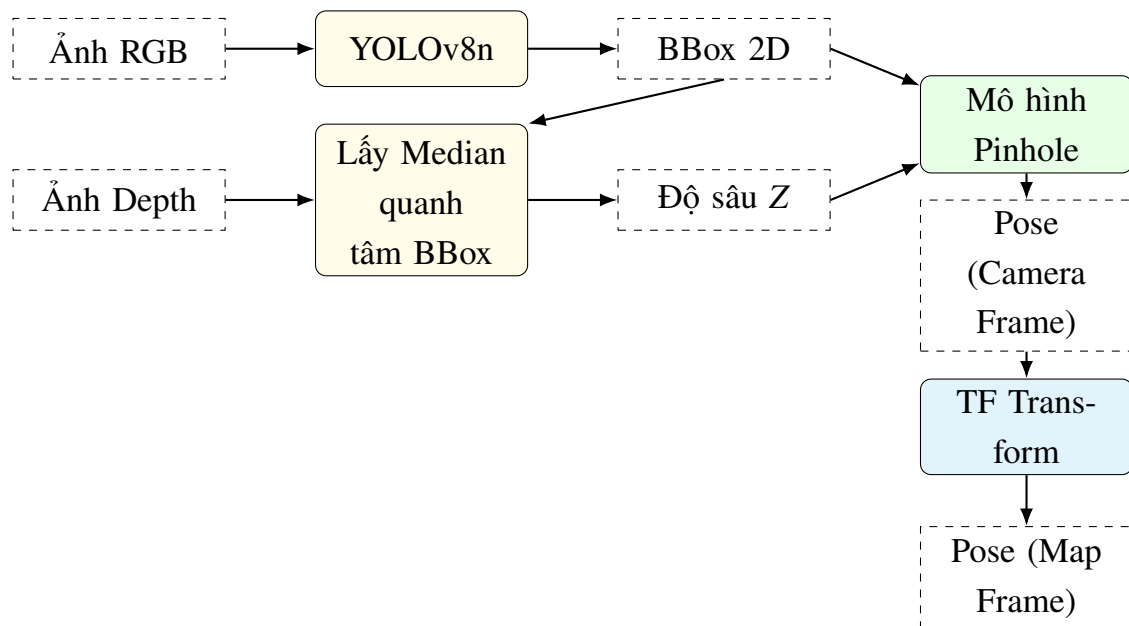
$$Y_c = \frac{(v - c_y)Z}{f_y}, \quad (6.7)$$

$$Z_c = Z. \quad (6.8)$$

Trong triển khai, không nên lấy duy nhất một pixel depth vì ảnh depth có thể có nhiễu hoặc điểm mất dữ liệu. Cách dùng trung vị trong một cửa sổ nhỏ quanh tâm bbox giúp giá trị Z ổn định hơn:

$$Z = \text{median} \{D(u + \Delta u, v + \Delta v)\} \quad (6.9)$$

Trong đó, D là ảnh depth, còn Δu và Δv là các độ lệch pixel nằm trong cửa sổ lân cận tâm bbox. Nếu Z không hợp lệ, node định vị mục tiêu không nên publish pose để tránh Mission Manager nhận sai vị trí.



Hình 6.5 Luồng YOLOv8 + depth chuyển bounding box 2D thành pose mục tiêu 3D

6.6 Mission Manager

Mission Manager là node điều phối hành vi cấp nhiệm vụ. Node này quyết định khi nào robot tiếp tục tuần tra, khi nào tạm dừng tuần tra và khi nào gửi goal tiếp cận mục tiêu. Điểm quan trọng là Mission Manager không điều khiển trực tiếp động cơ; node chỉ đưa ra quyết định cấp cao và gửi goal cho Nav2.

6.6.1 Trạng thái chính của Mission Manager

Bảng 6.8 Các trạng thái chính của Mission Manager

Trạng thái	Điều kiện vào trạng thái	Hành động chính	Điều kiện thoát
patrolling	Mặc định khi hệ thống bắt đầu hoặc sau khi hoàn thành tiếp cận.	Cho phép Patrol Node gửi waypoint tuần tra.	Nhận pose mục tiêu hợp lệ và mục tiêu ở xa hơn <code>safe_distance</code> .
approaching_target	Có mục tiêu hợp lệ cần tiếp cận.	Tạm dừng Patrol Node, tính safe goal và gửi goal đến Nav2.	Nav2 hoàn thành goal, thất bại hoặc hết thời gian chờ.
cooldown	Sau khi kết thúc tiếp cận.	Tạm thời không phản ứng lại ngay với cùng mục tiêu.	Hết thời gian cooldown thì bật lại tuần tra.

6.6.2 Tính điểm tiếp cận an toàn

Mission Manager không gửi robot đến đúng tọa độ mục tiêu mà chọn một điểm dừng cách mục tiêu một khoảng an toàn. Gọi vị trí robot và mục tiêu trong frame map là:

$$\mathbf{p}_r = \begin{bmatrix} x_r & y_r \end{bmatrix}^T, \quad \mathbf{p}_t = \begin{bmatrix} x_t & y_t \end{bmatrix}^T \quad (6.10)$$

Vector từ robot đến mục tiêu và khoảng cách phẳng được tính:

$$\Delta \mathbf{p} = \mathbf{p}_t - \mathbf{p}_r = \begin{bmatrix} x_t - x_r & y_t - y_r \end{bmatrix}^T \quad (6.11)$$

$$d = \|\Delta \mathbf{p}\|_2 = \sqrt{(x_t - x_r)^2 + (y_t - y_r)^2} \quad (6.12)$$

Gọi d_s là khoảng cách an toàn. Nếu $d > d_s$, điểm goal được chọn trên đoạn thẳng từ robot đến mục tiêu:

$$\alpha = \frac{d - d_s}{d} \quad (6.13)$$

$$\mathbf{p}_g = \mathbf{p}_r + \alpha \Delta \mathbf{p} \quad (6.14)$$

Tương đương theo từng tọa độ:

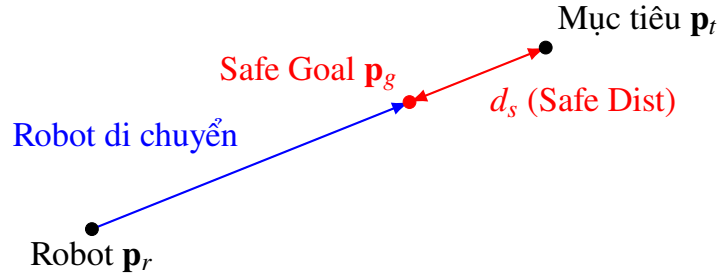
$$x_g = x_r + \alpha(x_t - x_r), \quad (6.15)$$

$$y_g = y_r + \alpha(y_t - y_r). \quad (6.16)$$

Nếu $d \leq d_s$, robot đã ở đủ gần mục tiêu nên không cần gửi goal tiếp cận mới. Góc quay tại điểm goal có thể chọn sao cho robot hướng về mục tiêu:

$$\theta_g = \text{atan2}(y_t - y_g, x_t - x_g) \quad (6.17)$$

Cách tính này giúp robot tiếp cận mục tiêu mà vẫn giữ khoảng cách an toàn, tránh trường hợp robot đi thẳng vào vị trí người hoặc vật thể được phát hiện.



Hình 6.6 Mission Manager tính safe goal từ vị trí robot và vị trí mục tiêu

6.6.3 Input và output của Mission Manager

Bảng 6.9 Input và output của Mission Manager

Mục	Nội dung
Input	Pose mục tiêu, TF giữa camera và map, pose hiện tại của robot, trạng thái Nav2, tham số safe_distance, thời gian cooldown.
Output	Tín hiệu bật/tắt tuần tra, goal tiếp cận an toàn gửi đến Nav2, trạng thái approaching hoặc patrolling.
Điều kiện xử lý	Chỉ tiếp cận khi pose mục tiêu hợp lệ, TF sẵn sàng và khoảng cách đến mục tiêu lớn hơn safe_distance.
Vai trò	Chuyển đổi dữ liệu perception thành hành vi tự hành cấp cao.

6.7 Các topic kiểm tra quan trọng

Trong quá trình chạy hệ thống, việc kiểm tra topic giúp xác định lỗi nằm ở cảm biến, perception, localization, navigation hay mission logic. Nếu một topic không có dữ liệu, các node phía sau trong pipeline có thể không hoạt động đúng.

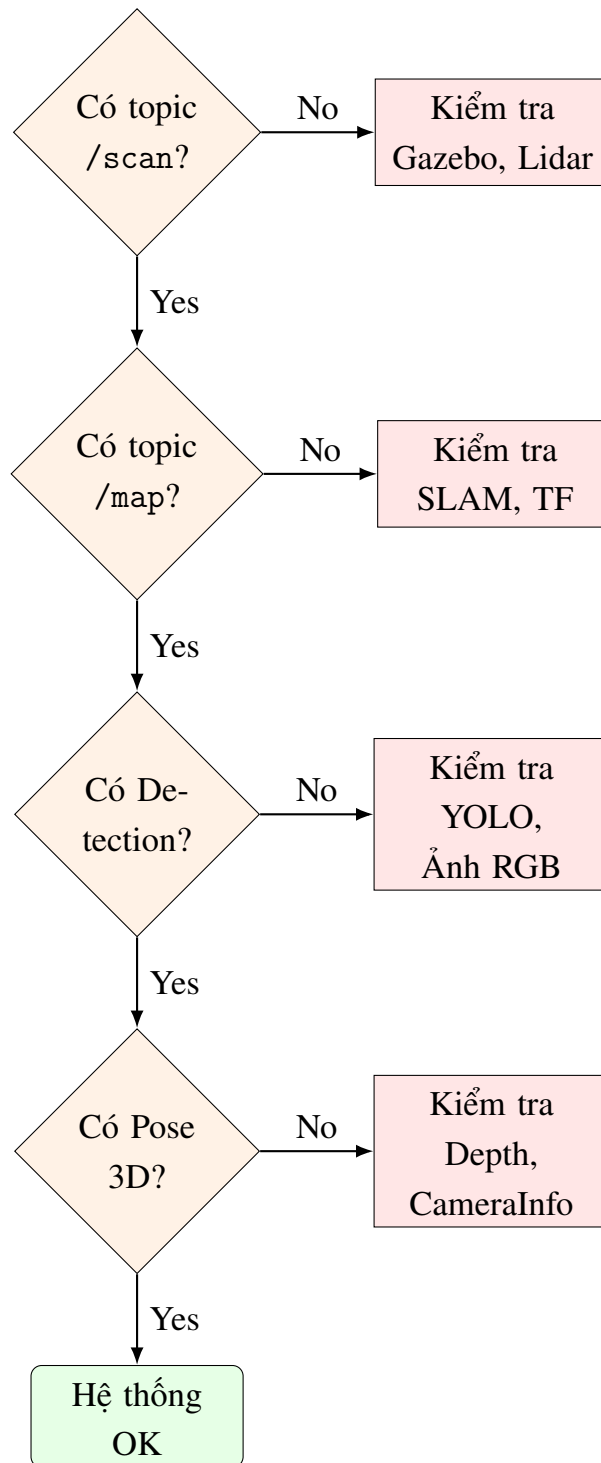
Bảng 6.10 Các topic và lệnh kiểm tra quan trọng

Topic/lệnh kiểm tra	Ý nghĩa	Kết quả mong muốn
<code>ros2 topic echo /scan -once</code>	Kiểm tra dữ liệu LiDAR.	Có bản tin LaserScan với mảng ranges hợp lệ.
<code>ros2 topic echo /odom -once</code>	Kiểm tra odometry của robot.	Có pose và twist của robot.
<code>ros2 topic echo /clock -once</code>	Kiểm tra thời gian mô phỏng.	Có thời gian mô phỏng tăng đều.
<code>ros2 topic echo /vision/detected_objects -once</code>	Kiểm tra kết quả YOLOv8n.	Có Detection2DArray khi có đối tượng trong ảnh.
<code>ros2 topic echo /target_object_pose_map -once</code>	Kiểm tra pose mục tiêu sau định vị.	Có PoseStamped khi bbox và depth hợp lệ.
<code>ros2 topic echo /target_object_marker -once</code>	Kiểm tra marker hiển thị RViz.	Có Marker phục vụ quan sát trực quan.
<code>ros2 topic echo /cmd_vel -once</code>	Kiểm tra lệnh vận tốc từ Nav2.	Có Twist khi robot đang di chuyển.

6.7.1 Cách đọc lỗi theo luồng dữ liệu

Bảng 6.11 Cách đọc lỗi theo luồng dữ liệu

Hiện tượng	Nguyên nhân có thể	Hướng kiểm tra
Không có /scan	LiDAR chưa được mô phỏng đúng, frame/collision lỗi, Gazebo chưa chạy.	Kiểm tra robot model, rplidar_link và RViz LaserScan.
Có /scan nhưng không có map	SLAM Toolbox chưa subscribe đúng topic hoặc TF thiếu.	Kiểm tra scan_topic, base_frame, odom_frame, map_frame.
Không có detection	Camera chưa publish ảnh hoặc YOLO chưa chạy.	Kiểm tra ảnh RGB, model yolov8n.pt và confidence threshold.
Có detection nhưng không có target pose	Depth không hợp lệ hoặc thiếu camera_info.	Kiểm tra ảnh depth, CameraInfo và vùng tâm bbox.
Có target pose nhưng robot không tiếp cận	TF sang map không sẵn sàng hoặc Mission Manager không gửi goal.	Kiểm tra TF tree, trạng thái patrol_enabled và action NavigateToPose.
Robot nhận goal nhưng không đi	Nav2/costmap/controller lỗi hoặc goal nằm ngoài vùng hợp lệ.	Kiểm tra Nav2 status, global costmap, local costmap và /cmd_vel.



Hình 6.7 Sơ đồ kiểm tra lỗi theo pipeline cảm biến – nhận diện – định vị – điều hướng

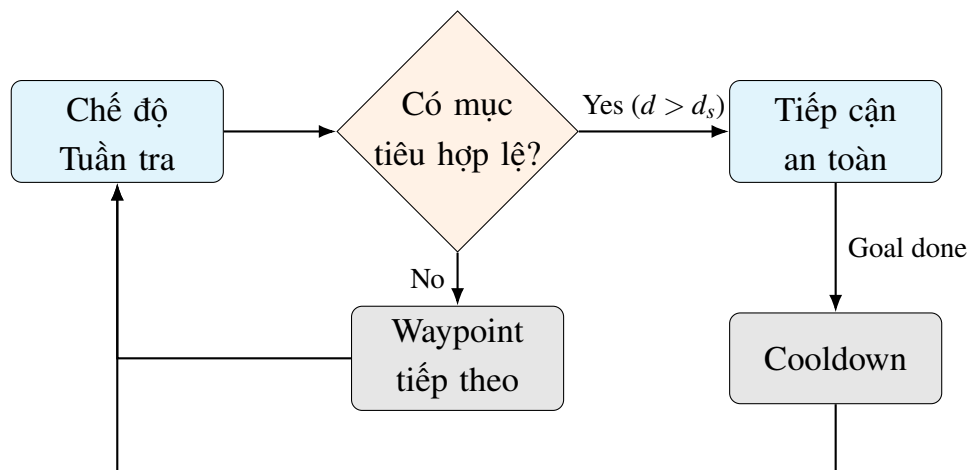
6.8 Sơ đồ hoạt động tổng hợp của nhiệm vụ

Sơ đồ hoạt động của nhiệm vụ có thể mô tả theo chuỗi quyết định sau. Robot bắt đầu bằng chế độ tuần tra. Nếu không phát hiện mục tiêu hợp lệ, robot tiếp tục gửi waypoint kế tiếp. Nếu có mục tiêu hợp lệ và khoảng cách đến mục tiêu lớn hơn khoảng

cách an toàn, Mission Manager tạm dừng patrol, tính safe goal và gửi goal này đến Nav2. Khi quá trình tiếp cận kết thúc, hệ thống bật lại tuần tra.

Bảng 6.12 Sơ đồ hoạt động tổng hợp của nhiệm vụ tự hành

Bước	Khởi thực hiện	Dữ liệu vào	Dữ liệu ra
1	Gazebo/Ignition	World, robot model, pose ban đầu.	Dữ liệu cảm biến mô phỏng.
2	SLAM/Localization	/scan, /odom, TF.	Map và pose robot.
3	Patrol Node	Danh sách waypoint, trạng thái patrol_enabled.	Goal tuần tra gửi Nav2.
4	YOLOv8n	Ảnh RGB.	Detection2DArray.
5	Object Localization	Detection, depth, CameraInfo.	Pose mục tiêu 3D.
6	Mission Manager	Pose mục tiêu, pose robot, safe_distance.	Safe goal hoặc lệnh tiếp tục tuần tra.
7	Nav2	Goal, map, costmap, TF.	/cmd_vel.
8	Differential Drive	/cmd_vel.	Chuyển động thực của robot trong mô phỏng.



Hình 6.8 Sơ đồ hoạt động tổng hợp của nhiệm vụ tự hành tuần tra – phát hiện – tiếp cận

6.9 Đánh giá quy trình triển khai

Quy trình triển khai trong chương này có ưu điểm là rõ ràng, dễ kiểm tra và phù hợp với kiến trúc module của ROS2. Mỗi chức năng được kiểm tra bằng topic hoặc

action riêng. Điều này giúp khi hệ thống không hoạt động, người phát triển có thể lần theo pipeline để xác định lỗi.

Tuy nhiên, quy trình hiện vẫn mang tính mô phỏng và chưa tối ưu hoàn toàn cho robot thật. Waypoint còn được đặt cố định, Mission Manager mới ở dạng logic trạng thái đơn giản, việc định vị mục tiêu phụ thuộc nhiều vào chất lượng ảnh depth và hệ thống chưa có cơ chế tracking mục tiêu theo thời gian. Khi triển khai trên robot thật, cần bổ sung đánh giá định lượng, kiểm tra độ trễ, nhiễu cảm biến, sai số odometry và khả năng tránh vật cản động.

Bảng 6.13 Đánh giá quy trình triển khai nhiệm vụ tự hành

Mặt đánh giá	Nhận xét
Tính rõ ràng	Quy trình triển khai theo từng bước, dễ chạy và dễ debug.
Tính module	Các node perception, localization, patrol và mission manager tách biệt, dễ thay thế hoặc mở rộng.
Tính an toàn	Robot không đi thẳng vào mục tiêu mà dừng ở khoảng cách <code>safe_distance</code> .
Tính hạn chế	Phụ thuộc TF, depth, cấu hình Nav2 và chưa xử lý nhiều mục tiêu động.
Khả năng mở rộng	Có thể nâng cấp Mission Manager thành finite state machine hoặc behavior tree.

6.10 Kết luận chương

Chương 6 đã trình bày quá trình triển khai nhiệm vụ tự hành của TurtleBot4 từ khi khởi động mô phỏng đến khi robot thực hiện tuần tra, nhận diện mục tiêu, định vị mục tiêu và tiếp cận mục tiêu an toàn. Nội dung chương cho thấy các thành phần đã phân tích ở các chương trước không hoạt động riêng lẻ mà được ghép thành một chuỗi xử lý hoàn chỉnh.

Thông qua quy trình triển khai, có thể thấy vai trò của từng lớp trong hệ thống: Gazebo/Ignition tạo môi trường và dữ liệu cảm biến; SLAM/localization tạo bản đồ và pose; Nav2 lập kế hoạch và điều khiển; YOLOv8n và RGB-D tạo thông tin mục tiêu; Mission Manager quyết định hành vi; cơ cấu chấp hành thực hiện chuyển động. Đây là cơ sở để chuyển sang chương đánh giá kết quả mô phỏng và chỉ ra những hạn chế cần cải thiện.

7. KẾT QUẢ MÔ PHỎNG VÀ ĐÁNH GIÁ

Chương này trình bày cách tổng hợp kết quả mô phỏng và đánh giá hệ thống robot tự hành TurtleBot4 sau khi các thành phần đã được triển khai trong môi trường Gazebo/Ignition. Khác với các chương trước chủ yếu trình bày cơ sở lý thuyết, kiến trúc, cảm biến và điều khiển, nội dung chương này tập trung trả lời câu hỏi: hệ thống đã chạy được những chức năng nào, chất lượng của từng khối ra sao, còn hạn chế gì và cần cải tiến theo hướng nào.

Việc đánh giá được thực hiện theo hướng hệ thống, tức là không chỉ xem riêng từng node ROS2 mà còn xem xét toàn bộ chuỗi xử lý từ mô phỏng, cảm biến, SLAM, điều hướng, nhận diện, định vị mục tiêu đến hành vi tiếp cận. Cách đánh giá này phù hợp với bản chất của robot tự hành, vì một sai số nhỏ ở cảm biến, TF hoặc navigation đều có thể ảnh hưởng đến kết quả cuối cùng của nhiệm vụ.

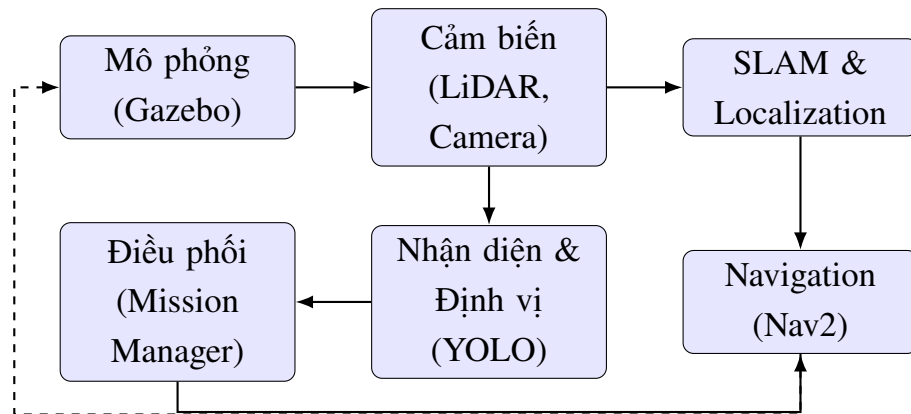
7.1 Mục đích và phạm vi đánh giá

Mục đích của phần đánh giá là xác nhận hệ thống đã đáp ứng các mục tiêu chính của đề tài ở mức mô phỏng. Các mục tiêu cần được kiểm tra gồm: robot có thể được spawn trong môi trường mô phỏng, các topic cảm biến có dữ liệu, hệ thống có thể xây dựng bản đồ hoặc sử dụng bản đồ để định vị, Nav2 có thể nhận goal, YOLOv8n có thể phát hiện mục tiêu, Object Localization có thể tạo pose mục tiêu và Mission Manager có thể điều phối quá trình tuần tra – tiếp cận.

Phạm vi đánh giá trong chương này dừng ở mức kiểm chứng chức năng và phân tích kỹ thuật. Do hệ thống chưa triển khai trên robot thật, các yếu tố như sai số cơ khí, trượt bánh thực tế, độ rung cảm biến, thay đổi ánh sáng và độ trễ phần cứng chưa được đo trực tiếp. Tuy nhiên, đánh giá mô phỏng vẫn có giá trị vì nó giúp kiểm tra logic tích hợp, luồng dữ liệu ROS2 và khả năng phối hợp giữa các module.

Bảng 7.1 Nội dung và mục đích đánh giá hệ thống

Nội dung đánh giá	Mục đích	Kết quả cần quan sát
Mô phỏng robot	Kiểm tra robot TurtleBot4 Lite có xuất hiện và hoạt động trong warehouse.	Robot spawn đúng vị trí, có clock mô phỏng và các plugin cảm biến hoạt động.
Hệ cảm biến	Kiểm tra LiDAR, camera RGB-D, odometry và TF.	Các topic /scan, /odom, ảnh RGB, ảnh depth và camera_info có dữ liệu ổn định.
SLAM/Localization	Kiểm tra khả năng tạo bản đồ hoặc định vị robot trên bản đồ.	Có map, pose robot và transform map -> odom hợp lệ.
Navigation	Kiểm tra Nav2 nhận goal và điều hướng theo waypoint.	Robot di chuyển đến waypoint, tránh vật cản và cập nhật trạng thái goal.
Nhận diện – định vị mục tiêu	Kiểm tra YOLOv8n và Object Localization.	Có detection 2D, pose mục tiêu 3D và marker hiển thị trên RViz.
Mission Manager	Kiểm tra hành vi tuần tra – phát hiện – tiếp cận.	Robot tạm dừng patrol, tính safe goal, tiếp cận mục tiêu rồi quay lại tuần tra.

**Hình 7.1 Quy trình đánh giá tổng thể hệ thống TurtleBot4 trong mô phỏng**

7.2 Kết quả đạt được

Kết quả mô phỏng cho thấy hệ thống đã đạt được mức tích hợp chức năng cơ bản. Robot TurtleBot4 Lite có thể được khởi chạy trong môi trường warehouse của Gazebo/Ignition. Các nguồn dữ liệu LiDAR, camera RGB-D, odometry và TF có thể được publish để các node xử lý sử dụng. Trên cơ sở đó, SLAM Toolbox, Nav2, YOLOv8n, Object Localization và Mission Manager được ghép lại thành một pipeline nhiệm vụ tự

hành.

Điểm quan trọng của kết quả này là hệ thống không chỉ chạy từng node độc lập mà đã hình thành được luồng nhiệm vụ tương đối đầy đủ: robot tuần tra theo waypoint, camera quan sát môi trường, YOLO phát hiện mục tiêu, ảnh depth hỗ trợ tính vị trí mục tiêu, Mission Manager chuyển mục tiêu thành goal an toàn và Nav2 điều khiển robot tiếp cận. Đây là kết quả phù hợp với mục tiêu xây dựng một hệ robot tự hành trong môi trường mô phỏng.

Bảng 7.2 Tổng hợp các kết quả đạt được

Nhóm kết quả	Kết quả đạt được	Ý nghĩa đối với hệ thống
Mô phỏng	TurtleBot4 Lite được spawn trong warehouse của Gazebo/Ignition.	Tạo môi trường thử nghiệm ổn định trước khi triển khai trên robot thật.
Cảm biến	LiDAR, camera RGB-D, odometry và TF có dữ liệu phục vụ xử lý.	Cung cấp đầu vào cho SLAM, Nav2, YOLO và định vị mục tiêu.
SLAM/Map	SLAM Toolbox có thể sử dụng dữ liệu LiDAR để tạo occupancy grid map.	Robot có cơ sở bản đồ để thực hiện navigation.
Navigation	Nav2 nhận goal từ Patrol Node hoặc Mission Manager.	Robot có thể di chuyển theo waypoint và tiếp cận mục tiêu.
Nhận diện	YOLOv8n phát hiện đối tượng từ ảnh RGB.	Bổ sung lớp thông tin ngữ nghĩa cho robot.
Định vị mục tiêu	Object Localization tạo pose mục tiêu từ bbox và depth.	Chuyển detection 2D thành vị trí không gian để dùng cho điều hướng.
Điều phối hành vi	Mission Manager điều phối tuần tra, phát hiện và tiếp cận.	Tạo hành vi tự hành thay vì chỉ chạy điều hướng đơn lẻ.

7.3 Phương pháp đánh giá và chỉ tiêu định lượng đề xuất

Trong báo cáo gốc, kết quả được đánh giá chủ yếu theo mức hoàn thành chức năng. Để phân đánh giá rõ ràng hơn, có thể bổ sung thêm một số chỉ tiêu định lượng. Các chỉ tiêu này không bắt buộc phải có số liệu thực nghiệm ngay, nhưng nên trình bày công thức để thể hiện cách đánh giá nếu hệ thống được đo trong nhiều lần chạy.

Giả sử hệ thống được thử nghiệm N lần, trong đó N_s là số lần robot hoàn thành nhiệm vụ tuần tra – phát hiện – tiếp cận. Tỷ lệ hoàn thành nhiệm vụ được tính:

$$S = \frac{N_s}{N} \times 100\% \quad (7.1)$$

Trong đó, S càng lớn thì độ tin cậy tổng thể của hệ thống càng cao. Nếu S thấp, cần xác định lỗi xảy ra ở bước nào: cảm biến không có dữ liệu, YOLO không phát hiện, TF không chuyển được frame, Nav2 không nhận goal hoặc Mission Manager xử lý sai trạng thái.

Thời gian hoàn thành nhiệm vụ trung bình được xác định theo công thức:

$$\bar{T} = \frac{1}{N} \sum_{i=1}^N T_i \quad (7.2)$$

Trong đó, T_i là thời gian hoàn thành nhiệm vụ ở lần thử thứ i . Đại lượng này phản ánh mức độ nhanh chậm của hệ thống khi thực hiện một chu trình tự hành. Nếu thời gian quá lớn, nguyên nhân có thể đến từ đường đi chưa tối ưu, robot quay nhiều, local planner dao động hoặc target pose không ổn định.

Sai số định vị mục tiêu có thể được đánh giá bằng khoảng cách Euclid giữa vị trí ước lượng $\hat{\mathbf{p}}_i$ và vị trí tham chiếu $\mathbf{p}_i^*$. Với $\hat{\mathbf{p}}_i = (\hat{x}_i, \hat{y}_i)$ và $\mathbf{p}_i^* = (x_i^*, y_i^*)$, ta có:

$$e_i = \sqrt{(\hat{x}_i - x_i^*)^2 + (\hat{y}_i - y_i^*)^2} \quad (7.3)$$

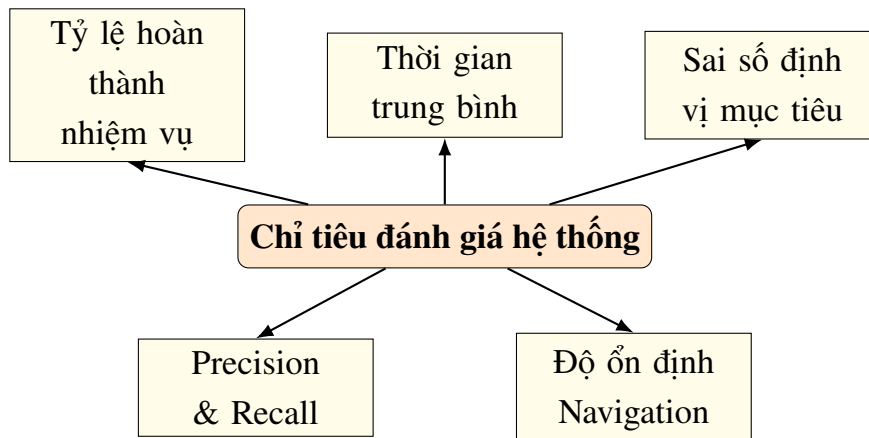
Sai số trung bình qua N lần đo là:

$$\bar{e} = \frac{1}{N} \sum_{i=1}^N e_i \quad (7.4)$$

Nếu $\bar{e}$ lớn, cần kiểm tra ảnh depth, camera intrinsic, phép biến đổi TF từ camera sang map và cách chọn điểm depth trong bbox. Đây là chỉ tiêu quan trọng nếu hệ thống cần tiếp cận mục tiêu chính xác.

Bảng 7.3 Các chỉ tiêu định lượng đề xuất

Chỉ tiêu	Công thức/cách kiểm tra	Ý nghĩa
Tỷ lệ hoàn thành nhiệm vụ	$S = \frac{N_s}{N} \times 100\%$.	Đánh giá độ tin cậy tổng thể của pipeline tự hành.
Thời gian hoàn thành trung bình	$\bar{T} = \frac{1}{N} \sum_{i=1}^N T_i$.	Đánh giá tốc độ thực hiện nhiệm vụ.
Sai số định vị mục tiêu	$e_i = \sqrt{(\hat{x}_i - x_i^*)^2 + (\hat{y}_i - y_i^*)^2}$.	Đánh giá độ chính xác của Object Localization và TF.
Tỷ lệ phát hiện đúng	$\text{Precision} = \frac{TP}{TP + FP}$.	Đánh giá chất lượng nhận diện YOLO.
Tỷ lệ bỏ sót mục tiêu	$\text{Recall} = \frac{TP}{TP + FN}$.	Đánh giá khả năng phát hiện đủ mục tiêu.
Độ ổn định navigation	Quan sát số lần recovery, dao động hoặc goal bị hủy.	Đánh giá chất lượng Nav2 và costmap.

**Hình 7.2 Các chỉ tiêu đánh giá hệ thống robot tự hành TurtleBot4**

7.4 Đánh giá hệ cảm biến

Hệ cảm biến là lớp đầu vào của toàn bộ hệ thống, vì vậy chất lượng dữ liệu cảm biến ảnh hưởng trực tiếp đến SLAM, navigation và định vị mục tiêu. Trong đề tài, các cảm biến chính gồm LiDAR, camera RGB-D, odometry và TF. Mỗi nguồn dữ liệu có ưu điểm và hạn chế riêng, do đó cần đánh giá theo cả khả năng cung cấp dữ liệu và vai trò trong nhiệm vụ tự hành.

7.4.1 *Đánh giá LiDAR*

LiDAR đóng vai trò chính trong bài toán SLAM và xây dựng costmap cho Nav2. Trong mô phỏng, LiDAR publish dữ liệu qua topic `/scan`. Khi topic này ổn định, các node như SLAM Toolbox hoặc local costmap có thể sử dụng khoảng cách đo được để xác định vật cản xung quanh robot.

Ưu điểm của LiDAR là dữ liệu có tính hình học rõ ràng, phù hợp để phát hiện tường, kệ hàng và vật cản trong môi trường warehouse. Tuy nhiên, LiDAR 2D chỉ quan sát trong một mặt phẳng quét, nên các vật thể quá thấp, quá cao hoặc nằm ngoài mặt phẳng quét có thể không được phản ánh đầy đủ trong `/scan`.

7.4.2 *Đánh giá camera RGB-D*

Camera RGB-D OAK-D cung cấp hai loại dữ liệu quan trọng: ảnh màu để nhận diện và ảnh chiều sâu để ước lượng khoảng cách. Trong hệ thống, ảnh RGB là đầu vào cho YOLOv8n, còn ảnh depth cùng `camera_info` được dùng để tính tọa độ 3D của mục tiêu.

Điểm mạnh của camera RGB-D là bổ sung thông tin ngữ nghĩa mà LiDAR không có. Nhờ ảnh RGB, robot có thể biết đối tượng đang quan sát là người hoặc vật thể nào. Nhờ ảnh depth, detection 2D có thể được chuyển thành pose tương đối trong không gian. Hạn chế chính nằm ở chất lượng ảnh depth: nếu mục tiêu bị che khuất, nằm ở biên vật thể hoặc depth bị nhiễu, pose mục tiêu có thể dao động.

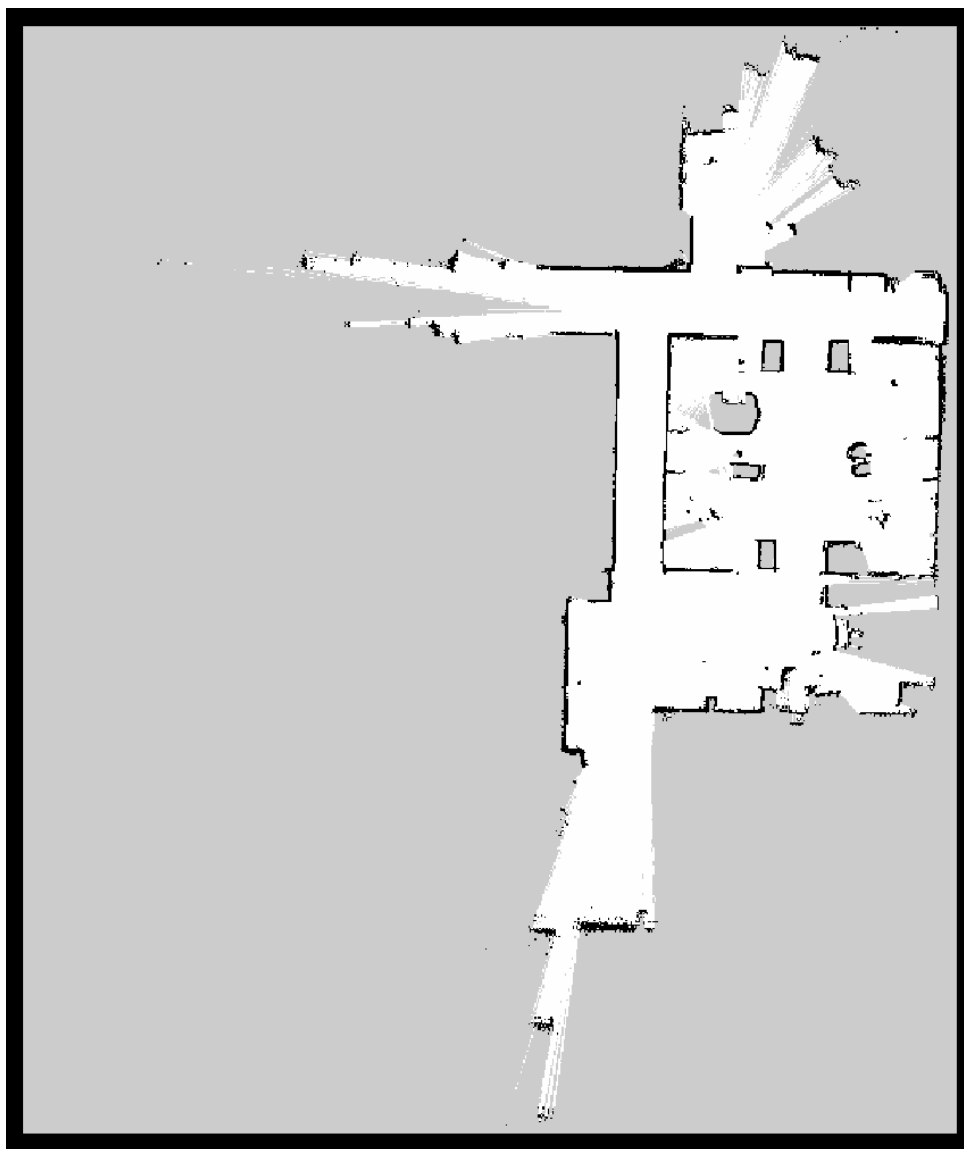
7.4.3 *Đánh giá odometry và TF*

Odometry giúp robot có thông tin chuyển động tương đối liên tục, phục vụ localization và điều khiển. Trong mô phỏng, odometry thường ổn định hơn so với thực tế vì chưa chịu ảnh hưởng đầy đủ của trượt bánh, sai số cơ khí và mặt sàn. TF/TF2 đóng vai trò liên kết các frame như `map`, `odom`, `base_link`, `oakd_link` và `rplidar_link`.

Nếu TF đầy đủ và đồng bộ, dữ liệu từ camera có thể chuyển sang frame `map` để Mission Manager tính goal tiếp cận. Nếu TF thiếu hoặc sai, robot có thể phát hiện mục tiêu nhưng không biết mục tiêu nằm ở đâu trên bản đồ. Vì vậy, đánh giá TF là bước bắt buộc khi kiểm thử hệ thống.

Bảng 7.4 Đánh giá các nguồn dữ liệu cảm biến

Nguồn dữ liệu	Vai trò trong đánh giá	Ưu điểm	Hạn chế cần chú ý
LiDAR /scan	Kiểm tra SLAM, localization và costmap.	Đo vật cản hình học ổn định, phù hợp môi trường trong nhà.	Chỉ quan sát trong mặt phẳng quét 2D.
Camera RGB	Đầu vào cho YOLOv8n.	Cung cấp thông tin ngữ nghĩa về mục tiêu.	Phụ thuộc ánh sáng, góc nhìn và chất lượng mô hình nhận diện.
Ảnh depth	Tính khoảng cách và pose 3D của mục tiêu.	Biến detection 2D thành vị trí không gian.	Dễ nhiễu ở biên vật thể hoặc vùng bị che khuất.
Odometry	Ước lượng chuyển động liên tục của robot.	Tần số cao, hỗ trợ localization.	Có thể tích lũy sai số nếu triển khai trên robot thật.
TF/TF2	Chuyển đổi dữ liệu giữa các frame.	Là điều kiện bắt buộc để hợp nhất cảm biến.	Sai frame hoặc lệch thời gian làm hỏng pose mục tiêu.



Hình 7.3 Minh họa đánh giá bản đồ và dữ liệu cảm biến (sử dụng map thực tế của dự án)

7.5 Đánh giá cơ cấu chấp hành và điều khiển

Cơ cấu chấp hành của TurtleBot4 là truyền động vi sai. Trong hệ thống, các node nhiệm vụ không điều khiển trực tiếp tốc độ từng bánh mà gửi goal cho Nav2. Nav2 sau đó sinh lệnh vận tốc `/cmd_vel` gồm vận tốc tuyến tính v và vận tốc góc ω . Bộ điều khiển cấp thấp chuyển lệnh này thành chuyển động bánh trái và bánh phải.

Kết quả điều khiển được đánh giá thông qua khả năng robot di chuyển đến way-point, chuyển hướng ổn định, tránh vật cản và tiếp cận mục tiêu ở khoảng cách an toàn. Nếu robot đi quá gần mục tiêu, cần kiểm tra tham số `safe_distance` trong Mission Manager. Nếu robot dao động hoặc quay nhiều trước khi tới goal, cần kiểm tra local planner, costmap và cấu hình footprint của robot.

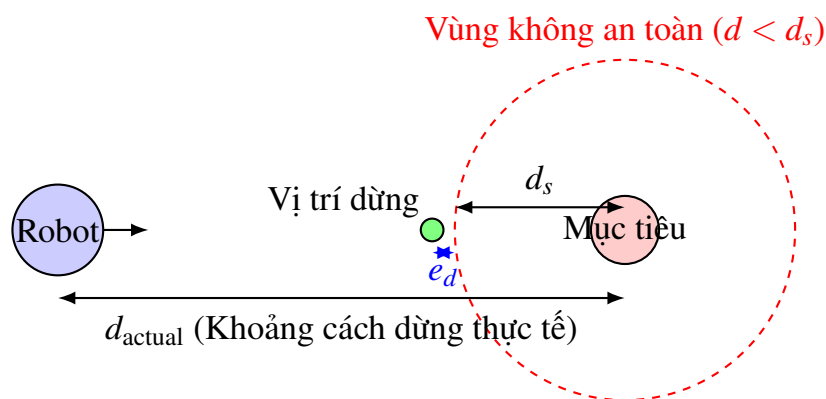
Bảng 7.5 Đánh giá cơ cấu chấp hành và điều khiển

Khía cạnh đánh giá	Dấu hiệu đạt yêu cầu	Dấu hiệu cần cải thiện
Nhận goal	Nav2 chấp nhận goal từ Patrol Node hoặc Mission Manager.	Goal bị từ chối, action server chưa sẵn sàng hoặc frame sai.
Bám đường	Robot di chuyển theo đường đi toàn cục và tránh vật cản.	Robot lệch đường, quay vòng hoặc kẹt gần vật cản.
Lệnh vận tốc	/cmd_vel thay đổi phù hợp với hướng di chuyển.	Lệnh vận tốc giật, đổi dấu liên tục hoặc bằng 0 bất thường.
Tiếp cận mục tiêu	Robot dừng trước mục tiêu với khoảng cách an toàn.	Robot dừng quá xa, quá gần hoặc không hướng về mục tiêu.
Khôi phục lỗi	Nav2 có thể recovery khi gặp tình huống khó.	Robot bị kẹt cục bộ và không thoát được.

Có thể bổ sung sai số dừng tại mục tiêu để đánh giá mức độ chính xác của hành vi tiếp cận. Gọi d_{actual} là khoảng cách robot đến mục tiêu sau khi dừng và d_s là khoảng cách an toàn mong muốn, sai số dừng được tính:

$$e_d = |d_{\text{actual}} - d_s| \quad (7.5)$$

Nếu e_d nhỏ, robot dừng gần đúng vị trí mong muốn. Nếu e_d lớn, cần kiểm tra sai số target pose, logic tính safe goal hoặc độ chính xác của Nav2 khi tới goal.



Hình 7.4 Minh họa robot tiếp cận mục tiêu và sai số dừng tại khoảng cách an toàn d_s

7.6 Đánh giá nhận diện và định vị mục tiêu

Khối nhận diện và định vị mục tiêu là phần tạo điểm khác biệt của hệ thống so với một robot chỉ chạy navigation. YOLOv8n chịu trách nhiệm phát hiện mục tiêu từ ảnh RGB, còn Object Localization sử dụng bbox, ảnh depth và camera_info để tính pose mục tiêu. Vì vậy, chất lượng của bước này phụ thuộc đồng thời vào mô hình nhận diện và dữ liệu chiều sâu.

Đối với YOLOv8n, có thể đánh giá bằng các đại lượng TP , FP và FN . TP là số lần phát hiện đúng mục tiêu, FP là số lần phát hiện sai đối tượng, FN là số lần có mục tiêu nhưng hệ thống không phát hiện. Từ đó có thể tính Precision và Recall:

$$\text{Precision} = \frac{TP}{TP + FP} \quad (7.6)$$

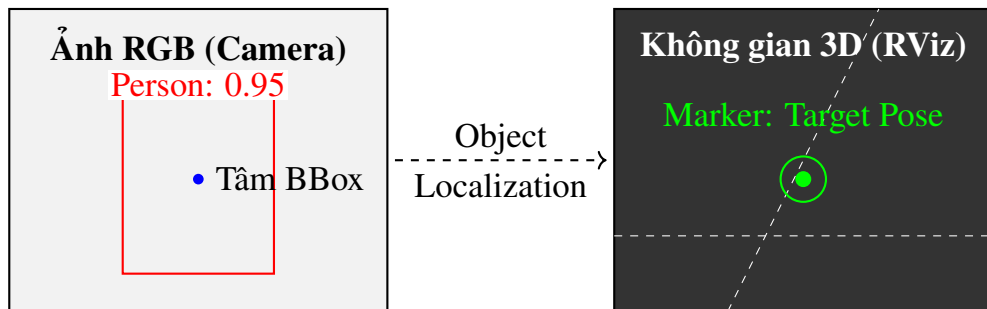
$$\text{Recall} = \frac{TP}{TP + FN} \quad (7.7)$$

Precision cao cho thấy hệ thống ít báo nhầm. Recall cao cho thấy hệ thống ít bỏ sót mục tiêu. Trong bài toán tuần tra và tiếp cận người, recall quan trọng vì robot cần phát hiện được người khi xuất hiện trong vùng quan sát; tuy nhiên precision cũng cần đủ cao để tránh robot phản ứng với đối tượng không mong muốn.

Đối với định vị mục tiêu, cần quan sát độ ổn định của pose mục tiêu. Nếu marker trong RViz dao động mạnh khi mục tiêu đứng yên, có thể nguyên nhân là depth nhiễu, bbox thay đổi giữa các frame hoặc transform TF không đồng bộ. Việc lấy median depth trong vùng gần tâm bbox giúp giảm ảnh hưởng của pixel lỗi so với lấy một điểm depth duy nhất.

Bảng 7.6 Đánh giá nhận diện và định vị mục tiêu

Thành phần	Kết quả cần đánh giá	Cách kiểm tra
YOLOv8n	Phát hiện đúng mục tiêu, confidence đủ cao.	Quan sát /vision/detected_objects và ảnh debug.
Bounding box	BBox bao quanh mục tiêu hợp lý.	So sánh bbox trên ảnh RGB với vị trí thực của mục tiêu.
Depth	Giá trị depth tại bbox hợp lệ và ít nhiễu.	Kiểm tra ảnh depth, loại bỏ NaN hoặc giá trị ngoài khoảng.
CameraInfo	Thông số f_x , f_y , c_x , c_y được nhận đúng.	Kiểm tra topic /camera_info và ma trận nội tại.
Pose mục tiêu	Pose có frame hợp lệ và ít dao động.	Quan sát /target_object_pose_map và marker RViz.



Hình 7.5 Mô phỏng ảnh RGB có bounding box YOLO và marker mục tiêu 3D trên không gian RViz

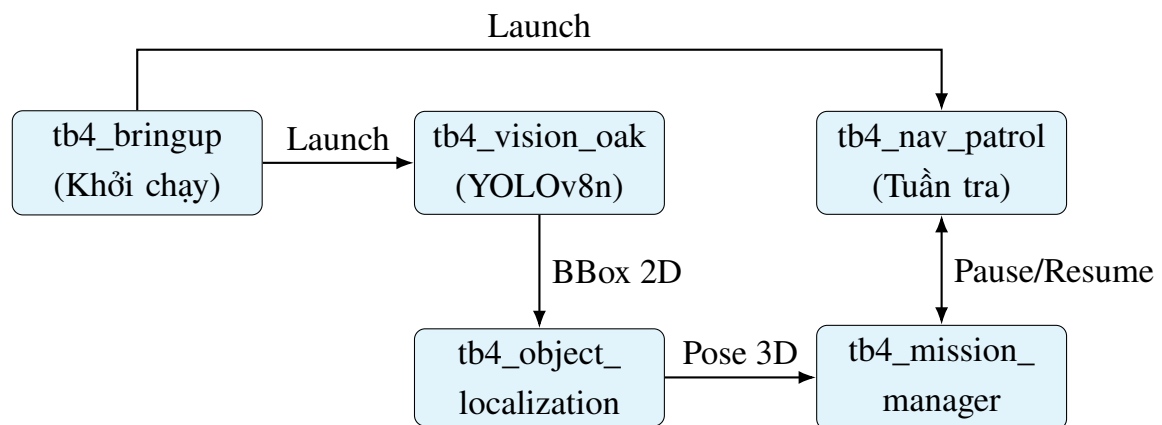
7.7 Đánh giá kiến trúc phần mềm

Kiến trúc phần mềm của hệ thống được tổ chức theo các package ROS2 độc lập. Đây là ưu điểm quan trọng vì mỗi package có nhiệm vụ riêng: bringup khởi chạy hệ thống, vision nhận diện, localization tính pose mục tiêu, patrol gửi waypoint, mission manager điều phối hành vi. Cách tổ chức này giúp việc debug thuận lợi hơn so với viết toàn bộ logic trong một node duy nhất.

Cơ chế publish/subscribe phù hợp với dữ liệu liên tục như ảnh, scan, odometry và pose. Cơ chế action phù hợp với nhiệm vụ kéo dài như NavigateToPose vì navigation không phải là một lệnh tức thời. Tham số ROS2 giúp điều chỉnh confidence threshold, safe_distance hoặc topic name mà không phải sửa trực tiếp trong code.

Bảng 7.7 Đánh giá kiến trúc phần mềm ROS2

Tiêu chí	Đánh giá	Nhận xét
Tính module hóa	Tốt	Các chức năng chính được tách thành package rõ ràng.
Khả năng debug	Tốt	Có thể kiểm tra từng topic và từng node độc lập.
Khả năng mở rộng	Khá tốt	Có thể thay YOLOv8n bằng model custom hoặc thay Patrol Node bằng behavior tree.
Phụ thuộc TF	Cần chú ý	Nhiều khối phụ thuộc vào frame và thời gian transform.
Cấu hình waypoint	Cần cải thiện	Waypoint hiện còn nên đưa ra file YAML để linh hoạt hơn.

**Hình 7.6 Sơ đồ đánh giá tính module hóa các package ROS2 của hệ thống**

7.8 Hạn chế của hệ thống

Mặc dù hệ thống đã đáp ứng mục tiêu mô phỏng, vẫn còn một số hạn chế cần nêu rõ để đánh giá khách quan. Hạn chế lớn nhất là hệ thống chưa được kiểm thử trên TurtleBot4 thật. Môi trường mô phỏng giúp giảm chi phí và rủi ro, nhưng chưa phản ánh đầy đủ nhiều cảm biến, sai số lắp đặt, trượt bánh, giới hạn động cơ và thay đổi ánh sáng ngoài thực tế.

Một hạn chế khác là waypoint tuần tra còn mang tính cố định. Nếu muốn robot hoạt động trong nhiều môi trường khác nhau, danh sách waypoint nên được đưa ra file YAML hoặc giao diện cấu hình. Mission Manager hiện mới là logic trạng thái đơn giản, chưa phải behavior tree hoặc finite state machine đầy đủ, nên chưa xử lý được nhiều tình huống phức tạp như mất mục tiêu, nhiều mục tiêu đồng thời, mục tiêu di chuyển hoặc

vùng cấm.

Bảng 7.8 Hạn chế của hệ thống và hướng khắc phục

Hạn chế	Ảnh hưởng	Hướng khắc phục
Chưa chạy trên robot thật	Chưa đánh giá được sai số phần cứng, độ trễ và nhiễu thực tế.	Triển khai trên TurtleBot4 thật và đo sai số định lượng.
Waypoint còn cố định	Khó thay đổi nhiệm vụ khi đổi môi trường.	Đưa waypoint ra file YAML hoặc giao diện cấu hình.
Mission Manager còn đơn giản	Khó xử lý nhiều trạng thái và ngoại lệ.	Nâng cấp thành FSM hoặc Behavior Tree.
Depth có thể nhiễu	Pose mục tiêu dao động hoặc sai khi mục tiêu bị che khuất.	Lọc theo thời gian, dùng median/Kalman hoặc multi-frame fusion.
Chưa tracking nhiều mục tiêu	Robot chỉ phản ứng với một mục tiêu ưu tiên.	Bổ sung multi-object tracking và lựa chọn mục tiêu theo ngữ cảnh.
Nav2 chưa tối ưu sâu	Robot có thể dao động hoặc kẹt trong môi trường phức tạp.	Tối ưu planner, controller, costmap và footprint.

7.9 Hướng phát triển

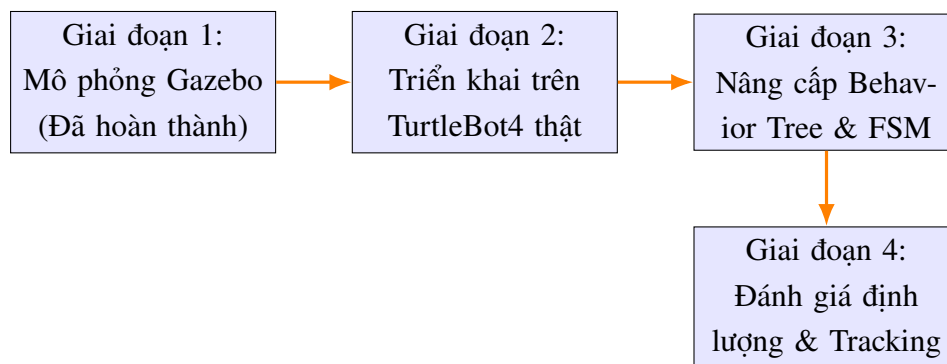
Từ các hạn chế trên, hệ thống có thể phát triển theo ba hướng chính: triển khai thực nghiệm, nâng cấp thuật toán và bổ sung đánh giá định lượng. Hướng đầu tiên là chạy trên TurtleBot4 thật để kiểm tra sai số cảm biến, độ ổn định của odometry, chất lượng camera RGB-D và phản ứng của Nav2 trong môi trường thực. Đây là bước quan trọng để chuyển từ mô phỏng sang ứng dụng thực tế.

Hướng thứ hai là nâng cấp logic nhiệm vụ. Mission Manager có thể được thiết kế lại thành finite state machine hoặc behavior tree. Khi đó robot có thể có nhiều trạng thái rõ ràng hơn như tuần tra, tìm kiếm, phát hiện, xác nhận mục tiêu, tiếp cận, theo dõi, quay lại tuyến tuần tra hoặc xử lý lỗi. Điều này giúp hệ thống dễ mở rộng và dễ kiểm chứng hơn.

Hướng thứ ba là bổ sung logging và tiêu chí định lượng. Thay vì chỉ quan sát bằng mắt trên RViz, hệ thống nên ghi lại thời gian hoàn thành nhiệm vụ, số lần phát hiện đúng, sai số vị trí mục tiêu, số lần Nav2 recovery và độ ổn định của `/cmd_vel`. Các số liệu này giúp phần đánh giá có tính khoa học hơn.

Bảng 7.9 Các hướng phát triển của hệ thống

Hướng phát triển	Nội dung đề xuất	Lợi ích
Triển khai robot thật	Chạy hệ thống trên TurtleBot4 thật trong phòng thí nghiệm.	Đánh giá khả năng ứng dụng thực tế.
Tối ưu Nav2	Tinh chỉnh costmap, planner, controller và recovery.	Giảm dao động, tăng độ mượt và giảm kẹt cục bộ.
YOLO custom	Huấn luyện mô hình cho đối tượng chuyên biệt.	Tăng độ chính xác nhận diện trong bài toán cụ thể.
Tracking mục tiêu	Theo dõi mục tiêu qua nhiều frame.	Giảm dao động pose và xử lý mục tiêu di chuyển.
Behavior Tree/FSM	Thay Mission Manager đơn giản bằng mô hình hành vi rõ ràng hơn.	Dễ mở rộng và kiểm soát luồng nhiệm vụ.
Semantic map	Gắn thông tin ngữ nghĩa vào bản đồ.	Robot hiểu môi trường ở mức cao hơn, hỗ trợ ra quyết định.
Logging định lượng	Ghi dữ liệu và tính chỉ tiêu đánh giá.	Tăng tính thuyết phục cho kết quả thực nghiệm.

**Hình 7.7 Lộ trình phát triển từ mô phỏng sang thực tế và nâng cấp hệ thống**

7.10 Kết luận chương

Chương này đã phân tích kết quả mô phỏng và đánh giá hệ thống robot tự hành TurtleBot4 theo nhiều khía cạnh: cảm biến, nhận diện, định vị mục tiêu, điều hướng, cơ cấu chấp hành, kiến trúc phần mềm, hạn chế và hướng phát triển. Kết quả cho thấy hệ thống đã đạt mục tiêu tích hợp chức năng ở mức mô phỏng: robot có thể thu nhận dữ liệu cảm biến, lập bản đồ, điều hướng, nhận diện mục tiêu, định vị mục tiêu và thực hiện hành vi tiếp cận.

Tuy nhiên, hệ thống vẫn cần được cải thiện ở các điểm như tối ưu Nav2, tăng độ ổn định depth, xử lý nhiều mục tiêu, đưa waypoint ra cấu hình ngoài, nâng cấp Mission Manager và bổ sung đánh giá định lượng. Những nội dung này là cơ sở để phát triển hệ thống thành một robot tự hành hoàn chỉnh hơn trong môi trường thực tế.

8. KẾT LUẬN

Chương 8 là phần tổng kết toàn bộ đề tài nghiên cứu và xây dựng hệ thống robot tự hành TurtleBot4 trong môi trường mô phỏng. Nội dung chương không chỉ nhắc lại kết quả đạt được, mà còn làm rõ ý nghĩa của đề tài đối với học phần *Cảm biến và Cơ cấu chấp hành*, chỉ ra các hạn chế còn tồn tại và định hướng phát triển nếu triển khai tiếp trên robot thật.

Khác với các chương trước tập trung vào cơ sở lý thuyết, kiến trúc, cảm biến, chấp hành, triển khai và đánh giá, Chương 8 có vai trò tổng hợp. Chương này cần thể hiện được mối liên hệ giữa các phần đã trình bày: cảm biến cung cấp dữ liệu, thuật toán xử lý dữ liệu, Nav2 lập kế hoạch và cơ cấu truyền động vi sai thực hiện chuyển động.

8.1 Kết luận chung

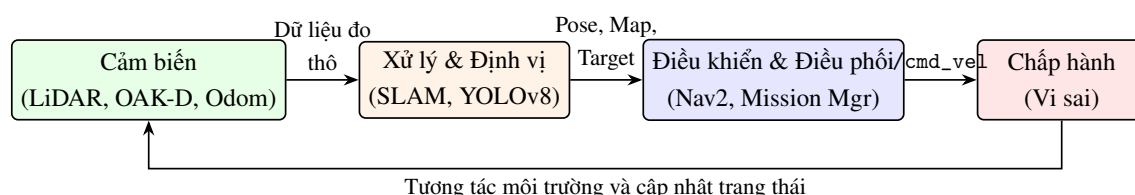
Đề tài đã xây dựng được một hệ thống robot tự hành dựa trên nền tảng TurtleBot4 Lite, ROS2 Humble, Gazebo/Ignition, SLAM Toolbox, Nav2 và YOLOv8n. Hệ thống được triển khai theo hướng mô phỏng, nhưng vẫn phản ánh tương đối đầy đủ cấu trúc của một robot di động tự hành hiện đại: có lớp cảm biến, lớp xử lý – ra quyết định và lớp cơ cấu chấp hành.

Về mặt cảm biến, hệ thống sử dụng LiDAR để đo khoảng cách và phục vụ SLAM/navigation; camera RGB-D OAK-D để nhận ảnh màu, ảnh chiều sâu và hỗ trợ định vị mục tiêu; odometry để ước lượng chuyển động tương đối của robot; TF/TF2 để liên kết các hệ tọa độ. Những dữ liệu này được xử lý trong ROS2 để tạo thành thông tin trạng thái có ý nghĩa cho điều khiển robot.

Về mặt điều khiển và chấp hành, robot sử dụng truyền động vi sai. Nav2 đóng vai trò điều khiển chuyển động cấp cao bằng cách nhận goal, lập đường đi, tính lệnh vận tốc và gửi lệnh đến robot. Mission Manager đóng vai trò điều phối hành vi, quyết định khi nào robot tuần tra, khi nào tạm dừng tuần tra và khi nào tiếp cận mục tiêu.

Bảng 8.1 Tổng kết các nội dung kỹ thuật chính của đề tài

Nội dung tổng kết	Ý nghĩa kỹ thuật
Nền tảng triển khai	ROS2, Gazebo/Ignition, TurtleBot4 Lite, Nav2, SLAM Toolbox và YOLOv8n được kết hợp thành một hệ thống thống nhất.
Cảm biến	LiDAR, RGB-D camera, odometry và TF cung cấp dữ liệu cần thiết cho nhận thức môi trường và định vị.
Xử lý dữ liệu	Dữ liệu cảm biến được chuyển thành bản đồ, pose robot, detection 2D và pose mục tiêu 3D.
Điều khiển	Nav2 chuyển goal thành lệnh vận tốc để robot di chuyển an toàn trong môi trường.
Điều phối nhiệm vụ	Mission Manager kết nối kết quả nhận diện mục tiêu với hành vi tiếp cận và quay lại tuần tra.



Hình 8.1 Sơ đồ tổng quan hệ cảm biến – xử lý – điều khiển – chấp hành của hệ thống TurtleBot4

8.2 Các nội dung đã hoàn thành

Các nội dung đã hoàn thành có thể được nhóm thành bốn hướng chính: nghiên cứu lý thuyết, thiết kế kiến trúc, triển khai chức năng và đánh giá hệ thống. Cách nhóm này giúp phần kết luận không bị liệt kê rời rạc mà thể hiện rõ quá trình phát triển của đề tài từ nền tảng lý thuyết đến hệ thống vận hành.

Bảng 8.2 Các nội dung đã hoàn thành

Nhóm nội dung	Kết quả đã hoàn thành	Vai trò trong đề tài
Nghiên cứu lý thuyết	Trình bày cơ sở về robot tự hành, cảm biến, cơ cấu chấp hành, SLAM, Nav2, YOLOv8 và robot vi sai.	Làm nền tảng để hiểu nguyên lý hoạt động của toàn hệ thống.
Thiết kế kiến trúc	Tổ chức hệ thống theo các package ROS2 như bringup, vision, localization, patrol và mission manager.	Giúp hệ thống dễ mở rộng, dễ debug và có cấu trúc rõ ràng.
Xử lý cảm biến	Tích hợp LiDAR, camera RGB-D, odometry và TF; xử lý detection và depth để tạo pose mục tiêu.	Cung cấp dữ liệu đầu vào cho SLAM, navigation và mission logic.
Điều hướng và chấp hành	Sử dụng Nav2 để gửi goal, lập đường đi, tạo lệnh vận tốc và điều khiển robot vi sai.	Chuyển mục tiêu điều hướng thành chuyển động thực tế của robot.
Nhiệm vụ tự hành	Triển khai logic tuần tra, phát hiện mục tiêu, tiếp cận an toàn và quay lại tuần tra.	Thể hiện khả năng phối hợp giữa perception, localization, planning và control.
Đánh giá hệ thống	Tổng hợp kết quả mô phỏng, nhận xét ưu điểm, hạn chế và hướng phát triển.	Làm cơ sở để cải tiến khi triển khai trên robot thật.

Nhìn tổng thể, đề tài đã hoàn thành mục tiêu ở mức tích hợp chức năng. Robot có thể được mô phỏng, nhận dữ liệu cảm biến, xử lý nhận diện mục tiêu, tính vị trí mục tiêu và gửi nhiệm vụ điều hướng. Đây là kết quả quan trọng vì nó chứng minh các khối phần mềm đã giao tiếp được với nhau theo đúng mô hình của một hệ robot tự hành.

8.3 Ý nghĩa đối với học phần Cảm biến và Cơ cấu chấp hành

Ý nghĩa lớn nhất của đề tài là đặt cảm biến và cơ cấu chấp hành trong một hệ thống robot hoàn chỉnh. Khi học riêng từng loại cảm biến hoặc từng dạng cơ cấu chấp hành, người học có thể hiểu nguyên lý hoạt động của từng phần tử riêng lẻ. Tuy nhiên, trong robot tự hành, giá trị của cảm biến và cơ cấu chấp hành chỉ thể hiện đầy đủ khi chúng được tích hợp trong một vòng kín điều khiển.

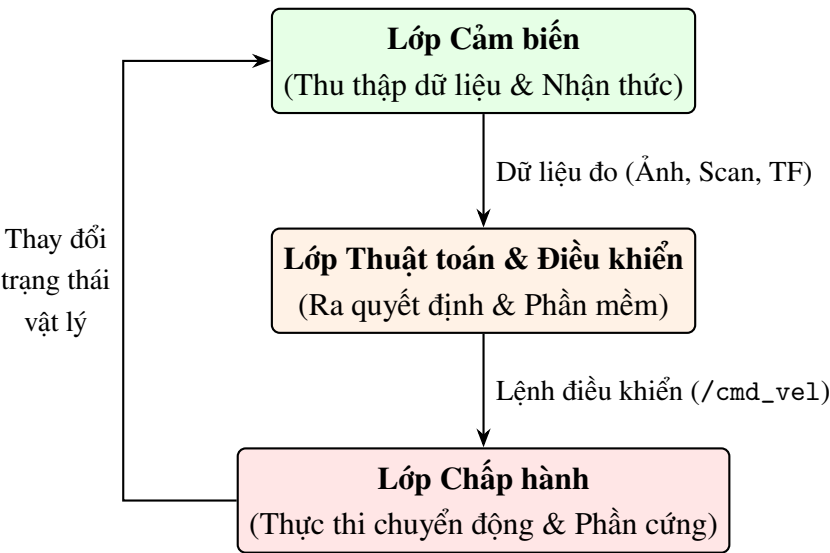
Trong hệ thống này, cảm biến quyết định robot biết gì về môi trường. LiDAR cho biết khoảng cách tới vật cản, camera RGB-D cho biết hình ảnh và chiều sâu, odometry cho biết chuyển động tương đối, TF cho biết quan hệ giữa các frame. Ngược lại, cơ cấu chấp hành quyết định robot có thể làm gì với thông tin đó. Nếu cảm biến phát hiện mục

tiêu nhưng cơ cấu chấp hành không thể thực hiện lệnh điều hướng, robot vẫn không thể hoàn thành nhiệm vụ.

Đề tài cũng giúp làm rõ vai trò trung gian của thuật toán. Dữ liệu cảm biến ban đầu thường chỉ là dữ liệu thô như ảnh, scan hoặc odometry. Các thuật toán như SLAM, YOLOv8, Object Localization và Nav2 biến dữ liệu thô thành bản đồ, pose, goal và lệnh vận tốc. Vì vậy, hệ robot tự hành không chỉ gồm phần cứng mà còn là sự kết hợp chặt chẽ giữa cảm biến, phần mềm và truyền động.

Bảng 8.3 Ý nghĩa của đề tài đối với học phần

Khía cạnh học phần	Biểu hiện trong đề tài	Kiến thức rút ra
Cảm biến	LiDAR, RGB-D camera, odometry, TF.	Cảm biến cung cấp dữ liệu đo và quyết định chất lượng nhận thức môi trường.
Xử lý tín hiệu/dữ liệu	SLAM, YOLOv8, depth processing, transform frame.	Dữ liệu cần được xử lý, lọc, chuyển đổi và đặt trong hệ tọa độ phù hợp.
Cơ cấu chấp hành	Truyền động vi sai, lệnh /cmd_vel, bánh trái và bánh phải.	Lệnh điều khiển phải được biến thành chuyển động vật lý.
Điều khiển hệ thống	Nav2 và Mission Manager.	Robot cần lớp điều khiển cấp cao để lập kế hoạch và lớp điều phối để chọn hành vi.



Hình 8.2 Mối quan hệ giữa cảm biến, thuật toán xử lý và cơ cấu chấp hành trong học phần

8.4 Hạn chế của đề tài

Mặc dù hệ thống đã thể hiện được luồng nhiệm vụ tự hành trong mô phỏng, đề tài vẫn còn một số hạn chế cần được nhìn nhận rõ. Các hạn chế này không làm mất ý nghĩa của đề tài, nhưng là cơ sở để xác định những phần cần cải tiến trong giai đoạn tiếp theo.

Bảng 8.4 Hạn chế của đề tài

Hạn chế	Nguyên nhân	Ảnh hưởng
Chưa triển khai trên robot thật	Đề tài giới hạn trong môi trường mô phỏng Gazebo/Ignition.	Chưa đánh giá được nhiều cảm biến thật, trượt bánh, sai số lắp đặt và độ trễ truyền thông.
Chưa tối ưu sâu Nav2	Planner, controller và costmap mới ở mức cấu hình cơ bản.	Robot có thể chưa đạt chuyển động mượt trong môi trường phức tạp.
Waypoint tuần tra còn đơn giản	Danh sách waypoint được thiết kế cố định cho kịch bản mô phỏng.	Tính linh hoạt khi thay đổi môi trường còn hạn chế.
Mission Manager còn đơn giản	Logic chủ yếu gồm hai trạng thái tuần tra và tiếp cận mục tiêu.	Chưa xử lý được nhiều tình huống như mất mục tiêu, mục tiêu động hoặc nhiều mục tiêu.
Định vị mục tiêu phụ thuộc depth	Pose mục tiêu được tính từ bbox và ảnh depth.	Kết quả có thể bị ảnh hưởng khi depth nhiễu, vật thể bị che khuất hoặc bbox không chính xác.
Chưa có đánh giá định lượng đầy đủ	Chưa xây dựng bộ chỉ số thử nghiệm chi tiết.	Khó so sánh chất lượng hệ thống giữa các lần cấu hình khác nhau.

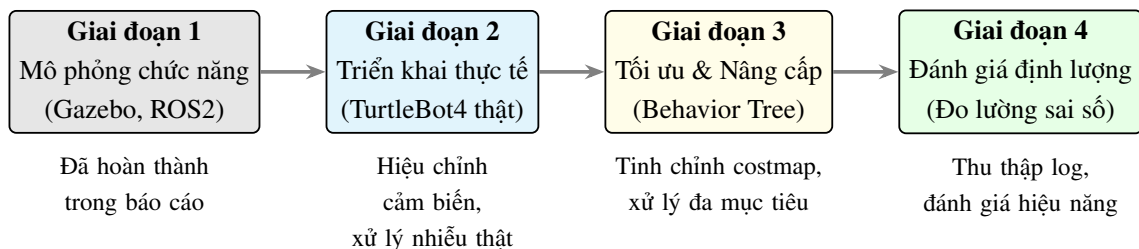
Các hạn chế trên cho thấy sự khác biệt giữa mô phỏng và thực nghiệm. Trong mô phỏng, nhiều yếu tố như ánh sáng, ma sát, sai số cơ khí hoặc độ trễ truyền thông thường được đơn giản hóa. Khi chuyển sang robot thật, các yếu tố này có thể ảnh hưởng đáng kể đến kết quả điều hướng và nhận diện mục tiêu.

8.5 Định hướng phát triển tiếp theo

Từ các hạn chế đã nêu, hệ thống có thể được phát triển theo nhiều hướng. Các hướng phát triển nên được chia thành nhóm phần cứng, nhóm thuật toán, nhóm phần mềm hệ thống và nhóm đánh giá định lượng để dễ triển khai.

Bảng 8.5 Định hướng phát triển tiếp theo

Nhóm phát triển	Hướng cải tiến	Kết quả mong muốn
Triển khai thực nghiệm	Chạy hệ thống trên TurtleBot4 thật, hiệu chỉnh LiDAR, camera, odometry và TF.	Đánh giá được sai số thực tế và độ ổn định khi robot vận hành ngoài mô phỏng.
Tối ưu điều hướng	Tinh chỉnh planner, controller, local costmap, global costmap và recovery behavior của Nav2.	Robot di chuyển mượt hơn, ít dao động hơn và tránh vật cản tốt hơn.
Cải tiến Mission Manager	Xây dựng finite state machine hoặc behavior tree.	Hệ thống có khả năng xử lý nhiều trạng thái phức tạp hơn.
Nâng cấp nhận diện	Huấn luyện YOLO custom cho các đối tượng chuyên biệt.	Tăng độ chính xác nhận diện trong môi trường ứng dụng cụ thể.
Theo dõi mục tiêu	Bổ sung multi-object tracking và lọc vị trí mục tiêu theo thời gian.	Robot theo dõi mục tiêu ổn định hơn, giảm nhiễu pose.
Đánh giá định lượng	Ghi log thời gian tiếp cận, sai số định vị, tỷ lệ phát hiện đúng và tỷ lệ hoàn thành nhiệm vụ.	Có cơ sở khách quan để so sánh và tối ưu hệ thống.

**Hình 8.3 Sơ đồ định hướng phát triển từ mô phỏng sang robot thật**

Trong các hướng trên, triển khai trên robot thật là bước quan trọng nhất nếu muốn kiểm chứng khả năng ứng dụng thực tế. Sau đó, việc tối ưu Nav2, cải tiến Mission Manager và bổ sung đánh giá định lượng sẽ giúp hệ thống từ mức demo chức năng tiến tới mức hệ thống có thể kiểm thử và so sánh một cách khoa học.

8.6 Tổng hợp giá trị đạt được của đề tài

Có thể tổng hợp giá trị của đề tài theo ba nhóm: giá trị học thuật, giá trị kỹ thuật và giá trị thực tiễn. Cách nhìn này giúp phần kết luận có chiều sâu hơn, thay vì chỉ dừng

ở việc liệt kê các công việc đã hoàn thành.

Bảng 8.6 Tổng hợp giá trị đạt được của đề tài

Loại giá trị	Nội dung thể hiện
Giá trị học thuật	Đề tài giúp củng cố kiến thức về cảm biến, cơ cấu chấp hành, robot tự hành, SLAM, navigation và nhận diện đối tượng.
Giá trị kỹ thuật	Đề tài thể hiện quy trình tích hợp nhiều module ROS2 thành một pipeline hoàn chỉnh, gồm perception, localization, planning, mission management và actuation.
Giá trị thực tiễn	Mô hình có thể làm nền tảng cho robot tuần tra kho, robot giám sát, robot dịch vụ hoặc các bài toán tìm kiếm mục tiêu trong môi trường trong nhà.
Giá trị mở rộng	Kiến trúc module hóa cho phép thay thế hoặc nâng cấp từng phần như YOLO, Nav2, Mission Manager hoặc cảm biến mà không phải viết lại toàn bộ hệ thống.

Như vậy, đề tài không chỉ giúp người thực hiện hiểu cách chạy một hệ thống robot trong ROS2 mà còn giúp hình thành tư duy thiết kế hệ thống. Người học cần biết cảm biến tạo dữ liệu gì, dữ liệu đó được xử lý như thế nào, thuật toán ra quyết định ra sao và cơ cấu chấp hành thực hiện quyết định đó bằng cách nào.

8.7 Kết luận cuối cùng

Thông qua đề tài nghiên cứu và xây dựng hệ thống robot tự hành TurtleBot4 trong môi trường mô phỏng, có thể khẳng định rằng mục tiêu cơ bản của báo cáo đã được đáp ứng. Hệ thống đã tích hợp được các thành phần quan trọng của một robot tự hành gồm cảm biến, xử lý dữ liệu, lập bản đồ, điều hướng, nhận diện mục tiêu, điều phối nhiệm vụ và chấp hành chuyển động.

Kết quả đạt được cho thấy mối liên hệ chặt chẽ giữa cảm biến và cơ cấu chấp hành. Cảm biến giúp robot nhận biết môi trường và mục tiêu; thuật toán biến dữ liệu cảm biến thành thông tin điều khiển; cơ cấu chấp hành biến lệnh điều khiển thành chuyển động. Đây chính là nội dung cốt lõi của học phần *Cảm biến và Cơ cấu chấp hành* trong một bài toán robot hiện đại.

Trong tương lai, nếu được triển khai trên robot TurtleBot4 thật, bổ sung đánh giá định lượng và tối ưu các tham số điều hướng, hệ thống có thể trở thành nền tảng tốt cho các ứng dụng robot giám sát, robot tuần tra, robot dịch vụ hoặc nghiên cứu sâu hơn về robot tự hành thông minh.

PHỤ LỤC A: CẤU TRÚC SOURCE CODE

```
1 Robots_Sensor_Actuators/  
2 |-- tb4_bringup/  
3 |   |-- launch/sim_demo.launch.py  
4 |   |-- launch/slam_demo.launch.py  
5 |   |-- config/slam_toolbox_tb4.yaml  
6 |   |-- rviz/tb4_debug.rviz  
7 |   '-- tb4_bringup/initial_pose_publisher.py  
8 |-- tb4_vision_oak/  
9 |   |-- launch/oak_detection.launch.py  
10 |   |-- config/camera_params.yaml  
11 |   |-- models/yolov8n.pt  
12 |   '-- tb4_vision_oak/detection_node.py  
13 |-- tb4_object_localization/  
14 |   |-- launch/localization.launch.py  
15 |   '-- scripts/localization_node.py  
16 |-- tb4_nav_patrol/  
17 |   |-- launch/nav_patrol.launch.py  
18 |   '-- tb4_nav_patrol/patrol_node.py  
19 '-- tb4_mission_manager/  
20     |-- launch/mission_manager.launch.py  
21     '-- tb4_mission_manager/mission_manager_node.py
```

PHỤ LỤC B: CÁC LỆNH CHẠY CHÍNH

Build workspace

```
1 cd ~/tb4_project_ab
2 source /opt/ros/humble/setup.bash
3 colcon build --symlink-install
4 source install/setup.bash
```

Launch simulator

```
1 cd ~/tb4_project_ab
2 export ROS_DOMAIN_ID=7
3 export RMW_FASTRTPS_USE_SHM=0
4 source /opt/ros/humble/setup.bash
5 source install/setup.bash
6
7 ros2 launch turtlebot4_ignition_bringup
  turtlebot4_ignition.launch.py \
8   model:=lite world:=warehouse x:=2.0 y:=0.0 z:=0.01
  yaw:=0.0
```

Chạy full mission

```
1 cd ~/tb4_project_ab
2 export ROS_DOMAIN_ID=7
3 export RMW_FASTRTPS_USE_SHM=0
4 source /opt/ros/humble/setup.bash
5 source install/setup.bash
6
7 ros2 launch tb4_bringup sim_demo.launch.py use_rviz:=true
```

Kiểm tra topic

```
1 ros2 topic echo /scan --once
2 ros2 topic echo /odom --once
3 ros2 topic echo /clock --once
4 ros2 topic echo /vision/detected_objects --once
```

```
5 ros2 topic echo /target_object_pose_map --once
6 ros2 topic echo /target_object_marker --once
```


PHỤ LỤC C: MỘT SỐ LỖI VÀ CÁCH KHẮC PHỤC

Lỗi ROS daemon hoặc session cũ

- Dừng ROS daemon bằng `ros2 daemon stop`.
- Kill các process ROS2/Gazebo cũ nếu còn chạy.
- Xóa cache shared memory nếu cần.
- Source lại môi trường ROS2 và workspace.

Lỗi /scan bất thường

- Kiểm tra topic `/scan` có dữ liệu hợp lệ hay không.
- Kiểm tra frame và vị trí lắp đặt `rplidar_link`.
- Kiểm tra collision giữa LiDAR và robot model.
- Quan sát LaserScan trong RViz để xác định lỗi hình học.

Lỗi không thấy marker mục tiêu

- Kiểm tra `/vision/detected_objects` có detection hay không.
- Kiểm tra ảnh depth và camera info có publish hay không.
- Kiểm tra `/target_object_pose_map` có dữ liệu hay không.
- Trong RViz thêm display Marker với topic `/target_object_marker`.

TÀI LIỆU THAM KHẢO

1. TS. Phạm Hoàng Anh, *Bài giảng học phần Cảm biến và Cơ cấu chấp hành*, Khoa Kỹ thuật Điện tử 1, Học viện Công nghệ Bưu chính Viễn thông.
2. Open Robotics, *ROS2 Documentation*, ROS2 Humble Hawksbill.
3. Open Navigation, *Nav2 Documentation*, Navigation2 project.
4. Slam Toolbox, *ROS2 SLAM Toolbox Documentation*.
5. TurtleBot4, *TurtleBot4 User Manual and Simulation Documentation*, Clearpath Robotics / Open Robotics.
6. Ultralytics, *YOLOv8 Documentation*.
7. R. Siegwart, I. R. Nourbakhsh, D. Scaramuzza, *Introduction to Autonomous Mobile Robots*, MIT Press.
8. B. Siciliano, L. Sciavicco, L. Villani, G. Oriolo, *Robotics: Modelling, Planning and Control*, Springer.